数据结构讲义

qwaszx

2026 年 7 月 25 日

前言

有一句著名的话叫做“程序 = 算法 + 数据结构”。我们使用算法来描述解决问题的步骤，而数据结构提供了**实现步骤**的方法。在学习数据结构时，首先应当明确的事情即是它支持的**操作**：以什么样的数据为输入，允许什么样的修改，能够回答什么样的查询。反过来，面对一个问题时，我们也要能够根据需要的操作选择合适的数据结构。

一般来说，数据结构的功能越强大，其代价也就越大。这种代价可以是多方面的：时空复杂度、实现难度、常数等等。也许 OIer 首次接触到通用、强大的方法就是在学习数据结构之时。在惊叹于强力的通用方法的同时，不应忽视那些功能较弱但代价也更低的方法，更不能放弃对问题本身的分析。挖掘问题的性质以使用更精练的方法，忽略问题的性质而使用强力的通用方法，两者都是处理问题时的重要思想。

目录

第一章 基础数据结构 **5**

1.1 数组 5

1.2 vector 5

1.2.1 基本操作 5

1.2.2 常见问题 5

1.2.3 实现 6

1.3 链表 6

1.3.1 基本操作 6

1.4 栈和队列 6

1.5 双端队列 6

1.5.1 维护信息 7

1.6 二叉堆 7

1.6.1 基本操作 7

1.7 扩展类问题 8

1.8 单调队列和单调栈 9

1.9 作业 11

1.9.1 基础题 11

第二章 静态区间数据结构 **12**

2.1 分治信息 12

2.2 静态区间数据结构 16

2.2.1 幂等律与 ST 表 16

2.2.2 分治与二区间合并 17

2.3 线段树 19

2.4 多叉分治 24

2.4.1 应用：线性 RMQ 24

2.5 * 理论最优的静态区间数据结构 25

第三章 带修区间数据结构 **26**

3.1 单点修改 26

3.2 全局修改 27

3.2.1 全局查询 27

3.2.2 区间查询 28

3.3 区间修改 31

3.4 特殊情况的优化 36

3.4.1 标记永久化 36

3.4.2 增量修改与前缀查询：树状数组 37

3.4.3 可差分的修改 . . . 38

第四章 线段树的进阶操作 40

4.1 zkw 线段树 . . . 40
4.2 动态开点 . . . 42
4.3 线段树上二分 . . . 44
4.4 线段树分裂与合并 . . . 45
4.5 单侧递归 . . . 45
4.6 李超线段树 . . . 45
4.7 Segment Tree Beats . . . 45
4.8 KTT . . . 45

第五章 扫描线与几何模型 46

5.1 平面数点问题 . . . 46
5.1.1 可减信息 . . . 46
5.1.2 不可减信息 . . . 47
5.1.3 转置 . . . 47
5.2 矩形并问题 . . . 47
5.3 常见平面建模 . . . 48
5.4 区间问题 . . . 49
5.4.1 数颜色类问题 . . . 49
5.4.2 mex 类问题 . . . 50
5.4.3 子区间类问题 . . . 51
5.5 维护时间维 . . . 51

第六章 分治 53

6.1 标准序列分治 . . . 53
6.2 非标准序列分治 . . . 54
6.3 其他例子：多项式乘法和矩阵乘法 . . . 55
6.4 偏序数点/CDQ 分治 . . . 56
6.4.1 半在线 . . . 57
6.5 整体二分 . . . 58
6.6 等价类分治 . . . 59

第七章 分块与莫队 61

7.1 序列分块基础 . . . 61
7.2 复杂度平衡 . . . 63
7.3 莫队 . . . 64
7.4 常用技巧 . . . 67
7.4.1 逐块处理 . . . 67
7.4.2 操作分块/定期重构 . . . 67

7.4.3 分块后分治 68

第八章 常见特殊性质 69

8.1 均摊 69
8.2 特殊函数 70
8.3 按阈值分类讨论/阈值分治 71
8.3.1 约数、倍数、取模 71
8.3.2 度数 72
8.3.3 出现次数 73
8.3.4 字符串 73
8.4 位运算 73

第九章 树上问题初步 75

9.1 树的基本性质 75
9.1.1 树的子结构 75
9.2 树的序列化 76
9.2.1 DFS 序 76
9.2.2 BFS 序 76
9.2.3 欧拉序 76
9.2.4 括号序 77
9.3 LCA 的计算 77
9.3.1 向上跳 77
9.3.2 RMQ 77
9.4 树的子结构的性质 78
9.4.1 子树 78
9.4.2 路径/链 78
9.4.3 虚树 78
9.5 树上差分与前缀和 79
9.6 直径 79
9.7 重心 81
9.8 链剖分 83

1 基础数据结构

本章提要

- vector、栈、队列、链表、堆支持的操作，及其实现
- 常见的可以使用这些数据结构解决的问题

提到复杂度时默认指最坏复杂度

1.1 数组

相信大家都会使用常数长度的数组。但是也有开变量长度的数组的方法：`int* a=new int[n]`。通过这种方法，我们可以开每维长度不同的二维数组：

很多编译器也支持直接定义 `a[n]`，但这不是标准的。

```
int pool[1000000]; // 数组充当内存池
int* a[1000]; // 定义一个指针数组充当二维数组
int* cur=pool; // 内存池指针
for (int i=0; i<1000; i++) {
    // a[i] 占用 [cur, cur+L[i]) 这一段空间
    a[i]=cur;
    cur+=L[i];
}
a[i][j]; // 访问 a[i][j]
```

相比 `vector` 套 `vector`，这种方法的内存是连续的，且常数更小。

1.2 `vector`

变长数组。基本可以完全替代数组。

1.2.1 基本操作

- 均摊 $O(1)$ 的末尾插入 `push_back`
- 均摊 $O(1)$ 的末尾删除 `pop_back`
- $O(1)$ 的随机访问[1]`a[i]`
- $O(\text{size}-i)$ 加均摊 $O(1)$ 在第 i 个位置插入删除

参考：https://cppreference.cn/w/cpp/container/vector

1.2.2 常见问题

在使用时，需要注意一些陷阱：

[1]指任意访问某个 `a[i]`

- 大量 `push_back` 会带来很大的常数。可以使用 `resize` 一次性分配需要的长度，这样和数组基本一样快
- 迭代器失效：修改导致长度 `size` 变化后，迭代器可能变得不可用
- `pop_back` 不会释放内存，所以占用空间正比于历史最大 `size` 而不是当前 `size`
- `clear` 不会释放内存。可以用 `vector<T>().swap(a)` 来释放

1.2.3　实现

我们给出一种 `vector` 的实现：如果 `size` 超过了当前容量，那么把当前容量加倍，重新申请空间，把所有东西都搬过去。

每次重新分配后，容量约等于两倍 `size`。触发下一次重新分配需要 $\Omega(\texttt{size})$ 次修改，重新分配的代价是 $O(\texttt{size})$。所以，平均每次修改的代价是 $O(1)$。

问题 1.1. 如果额外要求空间线性于 `size` 而不是历史最大 `size` 呢？

解答. `size` 小于当前容量的 1/4 时，把容量减半并重新分配空间（为什么是 1/4 而不是 1/2?）

1.3　链表

1.3.1　基本操作

链表支持

- $O(1)$ 在指定元素处插入删除单个元素/另一个链表（例如可以实现 $O(1)$ 拼接）
- $O(\text{位置})$ 随机访问
- $O(\texttt{size})$ 的空间

例 1.2. 洛谷 P10466 邻值查找 / 洛谷 P1168 中位数

插入往往需要平衡树之类的数据结构，但是如果倒着来就只需要先排序然后一直删除。如果排序可以线性，那么就能做到线性。

评注. 值域为 $n^{O(1)}$ 的排序可以通过**多关键字计数排序**，或者说**基数排序**来做到线性。

1.4　栈和队列

基本操作是广为人知的，不再赘述。显然，栈在功能上弱于 `vector`，因此可以用 `vector` 实现栈。而队列则不能直接用 `vector` 实现。如果队列大小已知不超过某个上界，那么可以用数组模拟循环队列。而如果想做到 $O(\texttt{size})$ 空间，就需要用链表或者双端队列实现

1.5　双端队列

双端队列在功能上强于 `vector` 和队列，但是实现更复杂，常数也更大。双端队列的支持的操作如下：

- 均摊 $O(1)$ 头尾插入删除
- $O(1)$ 随机访问

注意如果没有随机访问的要求，那么可以直接用链表完成。

STL 的 `deque` 即是标准的双端队列。然而，需要开很多双端队列时，要谨慎使用 STL 的 `deque`，因为一个空 `deque` 可能占用几百到几千字节的空间。

NOI 2022 时有许多人因为 MLE 失去了一整道题的分数。

更好的实现是对顶栈。设想有两个栈，其底部粘在一起。于是其整体就成为一个两端开口的双端队列。唯一的问题出在某个栈空了还要继续删的情况。此时，我们暴力把整个序列倒出来重构即可。换句话说，我们重新把序列均分到两个栈里。

使用 `vector` 来实现栈就能支持随机访问了。如果需要很紧的空间，可以用之前提到的 $O(\texttt{size})$ 空间的 `vector`。

每次重构后，两个 `vector` 都有大概 $\texttt{size}/2$ 个元素。到下次重构之前，至少有 $\texttt{size}/2 + |\texttt{size}' - \texttt{size}/2| = \Omega(\texttt{size}')$ 次操作。重构花费 $O(\texttt{size}')$ 的代价，所以插入删除是均摊 $O(1)$ 的。

1.5.1　维护信息

在两个 `vector` 中各自维护自己的前缀信息，这样可以 $O(1)$ 查询当前双端队列中跨过分断点的任意区间的分治信息。

分治信息的定义和例子将在下一章中给出。

例 1.3. 支持双端队列操作、每次询问以体积 V_i 做 01 背包的结果。需要 $O((n+q)V)$。

假设当前询问区间是 $[l, r]$，那么通过双端队列操作，我们可以 $O(1)$ 移动到 $[l \pm 1, r \pm 1]$。这样，我们就可以通过移动区间端点来在 $\sum_i |l_i - l_{i-1}| + |r_i - r_{i-1}|$ 的时间内回答多个区间询问。在很多问题中，这个总代价往往可以是线性的。

例 1.4. 滑动窗口. 给定序列 a 和整数 k，查询所有 $[i, i+k-1]$ 的分治信息（如最大子段和）。

例 1.5. “不删除双指针”. 询问最短的 $\gcd > 1$ 的区间。$\gcd > 1$ 可以换成 $\max \geq x$、按位或 $\geq x$ 之类的有单调性的条件。

1.6　二叉堆

1.6.1　基本操作

- $O(1)$ 查询最小值
- $O(\log n)$ 插入
- $O(\log n)$ 删除
- $O(\log n)$ Decrease-Key
- $O(n)$ 合并

评注. 理论上最好的堆——*Brodal Queue* 支持

- $O(1)$ 查询最小值

- $O(1)$ 插入
- $O(\log n)$ 删除
- $O(1)$ *Decrease-Key*
- $O(1)$ 合并

但是实现复杂、常数很大。

算法/数据结构总是在复杂度、实际表现、实现难度、常数、功能之间做取舍。

还有其他一些常见的堆：左偏树、斜堆、随机堆、配对堆、二项堆、斐波那契堆。

1.7　扩展类问题

从若干个初始状态开始，每个状态可以扩展出若干个后继状态，保证扩展出来的状态的权值小于原状态的权值。每次取权值最小的状态扩展，重复若干次。

可惜没时间讲。

通过一定的设计，很多求前 k 大/小的问题可以变成这类问题。使用堆来维护所有目前的状态，每次取出最小的一个来扩展，注意去重。扩展形成一张图，复杂度为 $O(M\log M)$ 或 $O(k\log k+M)$（取决于堆），其中 M 为前 k 小的点邻接的**边数**。

如果每个点的扩展方式都是相同的，并且只有常数种扩展方式，并且扩展出来的状态的权值关于原状态的权值是单调的，那么可以使用队列替换堆来做到 $O(M)$。

例 1.6. 序列合并.

解答. 方法一：(i,j) 扩展出 $(i,j+1)$，初始状态为所有 $(i,1)$

方法二：(i,j) 扩展出 $(i+1,j)$ 和 $(i,j+1)$，但是要去重

方法三：https://www.cnblogs.com/zkyJuruo/p/18428124

例 1.7. 蚯蚓.

解答. 首先处理掉“除了新的以外都加 q”，这可以变成全局加 q 然后新的减掉 q。

这样就是 x 扩展出 $\lfloor p(x+tq)\rfloor-(t+1)q$ 和 $(x+tq)-\lfloor p(x+tq)\rfloor-(t+1)q$

符合我们用队列替换堆时的要求

例 1.8. 常数种非负边权的最短路. BFS 扩展时，从 x 扩展出 $O(1)$ 种固定的新状态可以使用队列替换堆。

例 1.9. 前 k 小子集和.

解答. 考虑一个 DFS 过程，枚举下一个选择的数在哪

然而这样会扩展出 $\Theta(n)$ 个状态

多叉树转二叉树：左儿子右兄弟（注意兄弟的顺序）

例 1.10. 限制大小的前 k 小子集和. 要求集合大小在 $[L,R]$ 内。

解答. 考虑怎么枚举 size $= m$ 的集合。假设选了 $a_{i_1}, \ldots, a_{i_m}$，转为枚举 $d_j = i_j - i_{j-1}$，只需要 $\sum_j d_j \le n-1$。

初始所有 $d_j = 1$，每次枚举下一个需要增加的 d_j 即可。

同样用儿子兄弟表示法转为二叉树

评注. 这类问题应该还有很大研究空间，现在知道的有 这篇 和 这篇。有兴趣的同学可以自行研究或者和我一起讨论一下

例 1.11. 合并果子. 由于合并的结果总是越来越大的，可以使用队列替换堆。

评注. 这就是 *Huffman* 编码。我们有结论

- *Huffman* 编码可以最小化 $\sum_i \mathrm{dep}_i \cdot w_i$
- w_i 的深度大约为 $\log \frac{W}{w_i}$，其中 $W = \sum w_i$

在**不均匀分治**（如树剖、点分治、分治 *FFT* 等）中，可以使用 *Huffman* 树来降低复杂度或常数，以后会提到。

1.8　单调队列和单调栈

令 $b_i = \max\{a_i, a_{i+1}, \ldots, a_r\}$，即 b 是只考虑 $a_1, \ldots, a_r$ 时的 a 的后缀 max。我们可以在 r 从 1 扫到 n 的过程中均摊 $O(1)$ 地隐式维护所有 b_i。具体来说，维护一个栈，栈中元素是 a 的一个子序列。假设自底向上的第 k 个元素是 a_{i_k}，那么对任何 $i_{k-1} < j \le i_k$，都有 $b_j = a_{i_k}$，也就是说栈中每个元素都表示一段以它为结尾的区间的 b_i。由于 b 是不升的，栈中元素也是不升的。当 r 移动到 $r+1$ 时，考虑 a_{r+1} 的影响。假设当前栈中有 k 个元素。如果 $a_{r+1} < a_{i_k}$，那么 a_{r+1} 不会对 $b_{[1,r]}$ 造成任何影响，只需要把 a_{r+1} 加入栈中表示 b_{r+1} 即可。如果 $a_{r+1} \ge a_{i_k}$，那么我们可以想象 $b_{[1,r]}$ 发生的变化：一个后缀会全都变成 a_{r+1}。在栈中，这表现为弹出若干栈顶，直到栈顶的 $a_{i_k} > a_{r+1}$，最后再把 a_{r+1} 加入栈中。暴力做这个过程的均摊复杂度就是正确的，因为每个 a_i 最多入栈出栈各一次。

练习 1.12. 把 max 换成 min。

例 1.13. 询问端点单调 RMQ. 多次查询 $\max\{a_{l_i}, \ldots, a_{r_i}\}$，其中 l_i, r_i 都单调不降。可以强制在线（指 a 可能依赖先前的询问结果）

虽然双端队列也可以做，但是这种问题单调队列更方便。

解答. 单调栈中维护了所有的后缀 max，答案就是下标 $\ge l_i$ 的第一个元素。由于 l_i 不降，我们直接弹出队头就可以均摊 $O(1)$ 地维护这个事情。

练习 1.14. 滑动窗口. 对每个 $1 \le l \le n-k+1$ 查询 $\max\{a_i \mid i \in [l, l+k-1]\}$。

练习 1.15. 在例 1.13 的基础上减弱问题的假设。

1. 去掉 l_i 单调的假设，允许询问复杂度变大一些
2. 再通过离线去除 r_i 单调的假设

3. * 再通过可持久化数组支持在线
4. 在 $n \le 64$ 时，通过位运算来存储每个 r 的栈中元素集合，从而实现 $O(n)-O(1)$ 的在线 RMQ。我们将在下一章中配合其他技巧做到任意 n 的线性 RMQ。

例 1.16. 求前驱后继. 对每个元素，查询它左/右边第一个比它大/小的数的位置

解答. 右边就是把序列倒过来的左边，所以只考虑左边。这就是单调栈中倒数第二个元素。

例 1.17. 稍复杂一些的信息维护. 对每个 r，求 $\sum_{l=1}^{r}(r-l+1)\cdot\max\{a_l,\ldots,a_r\}$。强制在线。（提示：要记得单调栈中每个元素都代表了一段区间的 $\max\{a_l,\ldots,a_r\}$。）

解答. 假设栈中元素是 $a_{i_1},a_{i_2},\ldots,a_{i_k}$，其中 $i_1<i_2<\ldots<i_k$，那么答案是

$$\begin{aligned}\sum_{j=1}^{k}a_{i_j}\sum_{i_{j-1}<l\le i_j}(r-l+1)&=\sum_{j=1}^{k}a_{i_j}\cdot(i_j-i_{j-1})\cdot(r-i_j+1)\\&=(r+1)\sum_{j=1}^{k}a_{i_j}-\sum_{j=1}^{k}a_{i_j}(S_{i_j}-S_{i_{j-1}}),\end{aligned}$$

其中 $S_i=\sum_{k=1}^{i}k$。直接维护这个式子即可。

离线的另解. 如果允许离线，那么可以考虑换种方式计算答案：对每个 a_i，看看它是哪些区间 $[l,r]$ 的 max。这可以由先前提到的前驱后继来做。

例 1.18. 最值版本. 求 $\max_{l,r}(r-l+1)\cdot\max\{a_l,\ldots,a_r\}$。

解答. 使用离线算贡献的方法是最容易的

在线对每个 r 算的话需要维护凸包。

例 1.19. 悬线法. 有些题给了个二维黑白网格，要求全黑的矩形/正方形的最大面积/面积和之类的东西

对每个格子计算每个方向全黑能延伸到哪，然后就变成类似例三的问题

练习 1.20. 困难的例子 给定两个序列 a,b，求有多少个区间 $[l,r]$ 满足 $\max a[l,r]=\max b[l,r]$。

解答. a,b 交替形成一个新序列，在这个新序列上扫的时候用单调栈维护所有后缀 max。在维护这个交替序列的后缀 max 的同时，我们可以顺便维护出每个后缀 max 是否同时也是只考虑 a/b 的后缀 max，以及只考虑 a/b 时何时达到此后缀 max。之后容易计算单调栈中每个项的贡献：假设只考虑 a/b 时在原序列的 l_a/l_b 位置达到此后缀 max，则这一项的贡献即为 $\min(l_a,l_b)-\text{pos}$，其中 pos 为此项在原序列中的位置，另外在此项是由 b 产生的时候 pos 需要再加一。

也可以扩展到多个序列。

本章总结

- 熟知讲过的几种线性数据结构的功能，能够把问题与数据结构进行对应
- 扩展类问题的思路

1.9　作业

1.9.1　基础题

2 静态区间数据结构

在本节中，我们将尝试解决一类这样的问题：给定一个长度为 n 的序列 a，以及一个输入为区间 $[l,r]$ 和 a 的函数 f，我们需要进行一定的预处理，使得对于任意的 l,r，我们都可以快速地计算出 $f_{[l,r]}(a)$ 的值。这类问题通常称为**静态区间信息查询**。

2.1 分治信息

我们首先需要确定 f 的一个合适**陪域** S。如果 S 本身就是一个巨大的集合，那么我们自然无法期望得到一个快速的算法。

例 2.1. 假设 a 的所有元素都在集合 A 中。

- $f_{[l,r]}(a)=(a_l,\dots,a_r)$ 是一个平凡的无法快速查询的函数，因为它本身就需要写出所有的值。此时，$S=\{\epsilon\}+A+A\times A+A\times A\times A+\cdots$ 是一个巨大的集合。
  - 假设 a 中的元素都在 $[V]$ 中（不超过 V 的正整数集），那么 $f_{[l,r]}(a)=\sum_{i=l}^{r}a_i$ 的陪域可以是 $[nV]$；而 $f_{[l,r]}(a)=\gcd(a_l,\dots,a_r)$ 和 $f_{[l,r]}(a)=\max(a_l,\dots,a_r)$ 的陪域都可以是 $[V]$。
  - 假设 a 中的元素都在 $\mathbb{F}_p$ 中（也就是 mod p 下做运算），那么 $f_{[l,r]}(a)=\sum_{i=l}^{r}a_i$ 和 $f_{[l,r]}(a)=\prod_{i=l}^{r}a_i$ 的陪域都可以是 $\mathbb{F}_p$。
  - $f_{[l,r]}(a)=a_l\odot a_{l+1}\odot\cdots\odot a_r$ 抽象了以上情况，其中 $\odot$ 是一个定义在 A 上的满足结合律的二元运算。此时，$S=A$。 这是半群
- $f_{[l,r]}(a)=\bigoplus_{[i,j]\subseteq[l,r]}(a_i\odot\cdots\odot a_j)$ 是另一个常见的情况，其中 $\odot$ 同上一条，$\oplus$ 是另一个定义在 A 上的满足交换律和结合律的二元运算（请注意 $\odot$ 和 $\oplus$ 在这里并不暗示分配律）。也就是说，枚举所有的子区间，子区间内做运算 $\odot$，然后再把所有子区间的结果用 $\oplus$ 合并。 但是在下面暗示了。
- $f_{[l,r]}(a)=$ 区间众数的出现次数 是一个虽然 S 很小但也没有 polylog 做法的问题。

评注. 从信息论的角度看，至少需要 $\log|S|/w$ 个变量才能表示一个 S 中的元素。从 *OI* 的角度看，S 中每个元素都是表示成一个结构体的，那么存储 S 中的元素就需要花费结构体中的变量个数那么多的代价。

一般的 f 并不容易快速回答。但是，f 经常有一些好的性质。如果学过一些基本的线段树知识，那么可以发现我们在对询问区间进行切分，使得每个切出来的区间都被预处理过了，然后再用这些预处理的区间的 f 拼出询问区间的 f。我们将此抽象为**可分解性**，这是最重要和常见的性质： 这是 GPT 起的名字

定义 2.2. 可分解性. 如果存在 S 上的二元运算 $\odot$，使得对任何 $l\le m<r$ 都有 $f_{[l,r]}(a)=f_{[l,m]}(a)\odot f_{[m+1,r]}(a)$，则称 f **可分解**。

满足可分解性的 f 一般称为**分治信息**。对于这样的 f，我们可以用子区间的信息来拼出大区间的信息。 然而我也不知道这种有被称为什么的名字，也许这太平凡了

练习 2.3. 半群信息. 假设 f 可分解。

1. 假设 f 的值域为 $T \subseteq S$，证明 $\odot$ 在 T 上满足结合律。
2. 证明 $f_{[l,r]}(a) = f_l(a) \odot f_{l+1}(a) \odot \cdots \odot f_r(a)$。
3. 说明为什么我们可以不失一般性地只考虑形如 $g_{[l,r]}(b) = b_l \odot b_{l+1} \odot \cdots \odot b_r$ 的问题。这也被称为**半群信息**。

练习 2.4. 幺元. 假设存在一个幺元 $0 \in S$（注意这里的 0 是抽象的 0）使得 $x \odot 0 = 0 \odot x = x$。此时，我们可以避免一些边界上的麻烦。例如，我们可以设 $f_\varnothing(a) = 0$，从而在 $m = r$ 时也有 $f_{[l,r]}(a) = f_{[l,m]}(a) \odot f_{[m+1,r]}(a)$。
如果不存在幺元，说明我们总是可以人为补上一个。

下面是一些典型的分治信息的例子。

例 2.5.

- $f_{[l,r]}(a) = \{a_l\} \times \{a_{l+1}\} \times \cdots \times \{a_r\}$ 也是满足结合律的（在同构意义下）！然而，我们显然不想用它来计算。
- $f_{[l,r]}(a) = a_l \odot \cdots \odot a_r$ 显然是可分解的。
- $f_{[l,r]}(a) = \bigoplus_{[i,j]\subseteq[l,r]} (a_i \odot \cdots \odot a_j)$ 并不直接可分解。但是，如果 $\odot$ 和 $\oplus$ 满足分配律 $x \odot (y \oplus z) = (x \odot y) \oplus (x \odot z)$，那么我们可以构造出一个包含 f 的可分解的函数。令

这是环

$$g_{[l,r]}(a) = \begin{pmatrix} f_{[l,r]}(a) \\ fl_{[l,r]}(a) \\ fr_{[l,r]}(a) \\ flr_{[l,r]}(a) \end{pmatrix} = \begin{pmatrix} \bigoplus_{[i,j]\subseteq[l,r]} (a_i \odot \cdots \odot a_j) \\ \bigoplus_{[l,j]\subseteq[l,r]} (a_l \odot \cdots \odot a_j) \\ \bigoplus_{[i,r]\subseteq[l,r]} (a_i \odot \cdots \odot a_r) \\ a_l \odot \cdots \odot a_r \end{pmatrix},$$

找出使得 $g_{[l,r]}(a) = g_{[l,m]} \otimes g_{[m+1,r]}$ 的运算 $\otimes$。（提示：$[l,r]$ 的子区间由三部分组成：$[l,m]$ 的子区间、$[m+1,r]$ 的子区间、跨越 $[m,m+1]$ 的子区间，最后一部分相当于由一个 $[l,m]$ 的后缀拼上一个 $[m+1,r]$ 的前缀。可以将 $(\oplus,\odot)$ 想象成 $(+,\times)$。）

评注. 有许多运算 $(\oplus,\odot)$ 都构成**环**，也就是 $\oplus$ 交换结合、$\odot$ 结合、$\odot$ 对 $\oplus$ 有分配律。常见的例子有 $(+,\times)$, $(\max,+)$, $(\max,\min)$。这里 max 和 min 是等效的。还有很多运算相当于多维的 max 或 min，如位运算 & 和 |，lcm 和 gcd，集合运算 $\cup$ 和 $\cap$。

另一方面，环导出的矩阵乘法也是一个环。所以，我们已经可以处理相当多的运算。

评注. 我们从这个例子中可以看到，虽然原本的 f 无法直接处理，但放到更大的结构 g 中反而就能够处理了。往小了说，这是设计信息时的常见技巧：补充缺少的元素，直到结构变得封闭；往大了说，这是一种深刻的哲学：发展新的概念和理论，让困难的问题变得平凡。

「比起用锤头敲开核桃，不如用海水把它泡软。」

一大类静态区间信息查询问题都可以在找到合适的分治信息后直接使用分治数据结构解决。我们在此非正式地表述寻找分治信息的一般思想。

原则 2.6. 分治信息的设计原则. 我们需要寻找一个可以回答询问 f 的分治信息。

- 如果 f 可以由一些更简单的函数组合出来（如区间方差），那么可以先找到这些更简

单的函数的分治信息，再将其组合。

- 观察 $f_{[l,r]}(a)$ 能否由 $f_{[l,m]}(a)$ 和 $f_{[m+1,r]}(a)$ 组合出来。如果不能，看看还需要哪些东西才能组合出来，把这些东西也加入信息中。
- 重复上一步若干次。如果顺利，那么可以得到一个分治信息。否则，可能说明不存在简单的分治信息。

让我们来做一些具体的练习，看看这个思想是如何使用的。

练习 2.7. 对以下询问，找到合适的能够回答询问的分治信息：

- 区间和 $a_l + \cdots + a_r$
- 区间乘积 $a_l \cdots a_r$，在 $\mathbb{F}_p$ 下做运算（也就是有一个全局固定的素数模数 p）
- 区间最值 $\max(a_l, \ldots, a_r)$
- 区间最大子段和 $\max_{[i,j]\subseteq[l,r]}(a_i + \cdots + a_j)$
- 区间异或和 $a_l \oplus \cdots \oplus a_r$
- 区间的所有子区间异或和的和 $\sum_{[i,j]\subseteq[l,r]}(a_i \oplus \cdots \oplus a_j)$
- 区间方差 $\frac{1}{r-l+1}\sum_{i=l}^{r}(a_i - \bar{a}_{[l,r]})^2$，其中 $\bar{a}_{[l,r]} = \frac{1}{r-l+1}\sum_{i=l}^{r} a_i$。
- 区间 k 次方和 $a_l^k + \cdots + a_r^k$
- 区间位置乘值的和 $\sum_{i=l}^{r} i a_i$
- 区间两两乘积的和 $\sum_{l\le i<j\le r} a_i a_j$
- 区间内选 k 个数的乘积的和 $\sum_{l\le i_1<\cdots<i_k\le r} a_{i_1}\cdots a_{i_k}$
- 区间相邻乘积的和 $\sum_{l\le i<r} a_i a_{i+1}$
- 区间字符串哈希 $\sum_{l\le i\le r} a_i B^{r-i}$
- 区间 gcd：$\gcd(a_l, \ldots, a_r)$
- 区间最长连续 0 长度 $\max_{[i,j]\subseteq[l,r]}(j-i+1)[\forall k \in [i,j], a_k = 0]$
- 区间做括号匹配剩下的不匹配括号
- 区间内有多少个子区间同时包含 $1, 2, \ldots, k$ （可以设计 $O(k)$ 大小的信息和 $O(k)$ 完成的运算）
- 区间众数，但是值域 k 很小

这里的 $\oplus$ 是异或而不是抽象运算。

在实现代码时，一种方便的写法是将 S 的表示封装进结构体，然后重载 S 上的 $+$ 运算符。然后，在写数据结构时，我们用 $+$ 即可完成「信息合并」。通过这种方式，我们**解耦**了数据结构与其维护的信息。

代码 2.1: 信息的封装

```
struct Info {
    // 表示 S 中的一个元素需要的若干变量。我们以最大子段和为例。
    long long s,sl,sr,slr;
    // 默认构造函数：对应幺元 0。如果没有直接的表示，那么可以再加一个
       bool 变量表示。
    Info() : s(-INF), sl(-INF), sr(-INF), slr(0) {}
    // 构造函数，将序列的元素 x 转成 S 中的元素
    Info(int x) : s(x),sl(x),sr(x),slr(x) {}
```

```
8
9       // 重载 + 运算符：实现运算 \odot
10      Info operator + (const Info &a, const Info &b)
11      {
12          Info res;
13          res.s = max(max(a.s, b.s), a.sl + b.sr);
14          res.sl = max(a.sl, a.slr + b.sl);
15          res.sr = max(b.sr, b.slr + a.sr);
16          res.slr = a.slr + b.slr;
17          return res;
18      }
19  };
20
21  // 在数据结构中使用示例：
22  // a[u] = Info(x);
23  // a[u] = a[u << 1] + a[u << 1 | 1];
```

有时，$\odot$ 是**可减**的，也就是已知 $a \odot b$ 和 a 时可以解出 b。此时，我们可以通过求出 $f_{[1,r]}$ 和 $f_{[1,l-1]}$ 来解出 $f_{[l,r]}$。这样的问题可以通过前缀和来做到线性。

评注. 这个概念对应**消去律**。这和**逆元**、**交换律**都是不同的概念。

练习 2.8. 练习 2.7 中，哪些是可逆的？据此得到它们的线性做法。

最后一个有用的性质是**幂等律**，它是说 $a \odot a = a$。典型的满足幂等律的运算是最值 $\max, \min$ 以及它们的多维版本。幂等律没有前面两个性质那么有用。在同时具备结合律的情况下，幂等律允许我们把待查询的区间拆成可重叠的区间，据此减少需要预处理的区间数量。具体来说，我们有

$$\begin{aligned}
\bigodot_{k=l}^{r} a_k &= \left(\bigodot_{k=l}^{i-1} a_k\right) \odot \left(\bigodot_{k=i}^{j} a_k\right) \odot \left(\bigodot_{k=j+1}^{r} a_k\right) \\
&= \left(\bigodot_{k=l}^{i-1} a_k\right) \odot \left(\bigodot_{k=i}^{j} a_k\right) \odot \left(\bigodot_{k=i}^{j} a_k\right) \odot \left(\bigodot_{k=j+1}^{r} a_k\right) \\
&= \left(\left(\bigodot_{k=l}^{i-1} a_k\right) \odot \left(\bigodot_{k=i}^{j} a_k\right)\right) \odot \left(\left(\bigodot_{k=i}^{j} a_k\right) \odot \left(\bigodot_{k=j+1}^{r} a_k\right)\right) \\
&= \left(\bigodot_{k=l}^{j} a_k\right) \odot \left(\bigodot_{k=i}^{r} a_k\right)
\end{aligned}$$

也就是说，我们不光可以用 $f_{[l,i]}(a)$ 和 $f_{[i+1,r]}(a)$ 拼出 $f_{[l,r]}(a)$，还可以用 $f_{[l,j]}(a)$ 和 $f_{[i,r]}(a)$（$j \geq i$）拼出 $f_{[l,r]}(a)$。

2.2　静态区间数据结构

在本节中，我们假设询问都是 $f[l,r]=a_l\odot\cdots\odot a_r$ 的形式，其中 $\odot$ 是有结合律的可以 $O(1)$ 计算的二元运算。我们在练习 2.3 中解释了为什么可以这么做。

我们没有解释为什么可以假设能 $O(1)$ 计算。实际上，如果需要 $O(T)$ 计算，那么最后复杂度往往要再乘一个 T。

2.2.1　幂等律与 ST 表

我们先来展示如何对于额外满足幂等律的 $\odot$ 设计简单的数据结构来回答区间询问，也就是 ST 表（Sparse Table）

翻译之后多出了一个表（Table）。

数据结构 2.9. ST 表.

预处理. 对于所有满足 $1\le i+2^k-1\le n$ 的 i,k，预处理 $st_k[i]=f[i,i+2^k)$。这个递推是容易的：$st_0[i]=f[i]=a_i$，$st_k[i]=st_{k-1}[i]\odot st_{k-1}[i+2^{k-1}]$。

询问. 询问 $f[l,r]$ 时，找到最大的 k 使得 $2^k\le r-l+1$。此时，$[l,l+2^k)$ 和 $(r-2^k,r]$ 可以覆盖整个区间 $[l,r]$。所以 $f[l,r]=st_k[l]\odot st_k[r-2^k+1]$。

现在来看一下复杂度。总共有 $O(n\log n)$ 个预处理的项，所以预处理时间为 $O(n\log n)$。每次查询只需要合并两个预处理的项，故查询复杂度 $O(1)$。对于寻找最大的 $k=\lfloor\log_2(r-l+1)\rfloor$，可以预处理或者使用内置的 `__builtin_ctz`。

这个函数是 GCC 私货。以前是 CCF 系列赛事原则上禁用的。

代码 2.2: ST 表

```
Info st[LOGN][N];
// O(nlogn) 预处理
void build() {
    lg[1] = 0;
    for (int i = 2; i <= n; i++) lg[i] = lg[i >> 1] + 1;

    for (int i = 1; i <= n; i++) st[0][i] = Info(a[i]);
    for (int k = 1; (1 << k) <= n; k++)
        for (int i = 1; i + (1 << k) - 1 <= n; i++)
            st[k][i] = st[k - 1][i] + st[k - 1][i + (1 << (k - 1))];
}
// O(1) 询问
Info query(int l, int r) {
    int k = lg[r - l + 1];  // 也可以用 k=__builtin_ctz(r - l + 1);
    return st[k][l] + st[k][r - (1 << k) + 1];
}
```

评注. 幂等律对于更低复杂度的数据结构设计并无太多帮助。然而，通过幂等律放松限制是一个重要的思想，这在最优化类型的动态规划中有时可以降低复杂度。

练习 2.10. 通过将 $[l,r]$ 拆成 $O(\log n)$ 个 $[p_i,p_i+2^i)$ 的不交并，说明 ST 表也可以在 $O(\log n)$ 的时间内查询不满足幂等律的信息。

练习 2.11. 说明 ST 表支持 $O(\log n)$ 的末尾插入。再拼合双端队列，我们就能支持 $O(\log n)$ 的头尾插入。

2.2.2　分治与二区间合并

现在，我们来考虑只有结合律的情况。一种想法是**分治**：将目前待处理的区间 $[l,r]$ 分为两半 $[l,m]$ 和 $[m+1,r]$，那么询问有三种情况：要么完全在左边，要么完全在右边，要么跨过中点。完全在某一边的情况可以变成子问题递归处理，而对于跨过中点的询问，我们可以将其变成一个 $[l,m]$ 上的后缀询问加上一个 $[m+1,r]$ 上的前缀询问，再用一次 $\odot$ 拼合。于是我们可以考虑预处理每个分治区间的这样的前后缀询问的结果，这样就得到了以下的数据结构：

在 OI 圈中，这个结构经常被叫做“猫树”。这是一个乱起名字的例子，参见此博客。

数据结构 2.12. 二区间合并.

预处理. 从 $[1,n]$ 开始分治。假设当前的分治区间为 $[l,r]$，设其中点为 $m=\lfloor (l+r)/2\rfloor$。如果 $l=r$，那么结束；否则，预处理所有 $[l,m]$ 的后缀询问的结果以及所有 $[m+1,r]$ 的前缀询问的结果，然后递归处理 $[l,m]$ 和 $[m+1,r]$。

查询. 对于查询 $[lq,rq]$，从 $[1,n]$ 开始分治。假设当前的分治区间是 $[l,r]$，中点为 $m=\lfloor (l+r)/2\rfloor$。如果 $l=r$，那么直接返回 a_l；如果 $[lq,rq]$ 完全在 $[l,m]$ 或者 $[m+1,r]$，则递归下去处理；否则，用预处理的前后缀计算 $f[lq,m]\odot f[m+1,rq]$ 并返回。

现在来看一下复杂度。设长为 n 的分治区间需要花费 $T(n)$ 的时空预处理。那么左右两个子问题各自花费 $T(n/2)$ 的时空处理，预处理前后缀询问花费 $O(n)$ 时空，所以总的时空复杂度是 $T(n)=2T(n/2)+O(n)=O(n\log n)$。对于询问，代价是 $Q(n)=Q(n/2)+O(1)=O(\log n)$。

评注. 这个数据结构在询问时只使用了一次 $\odot$。在这个限制下，$\Omega(n\log n)$ 的预处理复杂度是必要的。

另外，我们注意到询问的瓶颈事实上在于找到那个可以用前后缀回答的分治区间。我们将在练习中看到这个过程可以优化到 $O(1)$。

代码 2.3: 二区间合并

```
Info a[N];
// 每一层分治对应一层数组
Info pre[LOGN][N], suf[LOGN][N];
void build(int l, int r, int dep) {
    if (l == r) return;
    int m = (l + r) >> 1;
    suf[dep][m] = a[m];
    for (int i = m - 1; i >= l; i--)
```

```
        suf[dep][i] = a[i] + suf[dep][i + 1];   //注意：加号（实际代表 \
            odot）未必有交换律，所以两边不能反过来
    pre[dep][m + 1] = a[m + 1];
    for (int i = m + 2; i <= r; i++)
        pre[dep][i] = pre[dep][i - 1] + a[i];
    build(l, m, dep + 1);
    build(m + 1, r, dep + 1);
}
// 查询 [lq, rq]
Info query(int l, int r, int lq, int rq, int dep) {
    if (lq <= l && r <= rq) {
        if (l == r) return a[l];
    }
    int m = (l + r) >> 1;
    if (rq <= m) return query(l, m, lq, rq, dep + 1);
    else if (lq > m) return query(m + 1, r, lq, rq, dep + 1);
    else return suf[dep][lq] + pre[dep][rq];
}
Info query(int lq, int rq) {
    return query(1, n, lq, rq, 0);
}
```

树与区间划分

上面这个代码并没有那么显然，它隐式地使用了一个事实：“一层”内的区间是不交的。为了更直观的理解这个事实，我们来考察区间划分与树的联系。对每个分治区间，我们建立一个结点在分治中，一个分治区间会被划分为两个子区间，我们就把这两个子区间对应的结点作为它的孩子。这样，我们会得到一个树结构。图 2.1 和 图 2.2 展示了这两个结构，它们都可以帮助我们直观地考察数据结构。一般来说，我们会在区间划分结构上思考问题，而分治树结构则只用于提醒我们这是一棵树。

一些资料显示结点是比节点更规范的表述，虽然实际上并没有人在意。在英文中，它们都是 node。

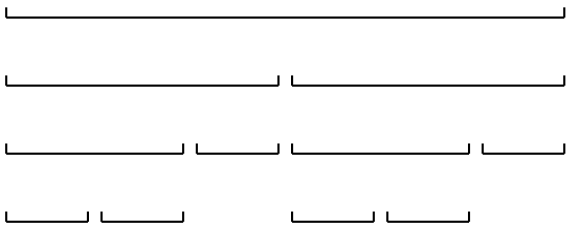


图 2.1: 区间划分

我们立刻可以从图中得到以下的结论：

性质 2.13. 分治树的性质.

- 一层中的区间是不交的。
- 叶子未必具有相同的高度。

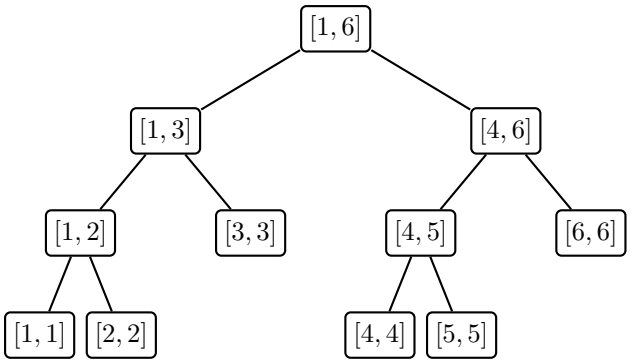


图 2.2: 分治树

- 所有结点的区间长度之和不超过 $n \times$ 高度。
- 两个叶子的 LCA 对应了它们被划分到两侧的那个分治区间。

练习 2.14. 组合数学复习. 如果允许任意选择分治点（而不是固定为中点），那么有多少种可能的分治树？

下面的练习将询问的复杂度优化到 $O(1)$。

练习 2.15. 优化二区间合并的询问复杂度.

1. 假设你有一个 $O(n)$ 预处理、$O(1)$ 查询 LCA 的算法。在这种情况下，说明询问复杂度可以达到 $O(1)$。
2. 即使没有这样的算法，我们也可以实现 $O(1)$ 查询。首先，在序列末尾填充一些 0 来将序列长度补齐为 2^k。这样我们总是可以假设 n 是 2 的幂，并且不改变预处理复杂度。说明对于这样的 n，两个叶子的 LCA 可以 $O(1)$ 计算，从而可以做到 $O(1)$ 询问。尝试用代码实现这个算法。
   - 提示一
   - 提示二
   - 提示三

这种算法真的存在，不过也许有点麻烦。

2.3　线段树

在2.12 中，我们通过预处理的方式回答了前后缀询问。那么，如果我们直接通过递归来回答前后缀询问呢？不妨考虑 $[l,r]$ 的前缀询问 $[l,i]$。如果 $i \le m$，那么直接递归 $[l,m]$；否则，可以用 $f[l,m] \odot f[m+1,i]$，而后者是 $[m+1,r]$ 的一个前缀。我们预处理所有**完整区间**的 $f[l,r]$，询问时通过递归回答。这就得到了线段树。

数据结构 2.16. 线段树.

预处理. 从 $[1,n]$ 开始分治。假设当前的分治区间为 $[l,r]$，设其中点为 $m = \lfloor (l+r)/2 \rfloor$。如果 $l = r$，那么结束；否则，递归处理 $[l,m]$ 和 $[m+1,r]$，然后用 $f[l,r] = f[l,m] \odot f[m+1,r]$

计算出本区间的结果。

查询. 对于查询 $[lq, rq]$，从 $[1, n]$ 开始分治。假设当前的分治区间是 $[l, r]$，中点为 $m = \lfloor (l+r)/2 \rfloor$。如果 $[lq, rq] = [l, r]$，那么直接返回预处理的 $f[l, r]$；如果 $[lq, rq]$ 完全在 $[l, m]$ 或者 $[m+1, r]$，则递归下去处理；否则，递归 `query(l,m,lq,m)` 和 `query(m+1,r,m+1,rq)` 来算出 $f[lq, m]$ 和 $f[m+1, rq]$，然后返回 $f[lq, m] \odot f[m+1, rq]$。

现在来看一下复杂度。预处理的复杂度是 $T(n) = 2T(n/2) + O(1) = O(n)$，而询问的复杂度初看似乎是 $Q(n) = 2Q(n/2) + O(1) = O(n)$。然而，根据我们先前的分析，在第一次两侧递归后，两个询问就都变成了前/后缀询问。而对于前/后缀询问，只有一侧需要递归计算。所以复杂度是 $Q(n) = Q(n/2) + O(1) = O(\log n)$。

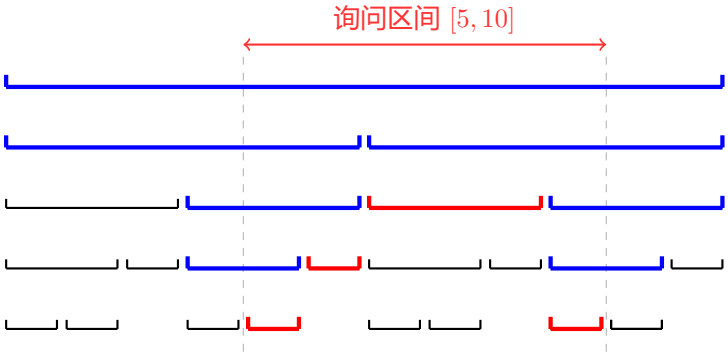


图 2.3: 线段树上定位区间

图 2.3 展示了在线段树上定位 $[5, 10]$ 的情况。红色的区间代表 $[lq, rq] = [l, r]$ 的终止区间，蓝色的区间代表需要到两侧递归询问的区间。结合上面对询问的分治过程的分析，我们可以得到以下定理：

定理 2.17. 区间在线段树上的分解. 设线段树树高为 $h \geq 3$。对任何询问区间，其在线段树上的分治过程中至多会访问 $2h - 3$ 个蓝色区间（需要进一步分治的区间）和 $2h - 4$ 个红色区间（可以直接返回的区间）。

评注. *常数 -3 和 -4 其实并不重要。但是系数 2 是重要的。*

证明. 先考虑前/后缀询问的情况。除了最后一层，每层至多产生一个蓝色区间；除了第一层，每层至多产生一个红色区间。所以任何前/后缀会访问不超过 $h - 1$ 个蓝色区间和 $h - 1$ 个红色区间。

现在考虑一般的询问区间。在其分解为前后缀询问之前，每层恰好访问一个蓝色区间。所以最坏情况是在第一层就分解为前后缀询问，此时总共访问的蓝色区间数量不超过 $1 + 2(h - 2)$，红色区间数量不超过 $2(h - 2)$。 □

使用上面的定理时需要知道树高。这是容易看出的：

性质 2.18. 线段树的树高. 按照 $m = \lfloor (l+r)/2 \rfloor$ 划分区间为 $[l, m]$ 和 $[m+1, r]$ 得到的线段树高度为 $h(n) = h(\lceil n/2 \rceil) + 1 = 1 + \lceil \log_2 n \rceil$。

评注. 这里暗示了不均匀划分的线段树的存在。虽然不均匀划分一般没什么用，但是会有人拿来出题。

在实现线段树的代码之前，我们需要思考一个问题：如何方便地表示所有预处理的 $f[l,r]$？自然的想法是，对分治树的每个结点，在这个结点上记录其对应的区间的 f。那么问题就转换为了如何存储这棵树。一般的做法当然是指针式的，即每个结点记录其左右儿子结点的编号（或者干脆就是指针）。不过在这里，我们可以类似二叉堆来编号，即让结点 u 的左右儿子分别编号为 $2u$ 和 $2u+1$。图 2.2 中的分治树的结点编号如 图 2.4 所示。这种编号方式称为**堆式编号**。

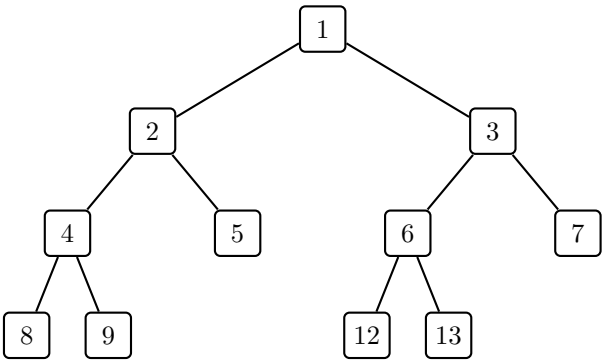


图 2.4: 图 2.2 中分治树的编号

有两个事情需要注意：第一，最后一层也就是叶子层的编号并不是连续的；第二，最大的编号可以达到 $4n-O(1)$ 级别，我们会在练习中仔细计算这个值。

代码 2.4: 线段树

```
INFO tree[N<<2];
void build(int u, int l, int r) {
    if (l == r) {
        tree[u] = a[l];
        return;
    }
    int mid = (l + r) >> 1;
    build(u << 1, l, mid);
    build(u << 1 | 1, mid + 1, r);
    tree[u] = tree[u << 1] + tree[u << 1 | 1];
}
Info query(int u, int l, int r, int lq, int rq) {
    if (l==lq && r==rq) {
        return tree[u];
    }
    int mid = (l + r) >> 1;
    if (rq <= mid) return query(u << 1, l, mid, lq, rq);
    else if(lq > mid) return query(u << 1 | 1, mid + 1, r, lq, rq);
    else return query(u<<1,l,mid,lq,mid)+query(u<<1|1,mid+1,r,mid+1,rq);
}
```

```
21 Info query(int l, int r){
22     return query(1, 1, n, l, r);
23 }
```

评注. 事实上，线段树可以认为是在描述分治结构。在本讲义中，我们有时会混用“线段树”和“分治树”，即将线段树视为一种结构的描述，而非专门的数据结构。

练习 2.19. 堆式标号线段树的最大标号. 考虑 n 个点的堆式编号线段树，计算其结点的最大编号。提示

评注. *P7122* 是一个相关题目，不过要麻烦许多。

我觉得这种包装很无趣。

练习 2.20. 51nod 1792. 定义 $\mathrm{cost}(i,j)$ 为在线段树上分解 $[i,j]$ 时访问的结点数量，询问 $\sum_{[i,j]\subseteq[l,r]} \mathrm{cost}(i,j)$。
类似的题目是洛谷 P5210。

练习 2.21. 线段树的动态开点. 在本练习中，我们说明可以对长度巨大但是“有意义的位置”很少的序列在合理的时空复杂度内构建线段树。形式化地说，假设序列长度为 n，其中只有 m 个位置上的元素被特殊指定，其余位置上的元素由某种默认模式给出，如默认置为某个固定元素 x_0。对于一个不含特殊指定元素的区间，需要能够在 $O(1)$ 时间内计算这个区间的 $f[l,r]$。

1. 堆式编号还能用吗？应该如何存储树？
2. 在 `build` 中，如果当前区间 $[l,r]$ 不含特殊指定元素，那么可以直接计算此区间的 $f[l,r]$ 作为 `tree[u]` 并返回。事实上，甚至不需要建立这个区间对应的结点。基于这一观察，计算需要建立的结点的数量级。
3. 如何判断当前区间是否含有特殊指定元素？提示
4. 修改 `build` 和 `query`，使得 `build` 的时空复杂度为 $O(m\log n)$，并且 `query` 能在和原来相同的时间内回答询问。

评注. 可以通过压缩 *Trie* 进一步把空间降为 $O(m)$，但是一般没有必要。

评注. 在下一章中，我们会通过单点修改的方式完成相同的目标。

练习 2.22. 分治树的末尾插入. 在本练习中，我们给出一种从左到右动态构建分治树的算法，从而让二区间合并和线段树都支持末尾插入。
想象我们有完整的长为 $n=2^k$ 的序列，然后在其上建立分治树。而实际上，在任意时刻，我们只拥有这个序列的一个前缀。在分治树上，这个前缀对应“左下角”的一些部分。具体来说，如果一个结点对应的区间中的元素都已经被插入，那么我们就称这个结点是完整的。我们的算法思想非常简单：在末尾插入一个元素后，把分治树上所有因为这个插入变得完整的结点构建出来。

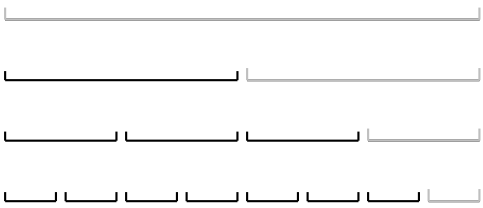


图 2.5: 序列前缀在分治树上

1. 假设目前给出了序列的前 m 个元素。说明我们相当于在长为 $2^{\lceil \log_2 m \rceil}$ 的序列形成的分治树上操作。
2. 末尾插入 a_i 后，哪些结点会变得完整？
3. 说明这个算法在插入 q 次后花费的总时间不超过 $T(2^{\lceil \log_2 q \rceil})$，其中 $T(m)$ 是对长为 m 的静态序列构建数据结构所需的时间。所以，在通常的情况下（也就是 $T(m) = m^{O(1)}$），每次插入的均摊复杂度不超过 $O(T(q)/q)$。分别对于二区间合并和线段树计算这个复杂度。
4. 说明不完整结点永远不会作为 `query` 中 $[l, r] = [lq, rq]$ 的结点。因此，`query` 几乎不需要任何改动即可回答询问（只需要修改存储树的方式）。

评注. 另一个理解这个算法的视角是**二进制分组**：考虑目前序列长度 m 的二进制拆分，$2^{k_1} + 2^{k_2} + \cdots + 2^{k_t}$（$k_i$ 递减），那么可以视为我们维护了 t 个大小指数递减的分治树，第 i 个分治树对应的区间是 $[2^{k_1} + \cdots + 2^{k_{i-1}} + 1, 2^{k_1} + \cdots + 2^{k_i}]$。在末尾插入时，我们首先新增一棵大小为 1 的线段树，然后暴力做“进位”操作：只要有两棵树的大小相同，那么就将其合并得到两倍大小的线段树。

练习 2.23. 分治树的末尾插入删除. 本练习在 练习 2.22 的基础上给出一种支持末尾插入删除的算法。这个算法可以配合双端队列实现双端的插入删除。

1. 说明通过反复在某一个特定位置删除插入可以使得 练习 2.22 中的算法复杂度变劣。如果构建一个结点需要 $\Theta(1)$ 时间，那么均摊复杂度将劣化至 $\Theta(\log q)$；如果构建一个结点需要 Θ(区间长度) 时间，那么均摊复杂度将劣化至 $\Theta(q)$。
2. 改变创建非叶子的完整结点的时刻：对于一个非叶结点 u，当它**右边**的结点变得完整时才构建 u；当已构建结点变得不完整时，摧毁此结点。为什么这样就能避免前面的问题？
3. 分析线段树一层内的结点的总构建次数与操作次数 q 的关系。据此说明这个构建算法的均摊复杂度不超过 $O(T(q)/q)$。
4. 修改询问函数，使其能正确组合**已构建**的结点的信息来回答询问，并且复杂度不变。提示

评注. 也可以通过二进制分组的视角得到类似的算法。我们允许每种大小的分治树有**两个**，当出现第三个相同大小的分治树时，合并前两个。

评注. *TODO* 插入单元素容易但合并信息复杂。

2.4　多叉分治

在前面，我们用的都是二叉分治，也就是把序列每次分成两半。实际上，我们也可以分成更多叉。比如，我们可以把序列分成 k 个长为 n/k 的子区间，然后递归每个子区间。然后，我们考虑一个询问在这种分治中会被如何划分。有两种情况：完全在某个子区间内，或者跨过两个或更多子区间。后者又进一步分成三个部分：左侧的后缀、右侧的前缀、中间的完整区间。对于前/后缀的询问，我们可以选择像二区间合并一样预处理，也可以选择像线段树一样递归，递归下去会再一次产生一个子区间内的前/后缀询问以及若干个完整区间的询问。

这不是分块吗？

我们注意这里的子问题结构。每个子区间当然是子问题，而**完整区间的询问**实际上也是子问题。这是因为我们可以计算出每个完整区间的 $f[l,r]$，然后问题就变成以这些 $f[l,r]$ 为元素的序列上的区间询问。所以，长为 n 的问题会分出来 k 个大小为 n/k 的（块内）子问题以及一个大小为 k 的（块间）子问题。

评注. 如果选择了预处理前后缀询问，那么一个询问就只会在若干次子区间内递归后被拆成前缀 + 若干完整区间 + 后缀的形式直接回答。如果再结合*练习 2.15*中的技巧，我们还可以进一步省掉子区间内递归的过程。

我们可以对于两种子问题使用不同的数据结构，并选择是否预处理前后缀询问，然后组合出各种各样的数据结构。在实践中，最常使用的组件是**暴力**。我们有两种暴力：一种暴力是 $O(n^2)$ 预处理 $O(1)$ 查询的，一种是 $O(n)$ 预处理 $O(n)$ 查询的。我们用“$O(f(n))-O(g(n))$”来表示 $O(f(n))$ 预处理、$O(g(n))$ 查询。让我们来看一些例子：

例 2.24. 假设初始序列长度为 N。待处理的子问题的大小则用 n 表示。

- 递归处理大小为 n/k 的块内子问题，使用 $O(k)-O(k)$ 的暴力处理大小为 k 的块间子问题。分别取 $k=2$，$\log N$，观察得到的数据结构的预处理和查询复杂度。
- 递归处理大小为 n/k 的块内子问题，使用 $O(k^2)-O(1)$ 的暴力处理大小为 k 的块间子问题。取 $k=\sqrt{n}$，观察得到的数据结构的预处理和查询复杂度。
- 使用 $O(n/k)-O(n/k)$ 的暴力处理大小为 n/k 的块内子问题，使用线段树处理大小为 k 的块间子问题。取 $k=n/\log n$，观察得到的数据结构的时间和**空间**复杂度（不计算存储序列 a 需要的空间）。另外，这个结构的常数也比直接上线段树更小。这是被称为**底层分块**的技巧。
- 使用 $O(n/k)-O(n/k)$ 的暴力处理大小为 n/k 的块内子问题，使用 $O(k)-O(k)$ 的暴力处理大小为 k 的块间子问题。取 $k=\sqrt{n}$，观察得到的数据结构的预处理和查询复杂度。

2.4.1　应用：线性 RMQ

令 $k=\Theta(n/\log n)$，使用任意 $O(k\log k)-O(1)$ 的数据结构处理大小为 k 的块间子问题，使用**某种** $O(n/k)-O(1)$ 的数据结构处理大小为 $n/k=\log n$ 的块内子问题。这样，我们就能做到 $O(n)-O(1)$ 的 RMQ。然而，为什么块内的线性复杂度的数据结构可以存在呢？事实上，这非常依赖问题的性质。我们将看到 RMQ 具有这样的性质。

我们先简要介绍“Four Russians”方法的思想。对于 RMQ 来说，我们通过笛卡尔树可以将一般的 RMQ 变成一种特殊的 RMQ——±1 RMQ，也就是在一个相邻项之差总为 ±1 的序列上做的 RMQ。此时，长为 k 的序列的差分序列只有 2^{k-1} 种，而 RMQ 的结果只由差分序列即可确定。所以，取 $k = \frac{1}{2}\log n$，我们可以**打表**所有长度不超过 k 的差分序列上的所有 RMQ 结果，这部分预处理的复杂度是 $\tilde{O}(2^k) = \tilde{O}(\sqrt{n}) = o(n)$ 的。这样，我们在按 k 分块后就可以对每个块在 $O(k)$ 时间内识别出这个块的差分序列在预处理的表中的位置，查表即可 $O(1)$ 回答询问。一般来说，如果存在一个不太小的函数 $f(n)$ 使得长度为 $f(n)$ 的序列只有 $o(n)$ 种本质不同的结果，并且可以在线性时间内找出一个序列对应的结果，就可以使用这种方法来做到 $O(n) - O(1)$。例如，01 序列上的所有询问都满足这个条件。

俗称四毛子。

不过，我们还有一种更简单的方法在 $n \le w$ 的时候实现 $O(n) - O(1)$ RMQ。参见练习 1.15 。

2.5　* 理论最优的静态区间数据结构

在前面的多叉分治结构中，我们预处理前后缀，在块内子问题上递归，在块间子问题上选择另一个数据结构，并且用 LCA 定位来消去递归到分裂区间的时间。这样，我们可以实现 $T(n) = kT(n/k) + T'(k) + \Theta(n)$ 的预处理和 $Q(n) = \Theta(1) + Q'(k)$ 的询问复杂度。那么二区间合并就是令 $k = 2$，此时块间子问题不存在，$T_0(n) = \Theta(n \log n)$，$Q_0(n) = \Theta(1)$。接下来，我们令 $k = n/\log n$，并且让块间子问题使用二区间合并。这样我们得到 $T_1(n) = (n/\log n)T_1(\log n) + T_0(n/\log n) + \Theta(n)$，$Q_1(n) = \Theta(1) + Q_0(n)$。展开递归式可以发现此时 $T_1(n) = \Theta(n \log^* n)$，$Q_1(n) = \Theta(1)$。然后，我们再把这个数据结构作为下一个数据结构的块间数据结构，取 $k = \log^* n$，就能得到 $T_2(n) = \Theta(n \log^{**} n)$，$Q_2(n) = \Theta(1)$。如此嵌套下去，我们能得到一个理论最优的数据结构。当然，具体还有许多取整和常数上的细节需要处理，但是大体的思路即是如此。这样，我们可以得到 $O(n\alpha(n)) - O(\alpha(n))$ 的静态区间数据结构。事实上，这个已经是下界了。不过，在实际的数据范围下，这个做法只有理论上的意义。

本章总结

- 分治信息的概念与设计原则：首先将复杂的信息分解为相对简洁的信息，再从简洁的信息开始逐步添加合并信息所需的辅助信息，直到信息能够合并。
- 静态区间数据结构的设计方法：倍增（ST 表）与分治（二区间合并、线段树、多叉分治、一般的分治结构）
- 线段树的结构与区间分解过程

3 带修区间数据结构

本章中，我们考虑**带修区间信息查询**，也就是在上一章中的问题的基础上，允许对序列 a 的修改。我们仍然假设需要查询的函数 $f_{[l,r]}(a)$ 具有可分解性，即存在二元运算 $\odot$ 使得 $f_{[l,r]}(a) = f_l(a) \odot \cdots \odot f_r(a)$。此外，我们还假设 $f_{[l,r]}(a)$ 只与 $a_{[l,r]}$ 有关。

3.1 单点修改

假设 a_t 发生了变化，那么我们需要在先前的静态数据结构中更新所有包含 t 的预处理区间 $[l,r]$ 的 f。如果这样的区间太多，我们就无法支持快速的修改。在分治树上看，a_t 的变化会导致从根到叶子 $[t,t]$ 这条链上所有的结点的预处理区间的 f 更新。如果预处理区间的数量至少是结点对应的区间长度级别，那么我们就无法支持快速的修改。所以，我们不能预处理前后缀询问。现在的子问题结构如下：

- 对于询问，首先以块内子问题递归若干次，然后分裂成一个块内前缀询问、一个块间询问、一个块内后缀询问。
- 对于前/后缀询问，每层都会分裂出一个块间询问以及一个块内前/后缀询问。
- 对于修改，每次是一个块内子问题，以及更新这个块的信息导致的一个块间子问题。

在一般情况下，这个子问题结构会导致 $T(n) = \min_k T(n/k) + T(k) + 1$ 的复杂度，而这是 $\Theta(\log n)$ 的。所以，线段树已经足够优秀。但有时，修改序列中的一个元素后，我们可以 $O(1)$ 更新完整区间的 f。此时，可以调整 k 来让修改/查询中的一个复杂度降低、另一个复杂度提高。

评注. 事实上，这里有一个下界。假设表示半群元素需要 δ 个 *bit*，*[Pătraşcu, Demaine '04]* 证明了单点修改、查询前缀和有 $t_u \log_2 \left(2 + \frac{w}{\delta} + \frac{t_q}{t_u}\right) = \Omega(\log n)$ 和 $t_q \log_2 \left(2 + \frac{w}{\delta} + \frac{t_u}{t_q}\right) = \Omega(\log n)$ 的下界，且允许均摊与随机化。比如说，在 $\delta = \Omega(w)$ 的时候，如果想要实现 $t_u = O(\log n)$ 则必须有 $t_q = \Omega(\log n)$，而如果想要实现 $t_u = O(1)$ 则必须有 $t_q = n^{\Omega(1)}$。

例 3.1. 多叉线段树. TODO

线段树实现单点修改非常容易。

代码 3.1: 线段树单点修改

```
void update(int u, int l, int r, int q, Info val) {
    if(l == r) {
        tree[u] = val;
        return;
    }
    int m = (l + r) >> 1;
    if(q <= m) update(u << 1, l, m, q, val);
    else update(u << 1 | 1, m + 1, r, q, val);
    tree[u] = tree[u << 1] + tree[u << 1 | 1];
}
```

对于线段树来说，静态查询和单点修改几乎是等难的。我们现在可以完成一大批只需要单点修改的问题。

练习 3.2. 网格图连通性. 给一个 $m \times n$ 的黑白网格，m 很小。支持

- 反转一个格子的颜色
- 查询两个点是否在同一个同色连通块中
- 查询一个子矩形内的同色连通块个数

P4121 P3300 P4246 都是类似的问题。P5897 最短路也可以类似地做（但是转移需要聪明一些）。

3.2　全局修改

在讨论区间修改之前，我们先讨论一个更弱的问题，即全局修改。单点修改总是可以视为赋值。而对于批量修改，我们首先需要对修改进行刻画，以尝试提升计算效率。

定义 3.3. 修改. 假设序列元素都在集合 A 中，所有可能的序列集合记作 A^*。一个修改是一个 $A^* \to A^*$ 的**变换**。所有 $A^* \to A^*$ 的变换的一个子集 $\mathcal{M}$ 刻画了允许的修改操作。

什么花里胡哨的，这就是函数。

显然，一般的修改可以非常复杂，这样的修改通常也没法处理。下面是一些没那么复杂的例子：

例 3.4.

1. 加法：$\Phi_c(a) = (a_1 + c, \ldots, a_n + c)$
2. 和 x 取 max：$\Phi_x(a) = (\max(a_1, x), \ldots, \max(a_n, x))$
3. 加等差数列：$\Phi_d(a) = (a_1, a_2 + d, \ldots, a_n + (n-1)d)$
4. 翻转：$\Phi(a) = (a_n, \ldots, a_1)$
5. 排序：$\Phi(a) = a$ 的升序排列
6. 做前缀和：$\Phi(a) = (a_1, a_1 + a_2, \ldots, a_1 + \cdots + a_n)$

我们到目前为止讨论的数据结构都是假设序列结构不发生改变的，这类数据结构能处理的基本上只有**原地修改**，也就是每个 a_i 的修改只依赖于 a_i 本身，而不依赖于其他位置的元素。形式化地说，这就是 $\Phi(a) = (\phi_1(a_1), \ldots, \phi_n(a_n))$。原地修改的一个常见的例子是**一致修改**，也就是 $\phi_1 = \cdots = \phi_n$ 的情况。

练习 3.5. 上面的例子中，哪些是原地修改甚至一致修改？写出对应的 ϕ_i。

3.2.1　全局查询

现在，我们考虑一下如何回答全局查询。我们希望全局修改 Φ 可以“整体”地应用到 f 上。具体来说，也就是存在某个 S 上的变换 τ_Φ，使得 $f(\Phi(a)) = \tau_\Phi(f(a))$。这样，我们就不需要实际地做修改 $a \leftarrow \Phi(a)$ 再计算 f，而是直接把 τ_Φ 应用到 $f(a_1, \ldots, a_n)$ 上。

紧接着的问题是，如何处理多次修改？我们有

$$\begin{aligned} & f(\Phi_2(\Phi_1(a))) \\ &= \tau_{\Phi_2}(f(\Phi_1(a))) \\ &= \tau_{\Phi_2}(\tau_{\Phi_1}(f(a))) \end{aligned}$$

所以只要逐个应用 τ 即可。

例 3.6. 矩阵乘法. 假设 $\mathbf{a}_i$ 都是 k 维向量，修改是矩阵乘法 $\Phi_{\mathbf{M}}(\mathbf{a}) = (\mathbf{M}\mathbf{a}_1, \ldots, \mathbf{M}\mathbf{a}_n)$，查询是 $f(\mathbf{a}) = \sum_i \mathbf{a}_i$。

3.2.2　区间查询

现在考虑如何回答区间查询。自然的想法是，我们寻找一个能把全局修改“落到”区间上的方法。也就是说，我们寻找 $\tau_{\Phi,[l,r]}$ 使得 $f_{[l,r]}(\Phi(a)) = \tau_{\Phi,[l,r]}(f_{[l,r]}(a))$。这样，我们求出 $f_{[l,r]}(a)$ 后应用 $\tau_{\Phi,[l,r]}$ 即可。

然而我们有多次修改，逐个计算 τ 并应用的代价过高。所以，我们希望首先复合出 $\Phi = \Phi_m \circ \cdots \circ \Phi_1$，然后再计算 $\tau_{\Phi,[l,r]}$。这一过程中，我们需要如下性质：

- 修改集合 $\mathcal{M}$ 在复合下的闭包 $\overline{\mathcal{M}}$ 不太大，且 $\overline{\mathcal{M}}$ 中的任意两个修改 Φ_1, Φ_2 都能够快速复合
- 对任何 $\Phi \in \overline{\mathcal{M}}$，都可以快速计算 Φ 对 $f_{[l,r]}(a)$ 的影响 $\tau_{\Phi,[l,r]}(f_{[l,r]}(a))$.

满足这种性质的 $\overline{\mathcal{M}}$ 中的元素通常也叫做**标记**，它使用较少的信息描述了一个批量修改。

评注. 注意第二个性质与修改 Φ 和信息 f 都相关。有时，无法直接计算 Φ 对 f 的影响，但是当我们把 f 放到一个更大的信息 g 中时，Φ 对 g 的影响也许就会变得容易计算。

一个常见的做法是把 $[l,r]$ 或者 $r-l+1$ 放到 $f_{[l,r]}(a)$ 中，这样就可以把 $[l,r]$ 从 τ 的参数中去掉，也就是我们只需要考虑 τ_Φ。

评注. 用矩阵乘法来描述修改几乎总是方便的，因为矩阵乘法自动是封闭的。这里矩阵乘法可以是广义运算意义下的，如 $(\max, +)$、$(\max, \min)$ 等。

在实现时，由于修改对应的矩阵乘法可能有特殊的性质，如上三角等。此时，可以只存储矩阵的非零元，并且手工计算这种矩阵乘法来加快效率。例如，计算 2×2 矩阵乘法需要 8 次乘法和 4 次加法，而计算上三角矩阵乘法只需要 4 次乘法和 1 次加法。

例 3.7. 线段树 2. 支持全局加、全局乘，查询区间和。在 $\mathbb{Z}_m$ 或者说模 m 意义下做运算

解答. 这是一个一致修改，我们可以先关注每个元素上的变换。$\mathcal{M}$ 包含所有的 $x \mapsto x + b$ 以及 $x \mapsto kx$，这两种变换的复合是一次函数 $x \mapsto kx + b$。由于一次函数的复合还是一次函数，我们就得到了 $\mathcal{M}$ 的闭包 $\overline{\mathcal{M}}$ 的结构：所有逐元素的一次函数变换。

现在，考虑一个 $\overline{\mathcal{M}}$ 中的一个变换 Φ，假设其执行了逐元素的 $x \mapsto kx + b$。我们来寻找

$\tau_{\Phi,[l,r]}$，这就是

$$\begin{aligned} f_{[l,r]}(\Phi(a)) &= \sum_{i=l}^{r} ka_i + b \\ &= \left(k\sum_{i=l}^{r} a_i\right) + (r-l+1)b \\ &= kf_{[l,r]}(a) + (r-l+1)b. \end{aligned}$$

所以我们就得到 $\tau_{\Phi,[l,r]}(f_{[l,r]}(a)) = kf_{[l,r]}(a) + (r-l+1)b$。

让我们关注另一个视角。如果我们把区间长度 $r-l+1$ 也加入到 $f_{[l,r]}(a)$ 中，也就是 $f_{[l,r]}(a) = \begin{bmatrix} \sum_{i=l}^{r} a_i \\ r-l+1 \end{bmatrix}$，那么我们就可以把 τ 写成

$$\tau_{\Phi,[l,r]} f_{[l,r]}(a) = \begin{bmatrix} k & b \\ 0 & 1 \end{bmatrix} f_{[l,r]}(a).$$

这样 τ 和 $[l,r]$ 变得无关了。

例 3.8. 维护两个序列 a 和 b，支持

- 全局让 a_i 或者 b_i 乘 x 或加 x
- 全局让 a_i 变成 $a_i + b_i x$
- 全局让 b_i 变成 $b_i + a_i x$
- 查询区间 a_i 的和以及 b_i 的和

都在 $\mathbb{Z}_m$ 或者说模 m 意义下做运算。

解答. 直接思考比较困难。然而，我们很容易用矩阵乘法描述所有的修改，这是因为所有的修改都是**仿射变换**：

$$\begin{bmatrix} a_i \\ b_i \end{bmatrix} \mapsto \begin{bmatrix} \alpha & \beta \\ \gamma & \delta \end{bmatrix} \begin{bmatrix} a_i \\ b_i \end{bmatrix} + \begin{bmatrix} u \\ v \end{bmatrix}.$$

那么这就是一个矩阵意义下的一次函数变换。

评注. 现在，我们可以让每个 a_i 都代表平面上的一个点，支持全局关于某个点旋转、缩放、平移，查询区间内的点的坐标和。

练习 3.9. OI 中经常出现的情况是下面这样：给定序列 a 以及显式或隐式给定的序列 b（如 $b_i = \mathrm{Fib}_i$），支持全局 a_i 加 x 或乘 x 或让 a_i 变成 $a_i + b_i x$，查询区间内 a_i 的和。

例 3.10. 维护两个序列 a 和 b，支持

- 全局让 a_i 或者 b_i 乘 x 或加 x
- 查询区间内 $a_i b_i$ 的和

都在 $\mathbb{Z}_m$ 或者说模 m 意义下做运算。

解答. 根据线段树 2 的例子，这里的变换还是一次函数。我们考虑如何计算 τ。有

$$\begin{aligned} f_{[l,r]}(\Phi(a)) &= \sum_{i=l}^{r}(k_a a_i + d_a)(k_b b_i + d_b) \\ &= k_a k_b \sum_{i=l}^{r} a_i b_i + k_a d_b \sum_{i=l}^{r} a_i + d_a k_b \sum_{i=l}^{r} b_i + (r-l+1) d_a d_b. \end{aligned}$$

所以，我们需要 $\sum_{i=l}^{r} a_i b_i$、$\sum_{i=l}^{r} a_i$、$\sum_{i=l}^{r} b_i$、$r-l+1$ 这四个信息来计算 $\tau_{\Phi,[l,r]}$。那么自然的选择是把这四个信息都加入到 $f_{[l,r]}(a)$ 中。之后我们还需要考虑 $\sum_{i=l}^{r} a_i$、$\sum_{i=l}^{r} b_i$、$r-l+1$ 这三个量在一次函数变换下的变化，这是在线段树 2 中已经解决过的问题。

练习 3.11. 支持以上三个问题中的所有修改和询问。

例 3.12. 维护序列 a，支持全局加 x，全局和 x 取 max，全局赋值为 x，查询区间 max。x 可以为负值。

解答. 使用 $(\max, +)$ 矩阵乘法来描述修改。max 对应标准矩乘中的加法，$+$ 对应标准矩乘中的乘法。注意到修改也是类似仿射变换的形式：$x \mapsto \max(x+\alpha, \beta)$。将 a_i 扩充一个 0 来表示常数项，那么修改就可以用如下矩阵乘法来描述：加 x 对应 $\begin{bmatrix} a_i \\ 0 \end{bmatrix} \mapsto \begin{bmatrix} x & -\infty \\ -\infty & 0 \end{bmatrix} \begin{bmatrix} a_i \\ 0 \end{bmatrix}$，和 x 取 max 对应 $\begin{bmatrix} a_i \\ 0 \end{bmatrix} \mapsto \begin{bmatrix} 0 & x \\ -\infty & -\infty \end{bmatrix} \begin{bmatrix} a_i \\ 0 \end{bmatrix}$，赋值为 x 对应 $\begin{bmatrix} a_i \\ 0 \end{bmatrix} \mapsto \begin{bmatrix} -\infty & x \\ -\infty & -\infty \end{bmatrix} \begin{bmatrix} a_i \\ 0 \end{bmatrix}$。查询区间 max 就是 $(\max, +)$ 意义下的向量加法。

评注. 通过 *Segment Tree Beats* 技巧，还可以支持查询区间和。这个技巧依赖和 x 取 max 这个操作的均摊性质。

这是吉如一 (jiru_2) 发现的技巧，所以也有吉司机线段树、吉利线段树等名字。正文中这个名字来自番剧 Angel Beats!

在下一节中，我们将看到以上这些问题都可以简单地扩展到区间修改。现在，让我们来总结一下标记与信息设计的一般原则。

原则 3.13. 标记与信息的设计原则.

- 首先观察修改在复合运算下的闭包 $\overline{\mathcal{M}}$ 的结构。这通常可以由矩阵刻画。
- 根据 2.6 初步设计分治信息 f。
- 取一个 $\overline{\mathcal{M}}$ 中的一般修改 Φ，尝试计算 $\tau_{\Phi,[l,r]}$，把需要的辅助信息添加进分治信息 f。
- 重复上一步若干次。如果顺利，那么可以得到一个 τ。否则，可能说明不存在简单的可以快速修改的分治信息。

在实现代码时，方便的做法是把修改的闭包 $\overline{\mathcal{M}}$ 中的元素封装成一个结构体 `Tag`，并实现修改的复合以及修改对信息的影响 $\tau_\Phi(f)$。

代码 3.2: 修改的封装

```
1 struct Tag {
2     // 表示 M 中的一个元素需要的若干变量。我们以线段树 II 为例。
3     long long k, b;
```

```
    // 默认构造函数：对应恒等变换 x->x。如果没有直接的表示，那么可以再加一个 bool 变量表示。
    Tag() : k(1), b(0) {}
    // 构造函数
    Tag(long long k_, long long b_) : k(k_), b(b_) {}

    // 重载 Tag(Tag) 运算符：实现修改的复合。也可以用其他运算符表示，如 *。
    Tag operator ()(const Tag &tag)
    {
        return Tag(k * tag.k, k * tag.b + b.b);
    }
    // 重载 Tag(Info) 运算符：实现 \tau_\Phi(f)，也就是修改如何作用到信息上的函数。也可以用其他运算符表示，如 *。
    Info operator()(const Info &info)
    {
        return Info(k * info.sum + b * info.len, info.len);
    }
};
```

3.3　区间修改

现在，我们来讨论最后的问题，即区间修改。相比全局修改 Φ，区间修改 $\Phi_{[l,r]}$ 额外传入了一个区间作为参数。在 OI 的常规语境下，这代表只对 $a_{[l,r]}$ 进行修改。此处仍然只讨论原地修改，常见的情况有以下三类：

- 一致修改：$\Phi_{[l,r]}(a)$ 将每个 $i\in[l,r]$ 的 a_i 替换为 $\phi(a_i)$
- 与相对位置有关的修改：$\Phi_{[l,r]}(a)$ 将每个 $i\in[l,r]$ 的 a_i 替换为 $\phi_{i-l+1}(a_i)$
- 与绝对位置有关的修改：$\Phi_{[l,r]}(a)$ 将每个 $i\in[l,r]$ 的 a_i 替换为 $\phi_i(a_i)$

与询问相同，我们希望修改也具有可分解性。也就是说当对 $[l,r]$ 进行修改时，我们希望能够将修改分解到 $[l,m]$ 和 $[m+1,r]$ 上。形式化地说，对于修改 $\Phi_{[l,r]}$，我们希望存在 $\Phi_{[l,m]}$ 和 $\Phi_{[m+1,r]}$ 使得 $f_{[l,r]}(\Phi_{[l,r]}(a))=f_{[l,m]}(\Phi_{[l,m]}(a))\odot f_{[m+1,r]}(\Phi_{[m+1,r]}(a))$。

例 3.14. 一致修改和与绝对位置有关的修改显然可分解。对于与相对位置有关的修改，通常需要 ϕ_i 容易平移，也就是 $(\phi_{m+1},\dots,\phi_r)$ 容易用 $(\phi_1,\dots,\phi_{r-m})$ 表示出来。例如，$\phi_i(x)=x+(ki+b)$ 和 $\phi_i(x)=x+kr^i$ 就满足这样的性质。而 $\phi_i(x)=x+b_i$ 则不满足这样的性质。

现在，与静态区间询问类似，我们可以在分治树上递归，把修改 $\Phi_{[L,R]}$ 分解到若干分治区间上，在这些区间上计算 τ 并应用，然后再重算分解过程中影响到的所有区间的 f。考虑分治树上的一个区间 $[l,r]$，有四种情况：

- $[l,r]$ 是 $[L,R]$ 分解到的一个完整区间。那么我们通过 τ 算出了修改后的这个区间的 f。
- $[l,r]$ 是 $[L,R]$ 分解到的一个完整区间的真子区间。那么虽然我们还没有计算 $\Phi_{[L,R]}$ 对这个区间的 f 的影响，但是我们可以保证这个区间的 f 应用上其分治树上祖先结点存

储的修改后是正确的。

- $[l,r]$ 与 $[L,R]$ 有交但不被包含，也就是说 $[l,r]$ 是 $[L,R]$ 分解过程中的一个区间。此时，我们通过子区间的 f 计算出正确的本区间的 f。
- $[l,r]$ 与 $[L,R]$ 没有交集，那么这个区间的 f 不受 $\Phi_{[L,R]}$ 的影响。

此时，假设又有一个修改 $\Phi'_{[L',R']}$，我们仍然在分治树上做分解。然而，在递归时，有可能当前的分治区间 $[l,r]$ 上已经有先前分解出的修改 $\Phi_{[l,r]}$ 了。有两种情况：

- $[l,r]$ 已经是一个完整区间了。此时，我们直接复合两个修改，也就是令 $\Phi_{[l,r]} \leftarrow \Phi'_{[l,r]} \circ \Phi_{[l,r]}$。
- $[l,r]$ 不是一个完整区间。此时，如果直接向下继续分解，那么就会导致新的修改作用在旧的修改之前。由于修改之间往往不交换，这样会导致结果出错。因此，我们需要先把当前区间记录的 $\Phi_{[l,r]}$ 分解到子区间 $[l,m]$ 和 $[m+1,r]$ 上，然后再继续递归分解 $\Phi'_{[l,r]}$，以保证新的修改总是作用在先前的旧修改之后。需要注意的是把 $\Phi_{[l,r]}$ 分解到子区间时也可能出现上一点中的情况。把一个分治区间上的修改分解到子区间上的这个过程叫做 pushdown。

也就是说，我们维护了下面这个不变量：对于分治树上的一个区间 $[l,r]$，假设其分治树上的祖先自下往上分别存储了修改 $\Phi_1,\dots,\Phi_k$。那么在任意时刻，我们需要保证真实的 $f_{[l,r]}(a)$ 等于该结点当前存储的 $\hat{f}_{[l,r]}(a)$ **自下往上**地应用其分治树上所有祖先存储的修改得到的结果，也就是

$$f_{[l,r]}(a) = \hat{f}_{[l,r]}(\Phi_k(\cdots\Phi_2(\Phi_1(a)))) = (\tau_{\Phi_k}\circ\cdots\circ\tau_{\Phi_1})(\hat{f}_{[l,r]}(a)).$$

所以，为了保证修改的顺序正确，我们需要在递归之前对当前结点进行 pushdown。

现在来考虑区间询问。我们首先分解到若干完整的分治区间上，然后考虑每个分治区间的结果。根据我们维护的不变量，一个分治区间真实的 $f_{[l,r]}(a)$ 就是其存储的 $\hat{f}_{[l,r]}(a)$ 应用上其祖先存储的所有修改得到的结果。那么一种回答询问的方法就是在递归时记录祖先存储的修改复合的结果，最后再应用到存储的 $\hat{f}_{[l,r]}(a)$ 上。不过，更常见的做法是在递归过程中对访问到的所有结点 pushdown。这样做之后，祖先中的就没有修改了。根据我们的循环不变式，此时真实的 $f_{[l,r]}(a)$ 就等于存储的 $\hat{f}_{[l,r]}(a)$。

评注. *在递归之前对当前结点 pushdown 总是对的。这是最通用和简便的写法。*

这样我们就得到了完整的支持区间修改和区间查询的线段树。

代码 3.3: 线段树区间修改

```
struct SegTree {
    int n;
    vector<Info> tree;
    vector<Tag> tag;  // 维护分治树上每个结点的修改。对于没有修改的结点，tag[u] 就是恒等变换。
    SegTree(int n_) : n(n_), tree(4 * n_), tag(4 * n_) {}

    void build(int u, int l, int r, const vector<int>&a) {
        if(l == r) {
            tree[u] = Info(a[l]);
```

```
10              return;
11          }
12          tag[u] = Tag();
13          int m = (l + r) >> 1;
14          build(u << 1, l, m, a);
15          build(u << 1 | 1, m + 1, r, a);
16          tree[u] = tree[u << 1] + tree[u << 1 | 1];
17      }
18      SegTree(const vector<int>&a) : SegTree(a.size()){
19          build(1, 1, n, a);
20      }
21      // 实现把修改作用到线段树结点上的函数。
22      void apply(int u, const Tag &newtag) {
23          tree[u] = newtag(tree[u]);
24          tag[u] = newtag(tag[u]);
25      }
26      void pushdown(int u, int l, int r) {
27          if(tag[u].isId()) return;   // 可以不加，但是加了通常快一些。
28          // 分解 tag[u]。通常就是 tag[u] 本身，此时不需要传入 l,r。
29          int m = (l + r) >> 1;
30          Tag tagL = tag[u], tagR = tag[u];
31          tag[u]=Tag()
32
33          // 将分解后的修改作用到到子区间上
34          apply(u << 1, tagL);
35          apply(u << 1 | 1, tagR);
36      }
37      void update(int u, int l, int r, int lq, int rq, const Tag &newtag) {
38          if(l == lq && r == rq) {
39              apply(u, newtag);
40              return;
41          }
42          pushdown(u, l, r);
43          int m = (l + r) >> 1;
44          if(rq <= m) update(u << 1, l, m, lq, rq, newtag);
45          else if(lq > m) update(u << 1 | 1, m + 1, r, lq, rq, newtag);
46          else {
47              update(u << 1, l, m, lq, m, newtag);
48              update(u << 1 | 1, m + 1, r, m + 1, rq, newtag);
49          }
50          tree[u] = tree[u << 1] + tree[u << 1 | 1];
51      }
52      void update(int u, int l, int r, int q, const Info &val) {
53          if(l == r) {
54              tree[u] = val;
```

```
55              return;
56          }
57          pushdown(u, l, r);
58          int m = (l + r) >> 1;
59          if(q <= m) update(u << 1, l, m, q, val);
60          else update(u << 1 | 1, m + 1, r, q, val);
61          tree[u] = tree[u << 1] + tree[u << 1 | 1];
62      }
63      Info query(int u, int l, int r, int lq, int rq) {
64          if(l == lq && r == rq) return tree[u];
65          pushdown(u, l, r);
66          int m = (l + r) >> 1;
67          if(rq <= m) return query(u << 1, l, m, lq, rq);
68          else if(lq > m) return query(u << 1 | 1, m + 1, r, lq, rq);
69          else {
70              return query(u << 1, l, m, lq, m) + query(u << 1 | 1, m + 1,
                    r, m + 1, rq);
71          }
72      }
73      void update(int l, int r, const Tag &newtag) {
74          update(1, 1, n, l, r, newtag);
75      }
76      void update(int q, const Info &val) {
77          update(1, 1, n, q, val);
78      }
79      Info query(int l, int r) {
80          return query(1, 1, n, l, r);
81      }
82  };
```

通过修改 Info 和 Tag，我们已经可以完成不计其数的线段树板子题。下面的练习是基本上是**最困难的一批**，然而如果你已经完全理解了修改与信息设计的原则，那么下面的问题基本上都可以一眼看破。

练习 3.15. 括号序列. 维护括号序列，支持

- 区间赋值
- 反转一个区间内的括号
- 查询区间内左/右括号的数量
- 查询区间内最多有多少个连续的左/右括号
- 查询区间内最长的合法括号子序列的长度
- 查询区间内至少要改变几个括号才能使得区间合法
- 查询区间内有多少个极长连续段
- 查询区间内所有极长连续段长度的最值和 k 次方和

P2572；CF1149C

评注. 利用单侧递归技巧，我们还能查询区间内最长的合法括号**子串**的长度，以及**不匹配位置**的半群信息。

练习 3.16. 拆位. 维护数列，支持

- 区间按位异或/按位与/按位或 x
- 查询区间和

练习 3.17. 区间最大子段和. 维护数列，支持

- 区间赋值
- 区间乘 -1
- 查询区间最大子段和

强于 P4513

评注. 配合模拟费用流可以求出区间最大 k 段子段和。

练习 3.18. 最值. 维护两个数列 a 和 b，假设 k 是比较小但是不固定的数，支持

- 两个数列上的区间加，区间赋值
- 给定长为 k 的序列 $s \in \{a,b\}^k$，查询区间 $[l,r]$ 的所有长度为 k 的子序列/子段（设下标为 $i_1 < \cdots < i_k$）中，$\sum_{j=1}^{k}[s_j]_{i_j}$ 的最值
- 给定长为 k 的序列 s，查询区间 $[l,r]$ 内是否存在长度为 k 的子段（设下标为 $i_1 < \cdots < i_k$），使得 $[s_j]_{i_j}$ 单调不降

P7706

练习 3.19. 历史最值. 维护两个数列 a 和 b，支持

- 两个数列上的区间加、区间赋值、区间和 x 取 max
- 区间内令 $a_i = \max(a_i, b_i + x)$
- 区间内令 $b_i = \max(b_i, a_i + x)$
- 查询两个数列的区间 max

评注. 如果没有第二个操作、第三个操作换成 $b_i = \max(b_i, a_i)$，那么 b_i 相当于记录了 a_i 的**历史最值**。

练习 3.20. 数列. 维护数列，在 $\mathbb{Z}_m$ 也就是 mod m 下做运算。下面的 k 都是比较小但不固定的数。支持

- 区间加 x，区间乘 x，区间赋值为 x
- 查询区间内所有数的 k 次方和
- 查询区间内所有相邻 k 个数的乘积的和
- 查询区间内所有 k 元组乘积的和

P4247, P5142

练习 3.21. 哈希. 维护数列，固定 B 和 p，支持

- 区间加，区间乘，区间赋值
- 计算区间的 B 进制哈希值 $\sum_{i=l}^{r} a_i B^{r-i} \bmod p$

练习 3.22. gcd. 维护数列，支持

- 区间加，区间赋值
- 查询区间内所有数的 gcd

提示

评注. 注意无论按照何种顺序，顺次计算 n 个数的 gcd 的复杂度都是 $O(n + \log V)$。

练习 3.23. 小值域. 维护数列 a 和另一个分治信息序列 b，a 在 $\mathbb{Z}_k$ 也就是模 k 下做运算且 k 很小，支持

- 给定 $x, y \in \mathbb{Z}_k$ 把区间内所有 $a_i = x$ 的 a_i 变成 y
- 查询区间内 a 的中位数和众数
- 给一个集合 $S \subseteq \mathbb{Z}_k$，查询区间内最短的子段，使得所有 $x \in S$ 都在这个子段内出现
- 给一个集合 $S \subseteq \mathbb{Z}_k$，把区间内所有 $a_i \in S$ 的位置拉出来形成一个子序列，然后询问这个子序列的 b 的合并结果

练习 3.24. 加矩阵幂. 固定矩阵 $\mathbf{M}$，维护向量列 $\mathbf{a}$，支持

- 给定向量 $\mathbf{v}$ 和区间 $[l, r]$，对所有 $i \in [l, r]$，令 $\mathbf{a}_i \leftarrow \mathbf{a}_i + \mathbf{M}^{i-l}\mathbf{v}$
- 区间加，区间乘任意矩阵，区间赋值
- 查询区间和

评注. 这个包括了区间加线性递推的情况，而线性递推又包括了区间加等差和等比数列的情况。

3.4　特殊情况的优化

3.4.1　标记永久化

需要 pushdown 的原因是标记之间不交换。然而标记经常是可交换的，比如只有区间加、只有区间乘、只有区间和 x 取 max 等等。对于这种情况，我们就不需要 pushdown 了。在查询时，我们通过递归计算出没有作用过当前结点以及祖先的修改的信息 $\hat{f}_{[lq,rq]}(a)$，然后再应用当前结点的修改 $\Phi_{[l,r]}$ 即可。

例 3.25. 矩形面积并. 给定平面上的若干矩形，我们要计算它们的并的面积。这个问题的做法是扫描线，也就是考虑每个横坐标段 (l, r)，满足所有矩形的横坐标区间要么与 (l, r) 不交，要么完全包含 (l, r)。这样的段可以由所有矩形的横坐标形成的 $2n$ 个点切分出来。之后，我们只需要考虑每个段的纵坐标区间并的长度，而这是一个一维的问题。从一个段移动到它的下一个段的时候，会加入或者离开一个纵坐标区间。以上这些操作形成了下面的子问题：

维护数轴，支持

- 插入一条线段
- 删除一条线段，保证曾经插入过，并且还没有被删除
- 查询当前所有线段的并的长度

练习 3.26. 上面的问题可以泛化成：维护数列，支持

- 区间加 1，区间减 1，保证任意时刻所有位置非负。
- 查询有多少个位置大于 0。

注意这个问题不能用上一题中的标记永久化方法解决。提示

评注. 如果没有所有位置始终非负的假设，这个问题有矩阵乘法归约，无法做到 polylog$n$。在有了这个假设时，这是个比较常见的技巧。

3.4.2　增量修改与前缀查询：树状数组

当查询都是前缀形式的时候，可以发现线段树的所有左儿子都是无用的。此时，可以直接将所有的左子结点删去，把剩余的结点重连成树。这就得到了树状数组。不过，实际使用时，另一种理解可能会更加方便。定义 $\texttt{lowbit}(i)$ 为 i 的二进制表示中最低位的 1 所代表的数，如 $\texttt{lowbit}(20) = \texttt{lowbit}((10100)_2) = (100)_2 = 4$。那么树状数组就是维护一个 n 大小的数组 c，其中 c_i 存储了 $f_{(i-\texttt{lowbit}(i),i]}(a)$。这样，查询前缀 $f_{[1,r]}(a)$ 就是把 $[1,r]$ 拆成若干 $(x-\texttt{lowbit}(x),x]$ 然后合并。对于**增量修改**（修改 a_i 后可以直接算出此修改对包含 i 的区间信息 $f_{[l,r]}(a)$ 的影响），可以找出所有包含 i 的 $(x-\texttt{lowbit}(x),x]$，然后修改对应的 c_x。

数据结构 3.27. 树状数组.

预处理. 初始令所有 $c_i = f_i(a)$。从 0 开始枚举所有 k，对每个 2^{k+1} 的倍数 i，令 $c_i = c_{i-2^k} \odot c_i$。

查询. 初始令 $s = 0$。对于查询 $[1,r]$，不断地执行 $s \leftarrow c_r \odot s$，$r \leftarrow r - \texttt{lowbit}(r)$，直到 $r = 0$。返回 s。

修改. 对于修改 $a'_x = \varphi(a_x)$，假设存在 $\tau_{\varphi,[l,r]}$ 使得对任何包含 x 的区间 $[l,r]$，都有 $f_{[l,r]}(a') = \tau_{\varphi,[l,r]}(f_{[l,r]}(a))$。不断地令 $c_x \leftarrow \tau_{\varphi,(x-\texttt{lowbit}(x),x]}(c_x)$，$x \leftarrow x + \texttt{lowbit}(x)$，直到 $x > n$。

不难发现预处理的复杂度为 $O(n)$，而修改查询的复杂度都是 $O(\log n)$。实现代码时，有一个细节需要处理——如何计算 $\texttt{lowbit}(x)$？可以注意到这等于 `x&(~(x-1))`。此外，根据整数的表示法，我们还有 `~(x-1)=-x`。在代码中，通常会用 `x&(-x)` 来实现 $\texttt{lowbit}$。

代码 3.4: 树状数组

```
1 Info s[N];
2 // 每一层分治对应一层数组
```

```
void build(int n) {
    for(int i=1;i<=n;i++)
        s[i]=Info(a[i]);
    for(int k=1;k<=n;k<<=1)
        for(int i=k<<1;i<=n;i+=(k<<1))
            s[i]=s[i-k]+s[i];
}
Info query(int r) {
    Info ans;
    for(;r;r-=r&(-r))
        ans=s[r]+ans;
    return ans;
}
void upd(const Modify &phi, int x, Info &info) {
    // 应用 phi 对区间 (x-lowbit(x),x] 的信息 info 的影响)
}
void update(int x, const Modify &phi) {
    for(;x<=n;x+=x&(-x))
        upd(phi,x,s[x]);
}
```

练习 3.28. 树状数组的常见应用.

- 单点加，单点赋值，查询区间和
- 单点加非负整数，单点和 x 取 max，查询前缀 max 和后缀 max

练习 3.29. 假设修改和查询的位置在 $[1,n]$ 中均匀随机，n 是 2 的幂。说明查询和修改的期望循环次数是 $\frac{1}{2}\log_2 n$。

练习 3.30. 树状数组的区间查询与任意单点修改. 说明树状数组可以在 $O(\log^2 n)$ 时间内查询区间信息。对于一般的单点修改，说明树状数组可以在 $O(\log^2 n)$ 内实现。

3.4.3　可差分的修改

像区间加、区间异或这种修改可以通过差分来变成单点修改，这样就能通过提高查询的代价来简化修改。由于区间修改通常是麻烦的，能差分的问题总是应该差分。之后，维护的信息还常常是可减的，此时可以通过树状数组大大减小常数。

例 3.31. 假设 p 是质数。在 $\mathbb{F}_p$ 或者说模 p 意义下，乘法不是直接可差分的修改，因为存在乘 0 的情况。然而如果我们把 0 视为变量 x 并且在 $\mathbb{F}_p(x)$ 下做运算，那么乘法就是可差分的了。具体来说，我们把每个元素视为 ax^b，那么区间乘除 0 就变成了 b 的区间加减。维护区间乘积也同理。需要注意这个做法只适用于**只有乘法**的情况。

例 3.32. 支持

- 区间加
- 查询区间和

差分后使用树状数组完成。

本章总结

- 静态区间数据结构的单点修改：修改所有发生变化的预处理区间
- 批量修改的概念
- 全局修改与全局询问：快速计算修改 Φ 对分治信息 f 的影响 τ_Φ
- 全局修改与区间询问：修改的复合闭包
- 区间修改与区间询问：修改的分解
- 标记与信息的设计原则：1. 根据 2.6 初步设计分治信息；2. 寻找修改的闭包 $\overline{\mathcal{M}}$；3. 取一个 $\overline{\mathcal{M}}$ 中的一般修改 Φ，尝试计算修改对信息的影响 $\tau_{\Phi,[l,r]}$，把需要的辅助信息添加进分治信息 f。4. 重复第三步直到所有东西都可以计算。
- 特殊情况的优化：可交换修改的标记永久化、增量修改与前缀查询的树状数组、可差分的修改

4 线段树的进阶操作

4.1 zkw 线段树

zkw 线段树放弃了线段树的自上而下分治结构，而是改为自下而上合并。初始所有的叶子结点都是一个单元素区间 $[i,i]$，我们每次把所有的结点两个两个地合并成更大的区间，两端可能剩下的孤点不管。这样得到新的一些长为 2 的区间，再继续做这个合并过程，直到只剩一个结点。加上每层得到的孤点，我们会形成一个森林结构。

zkw 线段树的精髓在于标号。叶子 $[i,i]$ 的标号为 $n+i-1$，一个结点 u 的父亲标号为 $\lfloor n/2 \rfloor$。通过这样的标号方式，我们可以直接获得叶子结点的标号。考虑如何在 zkw 线段树上分解区间。我们用左闭右开区间 $[l,r)$ 来避免一些特判。从叶子开始，我们有左右两个区间结点 $u=l+n-1, v=r+n$。在 v 是一个右儿子时，记 v 的左邻居为 v'。如果 u 是一个右儿子，那么 u 会作为一个完整区间出现；否则，u 会被其某个祖先作为完整区间包含。也就是说，u 是分解出的最终区间当且仅当 u 是一个右儿子。同理，v' 是分解出的最终区间当且仅当 v 是一个右儿子。确定完 u,v' 是否是分解出的最终区间后，我们就把 u,v 往上跳一层，递归寻找上面的结点，直到 u,v 相同为止。这样我们就找到了所有分解出的最终区间。在有区间修改时，我们还需要知道所有的非最终区间来 pushdown。非最终区间分为两部分：一部分是最终区间的父亲，另一部分是 u,v 相同后到根路径上的结点。

代码 4.1: zkw 线段树

```
struct SegTreeZKW
{
    vector<Info> info;
    vector<Tag> tag;
    int n;
    SegTreeZKW(int n): n(n)
    {
        info.resize(2*n+2);
        tag.resize(2*n+2);
    }
    template<typename SeqType>
    SegTreeZKW(const vector<SeqType>& a): SegTreeZKW(a.size()-1)
    {
        build(a);
    }
    void apply(int u, const Tag& newtag)
    {
        info[u] = newtag(info[u]);
        tag[u] = newtag(tag[u]);
    }
    void pushdown(int u)
    {
```

```
23          if (!tag[u].isId())
24          {
25              apply(u << 1, tag[u]);
26              apply(u << 1 | 1, tag[u]);
27              tag[u] = Tag();
28          }
29      }
30      template<typename SeqType>
31      void build(const vector<SeqType>& a)
32      {
33          for(int i=1;i<=n;i++) info[n + i] = Info(a[i]);
34          for(int i=n-1;i>=1;i--)
35              info[i] = info[i << 1] + info[i << 1 | 1];
36      }
37      Info query(int lq, int rq)
38      {
39          lq += n; rq += n+1;
40          for(int i=__lg(n)+1;i>=1;i--)
41          {
42              pushdown(lq >> i);
43              if ((rq >> i) != (lq >> i))
44                  pushdown(rq >> i);
45          }
46          Info resl, resr;
47          while(lq < rq)
48          {
49              if (lq & 1) resl = resl + info[lq++];
50              if (rq & 1) resr = info[--rq] + resr;
51              lq >>= 1; rq >>= 1;
52          }
53          return resl+resr;
54      }
55      void update(int lq, int rq, const Tag& newtag)
56      {
57          lq += n; rq += n+1;
58          for(int i=__lg(n)+1;i>=1;i--)
59          {
60              pushdown(lq >> i);
61              if ((rq >> i) != (lq >> i))
62                  pushdown(rq >> i);
63          }
64          int u=0, v=0;
65          while(lq < rq)
66          {
67              if (lq & 1)
```

```
68              {
69                  u=lq;
70                  apply(lq++, newtag);
71              }
72              if (rq & 1)
73              {
74                  v=rq;
75                  apply(--rq, newtag);
76              }
77              do{u>>=1; info[u]=info[u << 1] + info[u << 1 | 1];}while(lq==
                    rq && u);
78              do{v>>=1; info[v]=info[v << 1] + info[v << 1 | 1];}while(lq==
                    rq && v);
79              lq >>= 1; rq >>= 1;
80          }
81      }
82  };
```

注意到这种方式的空间从 $4n$ 降低到了 $2n$。在时间上，zkw 线段树只能节省**递归**的开销，而不能降低信息或标记合并的代价。zkw 线段树节省的递归开销大约相当于维护四到八个 unsigned 的和。所以，当信息或标记复杂时，这个性能提升就变得较为微弱。如果实现得不好，可能还会有额外的信息或标记合并开销。

4.2　动态开点

线段树的单点修改会访问至多 $\lceil \log_2 n \rceil + 1$ 个结点，区间修改会访问至多 $4\lceil \log_2 n \rceil - 3$ 个结点。没有被修改过的结点可以不显式建出结点，而是按照某种“默认模式”处理。

代码 4.2: 动态开点线段树

```
1   // 不需要 build 了，可以使用 update 来替代 build
2   // 假设不存在的结点编号为 0，使用 newNode(l,r) 来创建新结点，初始化为 [l,
        r] 的默认信息
3   // 大多数情况下，默认信息都是 0，此时 newNode 和 pushdown 都不需要传入 l,
        r。
4   int newNode(int l, int r)
5   {
6       ++nodeCnt;
7       ch[nodeCnt][0] = ch[nodeCnt][1] = 0;
8       info[nodeCnt] = defaultInfo(l, r);
9       tag[nodeCnt] = Tag();
10  }
11  void pushdown(int u, int l, int r)
12  {
13      int mid = (l + r) >> 1;
14      if(!tag[u].isId())
```

```
15      {
16          if(!ch[u][0]) ch[u][0]=newNode(l, mid);
17          if(!ch[u][1]) ch[u][1]=newNode(mid + 1, r);
18          apply(ch[u][0], tag[u]);
19          apply(ch[u][1], tag[u]);
20          tag[u] = Tag();
21      }
22  }
23  Info query(int u, int l, int r, int lq, int rq)
24  {
25      if(!u) return defaultInfo(lq,rq);
26      if (lq == l && r == rq)
27          return info[u];
28      int mid = (l + r) >> 1;
29      pushdown(u, l, r);
30      if (rq <= mid)
31          return query(ch[u][0], l, mid, lq, rq);
32      else if(lq > mid)
33          return query(ch[u][1], mid + 1, r, lq, rq);
34      else
35          return query(ch[u][0], l, mid, lq, mid) + query(ch[u][1], mid +
                1, r, mid + 1, rq);
36  }
37  void update(int &u, int l, int r, int lq, int rq, const Tag& newtag)
38  {
39      if(!u) u=newNode(l,r);
40      if(lq == l && r == rq)
41      {
42          apply(u, newtag);
43          return;
44      }
45      int mid = (l + r) >> 1;
46      pushdown(u, l, r);
47      if (rq <= mid)
48          update(ch[u][0], l, mid, lq, rq, newtag);
49      else if (lq > mid)
50          update(ch[u][1], mid + 1, r, lq, rq, newtag);
51      else
52      {
53          update(ch[u][0], l, mid, lq, mid, newtag);
54          update(ch[u][1], mid + 1, r, mid + 1, rq, newtag);
55      }
56      info[u] = (ch[u][0] ? info[ch[u][0]] : defaultInfo(l,mid)) + (ch[u
            ][1] ? info[ch[u][1]] : defaultInfo(mid+1,r));
57      // 如果 defaultInfo 的代价比较大，那么也可以选择在这里把不存在的子节
```

```
        点建出来。这样做会让空间变为两倍。
58  }
```

例 4.1. CF803G Periodic RMQ Problem. 维护序列 a，初始为某个序列 b 重复 k 次得到的序列。支持区间赋值、查询区间最值。

4.3　线段树上二分

有时，我们需要支持查询一个区间内满足某个条件的最左或最右位置，如查询区间内第一个前缀 max 大于等于某个值的位置。此时，直接的想法是套上一层二分，内部用线段树来判断。然而，我们可以利用线段树的结构，直接在线段树上进行二分查找，从而省掉一个 log。从理论上来说，我们可以先把区间在线段树上分解为不超过 $2\log n$ 个区间，然后从左到右遍历找到答案所在的完整区间，然后递归进那个完整区间分治查找。不过，实现上要容易得多：

代码 4.3: 线段树上二分

```
1   //以查询区间 [l,r] 内最左的满足 s[l,i]>=x 的 i 的位置为例
2   int query(int u, int l, int r, int lq, int rq, Info& preinfo, int x)
3   {
4       if(l==lq&&r==rq)
5       {
6           if(info[u].s<x)
7           {
8               preinfo = preinfo + info[u];
9               return -1;
10          }
11          //否则，说明答案就在此区间内
12      }
13      if(l==r)
14          return l;
15      pushdown(u);
16      int mid=(l+r)>>1;
17      if(rq<=mid)
18          return query(u<<1, l, mid, lq, rq, preinfo, x);
19      else if(lq>mid)
20          return query(u<<1|1, mid+1, r, lq, rq, preinfo, x);
21      else
22      {
23          int ans=query(u<<1, l, mid, lq, mid, preinfo, x);
24          if(ans!=-1) return ans;
25          else return query(u<<1|1, mid+1, r, mid+1, rq, preinfo, x);
26      }
27  }
28  int query(int lq, int rq, int x)
29  {
```

```
30      return query(1,1,n,lq,rq,Info(),x);
31  }
```

练习 4.2. 树状数组上二分. 在树状数组上二分来实现：查询序列中最靠左的满足某个条件的位置。

练习 4.3. 权值线段树.

- 假设值域为 V，通过维护每个值的出现次数来在 $O(\log V)$ 的时间和 $O((n+q)\log V)$ 的空间内实现普通平衡树的所有功能。
- 在 $V = O(n)$ 时，使用树状数组来实现。

4.4 线段树分裂与合并

4.5 单侧递归

4.6 李超线段树

4.7 Segment Tree Beats

4.8 KTT

本章总结

- zkw 线段树：合适的标号，自下而上的合并
- 动态开点：只构建修改时访问的结点
- 线段树上二分：利用区间分解定理来在线段树上进行二分查找

5 扫描线与几何模型

通过扫描问题的某一维度并考虑两个相邻状态之间的差，我们往往可以将一个多维范围查询问题变为少一维的范围修改查询问题。在本章中，我们尝试用一些例子让大家感受**维度**，并给出十分常用的**平面矩形建模**。

本章的大部分问题都可以直接通过可持久化数据结构实现强制在线，我们在此仅考虑离线情况。

5.1 平面数点问题

很多时候我们会遇到由不等式框定的查询范围，如 $l \le x \le r$。此时，一个可能有利于直观的办法是将不等式的限制转到平面坐标系中。每一个不等号都对应一个半平面，多个不等式的交集就对应一个平面图形。我们可以把每个元素看成平面上的一个点，那么查询就变成了数点问题：查询范围内所有点的信息和。

5.1.1 可减信息

对于可减的情况，可以通过容斥把 $\ge$ 变成 $\le$。所以每个维度只需要处理单侧的限制。

例 5.1. 矩形数点. 给定一些 (x_i, y_i, w_i)，多次查询

$$\sum_i [l_x \le x_i \le r_x][l_y \le y_i \le r_y] w_i.$$

这对应于：给定平面上 n 个带权点，多次查询一个矩形内的点权和。

通过坐标变换（或者说变量代换），可以把平行四边形变成矩形。

例 5.2. 平行四边形数点. 给定一些 (x_i, y_i, w_i)，多次查询

$$\sum_i [l_x \le x_i \le r_x][y_i - x_i \in [l_d, r_d]] w_i.$$

这对应于：给定平面上 n 个带权点，多次查询一个方向固定的平行四边形内的点权和。

评注. 在这里，“方向固定”指的是围成形状的这些半平面的边界线的斜率是固定的。

通过在无限远处差分，可以把直角三角形变成两种四边形的差。

例 5.3. 三角形数点. 给定一些 (x_i, y_i, w_i)，多次查询

$$\sum_i [l_x \le x_i \le r_x][y_i - x_i \in [l_d, r_d]][y_i + x_i \in [l_s, r_s]] w_i.$$

这对应于：给定平面上 n 个带权点，多次查询一个等腰直角三角形内的点权和。

练习 5.4. 把这个问题旋转 45 度。事实上，对于任何方向固定的三角形都可以做。

由于多边形总是可以切分成三角形，我们在理论上就可以做任意的方向固定的不规则图形的数点了。

例 5.5. 梯形数点. 给定一些 (x_i, y_i, w_i)，多次查询

$$\sum_i [l_x \le x_i \le r_x][y_i - x_i \in [l_d, r_d]][y_i + x_i \in [l_s, r_s]]w_i.$$

这对应于：给定平面上 n 个带权点，多次查询一个等腰梯形或者直角梯形内的点权和。

练习 5.6. * 半平面交数点. 给定一些 (x_i, y_i, w_i)，多次查询

$$\sum_i [x_i \in [l_x, r_x]][y_i \in [l_y, r_y]][x_i - y_i \in [l_d, r_d]][x_i + y_i \in [l_s, r_s]]w_i.$$

这对应于：给定平面上 n 个带权点，多次查询一个由一些半平面交出的一个“规则”多边形内的点权和。

5.1.2　不可减信息

把以上问题中的求和换成 max。此时，无法通过容斥把 $\ge$ 变成 $\le$，而是需要通过分治来实现。在此时，问题基本上就被一般化为 d **维正交范围查询问题**。通常来说，d 个半平面的限制会带来 $\tilde{\Theta}(\log^{d-1} n)$ 的复杂度。我们将在下一章中看到常见的做法。

5.1.3　转置

对于数点问题，我们可以有另一个理解。假设我们给点用 $1, \ldots, n$ 编号，询问用 $1, \ldots, q$ 编号。考虑矩形 $\mathbf{A}_{ij} = [$点j 在询问i 的范围内$]$，那么我们相当于在计算矩阵乘法 $\mathbf{A} \cdot \mathbf{w}$，其中 $\mathbf{w}$ 是点的权值向量。此问题的**转置**为：假设每个询问 i 都有一个权值 u_i，我们想要计算 $\mathbf{A}^T\mathbf{u}$，即对每个 i 计算 $\sum_j [$点j 在询问i 的范围内$]u_i$。这相当于：平面上有一些点，每个询问 i 需要对其范围内的点的答案加上 u_i，最后计算每个点的结果。这个也称为**对偶**数点问题。

什么是对偶？

通常来说，转置问题可以通过**转置原理**做到与原问题相同的复杂度。不过，在这里，我们只需要正常的思考即可得到合理的做法。

例 5.7. 对偶二维数点. 给定平面上 n 个带权矩形，多次查询包含某个点的矩形的权值和。

5.2　矩形并问题

例 5.8. 矩形面积并. 给定平面上的若干矩形，求其并的面积

为什么不叫矩形并面积？

练习 5.9. 至少被 k 个矩形覆盖的面积.

练习 5.10. 矩形并的周长.

5.3　常见平面建模

有许多东西都可以被视为维度：序列下标、权值、时间等等。我们可以把它们中的一个或几个转化为平面坐标系中的维度，从而把问题转化为数点问题来解决。

例 5.11. 序列 × 值域. 给一个序列 a，每次给定 l, r, x, y，查询有多少个 $i \in [l, r]$ 满足 $a[i] \in [x, y]$。

例 5.12. 多次询问：如果把 $a[l, r]$ 加一，那么序列中有多少个不同的值

例 5.13. 序列 × 序列. 给两棵树 T_1 和 T_2，每次询问有多少个点在 T_1 中位于 u 的子树内且在 T_2 中位于 v 的子树内。

例 5.14. 区间包含关系. 初始给定 n 个区间 $[l_i, r_i]$。每次给出 $[L, R]$，询问

- 有多少个区间包含 $[L, R]$?
- 有多少个区间与 $[L, R]$ 相交?
- 有多少个区间被 $[L, R]$ 包含?

例 5.15. 给一棵树，点从 1 到 n 编号，边从 1 到 $n-1$ 编号。每次询问：只保留编号在 $[l, r]$ 内的点和编号在 $[l, r]$ 内的边时，这些点的导出子图有多少个连通块。提示

例 5.16. NOIP2025 序列询问. 每次询问给定 $[L, R]$，对每个 i 求出所有包含 i 且区间长度在 $[L, R]$ 内的区间的和的最大值

解答. 对于 i，相关的区间 $[l, r]$ 为 $l \le i$，$r \ge i$，$L \le r - l + 1 \le R$ 圈出来的梯形，其中一个点的权值为 $s_r - s_{l-1}$，求最大值。由于最大值可重复贡献，可以将梯形变成两个平行四边形的并。对于平行四边形的询问，相当于计算形如

$$\max_{l \in [L'_i, R'_i]} \max_{r-l+1 \le d} s_r - s_{l-1}$$

的问题，其中 $[L'_i, R'_i]$ 是和 i 有关的定长区间，d 是和 $R - L$ 有关的定值。首先可以用单调队列计算所有 $ms_l = \max_{r-l+1 \le d} s_r - s_{l-1}$，然后再单调队列计算所有 $\max_{l \in L'_i, R'_i} ms_l$。

评注. 事实上三角形内的结果也是能算的，参见 *P6780*.

例 5.17. 序列 × 时间. 给一个序列 a，支持单点加、查询区间和。将此问题视为二维数点。

5.4　区间问题

我们经常通过扫描线来处理区间询问。此时，通常有两种理解：第一种是在扫描 r 的过程中维护所有 l 的答案；另一种是把 l, r 视为两个维度，则所有可能的区间构成平面上的一个三角形。虽然通常来说前一种理解更常用，但也不要忘记后者。

5.4.1　数颜色类问题

有一些问题是查询区间内出现过的值的集合的一些信息。我们展示一些常见的例子和处理手法。

例 5.18. 区间数颜色. 给定两个数列 a, b，查询所有 $a[l, r]$ 中出现过的值 w 的 b_w 之和。

做法一. 考虑一种拆贡献的方式：一个 $[l, r]$ 内的位置有贡献当且仅当其上一次出现的位置在 $[l, r]$ 之外。这样，我们预处理出每个值的上一次出现位置 p_i，则询问就变成 $i \in [l, r], p_i < l$ 的二维数点。

做法二. 也可以考虑另一种拆贡献的方式：枚举所有权值 w，当 $a[l, r]$ 中包含 w 时产生贡献。那么考虑一个值 w 的所有出现位置，当 $[l, r]$ 包含其中某个位置时，在平面上的 (l, r) 位置产生 b_w 的贡献。注意到如果做一步容斥，即考虑**不包含** w 的任意一次出现的区间，则所有这样的区间在平面上可以由 出现次数 $+1$ 个三角形表达。而由于我们不会询问 $l > r$，也可以把三角形换成矩形。这样，问题就变成：有 $O(n)$ 个矩形加，最后查询单点值，即对偶二维数点。

做法三. r 从 1 扫描到 n，维护每个 l 的答案。假设 $a[i]$ 的上一次出现位置是 j，那么 r 从 $i-1$ 移动到 r 的时候，$(j, i]$ 的答案加一，其他位置不变。

评注. 在下一章中我们来处理带修的情况。

练习 5.19. 区间内只出现一次的数的和。

练习 5.20. 区间内出现恰好 k 次的数的和。

练习 5.21. CF1000F. 找出区间内任何一个只出现了一次的数，或者报告不存在

练习 5.22. 区间内出现偶数次的数的异或和

例 5.23. 给定两个数列 a, b，每次给定 $[l, r]$，查询：$\sum_{w \in a[l,r]} \max\{b_i : a_i = w\}$，也就是对每个在 $a[l, r]$ 中出现过的值 w，计算其所有出现位置中 b 的 max，再求和的 b_w 之和。

解答. 在扫描 r 的时候维护每个 l 的答案。对每个 w 维护其对应的 b 的单调栈，则区间加减即可维护答案。

练习 5.24. CF1422F. 区间 lcm（对常数模数取模），值域和 n, q 同阶。

评注. *P5655* 是值域非常大但 n,q 比较小的情况。

练习 5.25. CF522D. 查询区间内等值对的最小距离

5.4.2　mex 类问题

集合 S 的 mex 定义为不在 S 中的最小非负整数。我们也可以通过扫描线来处理一些 mex 类问题。

例 5.26. 区间 mex. 查询区间内最小的没有出现过的自然数（mex）。

做法一. 在扫描线的过程中，维护每个值最靠右的出现。查询时在线段树上二分。

做法二. 在扫描线的过程中直接维护每个 l 的答案 m_l，则 m_l 构成若干单调不降的连续段。假设 r 右移时遇到了元素 x，则只有对于 $m_l = x$ 的那些 l，答案会发生变化。假设这些 l 对应的连续段为 L, R，我们可以 O(**新的段数**) 更新这一段的 m。首先使用做法一求出 m_R，然后查找 m_R 在原数组中上一次出现的位置 t，则 $(t, R]$ 这一段会更新为 m_R，然后令 $R \leftarrow t$ 继续做，直到 $t < L$。

事实上，上面的做法二可以导出区间 mex 的一个重要性质。

定义 5.27. 极小 mex 区间. 如果区间 $[l,r]$ 满足其任何真子区间的 mex 都不等于 $[l,r]$ 的 mex，则称 $[l,r]$ 是一个**极小 mex 区间**。

定理 5.28. 极小 mex 区间的结构. 极小 mex 区间即为做法二中的所有满足 $t \geq L$ 的 $[R,r]$。

练习 5.29. 证明以下性质：

- 除了单点以外，极小 mex 区间至多有 $n + \min(V, n)$ 个。
- 极小 mex 区间之间没有包含关系
- 极小 mex 区间 $[l,r]$ 的 mex 为 $\max(a_l, a_r) + 1$，且 a_l, a_r 在 $[l,r]$ 内都只出现过一次。
- 一个区间 $[l,r]$ 的 mex 为被 $[l,r]$ 包含的极小 mex 区间中 mex 的最大值
- 假设极小 mex 区间 $[l,r]$ 的 mex 为 k，则所有包含 $[l,r]$ 的 mex 为 k 的区间都是 (L,R) 的一个子区间，其中 L, R 分别为 k 在 $[l,r]$ 左/右边第一次出现的位置
- mex 为 k 的区间在平面上可以表示为 $O(\mathrm{cnt}_k)$ 个矩形，其中 cnt_k 是 mex 为 k 的极小 mex 区间的个数

有人想算算精确上界吗？

练习 5.30. 对每个 k，询有多少个区间的 mex $> k$。

做法一. 对每个右端点 r，维护 $l_k(r)$ 为可能的最靠右的左端点，则答案即 $\sum_r r - l_k(r) + 1$。当 k 增加时，相当于 O(出现次数) 次区间和 x 取 min。注意 $l_k(r)$ 关于 r 单调不降，故区间取 min 可以二分出发生变化的位置后变成区间赋值。还需要维护全局和。ODT 或者线段树处理都可以。

做法二. 计算出所有极小 mex 区间，然后对每个 k 使用矩形面积并计算出 mex 为 k 的区间数量。

练习 5.31. 求所有区间的 mex 之和。

5.4.3 子区间类问题

有一些问题查询区间内所有子区间的信息和。此时，我们可以扫描 r，然后维护每个 l 的答案的**历史和**。

例 5.32. 全局固定 k。每次询问区间内有多少个子区间满足 $\text{mex} > k$。

评注. 每次询问区间内所有子区间的 mex 的和也是能做的。以及 *CF1148H* 查询有多少个子区间满足 $\text{mex} = k$，k 每次不同。也可以使用极小 mex 区间机械地做。

例 5.33. CF997E Good Segments. 给一个排列，每次询问一个区间有多少子区间满足 $\max - \min = r - l$。提示

例 5.34. 带有询问矩形的矩形面积并. 矩形并局限在一个询问矩形内的面积。

例 5.35. NOIP 2022 比赛. 给定两个序列 a, b，每次询问

$$\sum_{[i,j]\subseteq[l,r]} \max a[i,j] \cdot \max b[i,j].$$

5.5 维护时间维

例 5.36. 维护序列，每个位置是一个栈，初始均为空。支持对区间内的栈压入一个分治信息；查询只考虑 $[l, r]$ 时刻的修改时，第 i 个栈中的信息和。

解答. 对序列扫描线，则在时间维上需要支持单点加入和删除分治信息。线段树即可。

评注. 这个问题的强制在线需要一些聪明的手段。

例 5.37. IOI2021 分糖果. 维护序列 a_i，每个位置有一个上限 c_i。支持：区间加（可能有负数）并在加完之后让 a_i 裁剪到 $[0, c_i]$（即小于 0 变成 0，大于 c_i 变成 c_i），查询单点值。

解答. 先忽略这个裁剪，对序列扫描线，维护一个位置在每个时刻上的值。如果我们可以计算出 t 时刻前最后一次裁剪发生的时刻，那么就容易计算出 t 时刻的真实值了。注意到如果一段时间内 0 与 c_i 都被到达过，则这段时间内的**极差**必然大于等于 c_i。我们在线段树上二分出最大的满足 $[t_0, t]$ 内的极差 $\geq c_i$ 的时刻 t_0，则最后一次裁剪发生的时刻即为 $[t_0, t]$ 内的 max

或者 min。如果 $[0,t]$ 内的极差小于 c_i，则说明 c_i 没有被到达过，$[0,t]$ 内的 min 即为最后一次裁剪发生的时刻。

某个时刻上的修改相当于时间轴上的后缀加，使用支持后缀加、查询区间最值的线段树即可。

练习 5.38. P7560.

例 5.39. 给一个序列 a，支持区间加，查询只考虑第 l 到第 r 个修改时的区间和。

例 5.40. P8512. 给一个初值为 0 的序列 a，支持区间赋值，查询只考虑第 l 到第 r 个修改时的区间和。

本章总结

- 半平面与二维数点；对偶二维数点
- 维度的概念：序列下标、权值、时间等等
- 区间问题的常用做法：扫描 r 的过程中维护每个 l 的答案；或者把 l,r 视为两个维度在平面上表示问题
- 数颜色类问题的常用做法：前驱
- mex 类问题的常用做法：极小 mex 区间
- 子区间问题的常用做法：历史和
- 数据结构维护时间维

6 分治

事实上分治仅仅是一个算法设计方法，并不特指某个算法。我们在这一章中介绍常见的想法和例子。

6.1 标准序列分治

均匀划分序列/点集来进行分治。

例 6.1. 可以用线段树维护的诸多信息. 由于线段树是基于分治设计的数据结构，我们已经做了很多分治的练习了。

例 6.2. 平面最近点对. 给定平面上的 n 个点，查询距离最近的点对。

解答. 先按 x 排序。`solve(l,r)` 计算第 l 到第 r 个点之内的最近点对。首先递归 `solve(l,m)` 和 `solve(m+1,r)`，得到两边的最近点对距离 d。接下来需要处理跨过中点的点对 (i,j)。注意到点对 (i,j) 如果能更新答案，则有 $x_i \in [x_m-d,x_m], x_j \in [x_m,x_m+d]$，$y_j \in [y_i-d,y_i+d]$。也就是说，$j$ 被框在了 $[x_m,x_m+d]\times[y_i-d,y_i+d]$ 这个矩形内。另一方面，我们知道右侧任意两个点之间的距离都至少是 d，因此以每个点为圆心作半径为 $d/2-\epsilon$ 的圆，则所有这些圆之间都不交。因此，$[x_m,x_m+d]\times[y_i-d,y_i+d]$ 这个矩形内最多有 $O(1)$ 个点。

那么做法就很明显了：按 y 排序后，筛出 x 在 $[x_m-d,x_m+d]$ 内的点，然后枚举左侧的每个点 i，逐个检查可能更新答案的 j。由于可能更新答案的点必须满足 $y_j \in [y_i-d,y_i+d]$，故这样的点在按 y 排序并按 x 筛选后位于序列的一段连续区间内，且长度为 $O(1)$。归并 y 即可做到总复杂度 $O(n\log n)$。

例 6.3. CF1609F. 查询有多少个区间的 max 和 min 的 `popcount` 相同。`popcount` 可以加强为任意函数 f。

做法一:序列分治. `solve(l,r)` 计算 $[l,r]$ 内的答案。首先递归 `solve(l,m)` 和 `solve(m+1,r)`，得到两边的答案。接下来需要处理跨过中点的区间 $[i,j]$。令 $mx[i]=\max[i,m]$，$mx[j]=\max[m+1,j]$，$mi[i]=\min[i,m]$，$mi[j]=\min[m+1,j]$，则 $\max[i,j]=\max(mx[i],mx[j])$，$\min[i,j]=\min(mi[i],mi[j])$。那么问题就变成

$$\sum_{i=1}^{m}\sum_{j=m+1}^{n}[f(\max(mx[i],mx[j]))=f(\min(mi[i],mi[j]))].$$

接下来，枚举 $mx[i],mx[j]$ 以及 $mi[i],mi[j]$ 之间的大小关系，共有四种情况，再由对称性可以分为两类情况：

- 最值都在左侧：$mx[i]\geq mx[j], mi[i]\leq mi[j]$。枚举 i，由于 mx 与 mi 都是单调的，满足这个限制的 j 必然是一个前缀 $[m+1,j_i]$，且 $O(r-l+1)$ 扫一遍即可计算出所有 j_i。

对于每个 i，满足这个限制的 j 的数量为

$$\sum_{j=m+1}^{j_i}[f(mx[i])=f(mi[i])]=(j_i-m)[f(mx[i])=f(mi[i])],$$

$O(n)$ 计算即可。

- 最值都在右侧：$mx[i]<mx[j],mi[i]>mi[j]$ 同理。
- max 在左侧，min 在右侧：$mx[i]\geq mx[j],mi[i]>mi[j]$。枚举 i，满足这个限制的 j 是一个区间 $[L_i,R_i]$，且 $O(r-l+1)$ 扫一遍即可计算出所有这些区间。对于每个 i，满足这个限制的 j 的数量为

$$\sum_{j=L_i}^{R_i}[f(mx[i])=f(mi[j])].$$

可以注意到这是一个特殊的二维数点：对每个 i，查询满足 $j\in[L_i,R_i]$，$f(mi[j])=f(mx[i])$ 的 j 的数量。差分后可以离线 $O(r-l+1)$ 完成（注意每个 $mi[j]$ 或者 $mx[j]$ 都是原序列中的某个 $a[i]$，因此在分治前离散化所有 $f(a[i])$ 即可将值域降到 $O(n)$）。
- max 在右侧，min 在左侧：$mx[i]<mx[j],mi[i]\leq mi[j]$ 同理。

评注. 这类题的通用做法都是枚举 $mx[i],mx[j]$ 与 $mi[i],mi[j]$ 之间的大小关系，然后变为偏序数点之类的问题。不过，某些题目可能具有特殊的性质，从而能够以更低的复杂度完成。

做法二：扫描线. 根据 max 的取值，所有的区间可以被表示为 n 个不交矩形：对每个 i 计算出 i 能够作为区间最值的所有区间 $[l,r]$，这些区间可以由 $(\mathrm{pre}_i,i]\times[i,\mathrm{suf}_i)$ 刻画，其中 pre_i 和 suf_i 分别表示 i 左/右侧第一个比 a_i 大的元素的位置。同理，根据 min 的取值，也可以画出 n 个不交矩形。

枚举每一个值 c，考虑那些 $\max=c$ 的区间与 $\min=c$ 的区间对应的矩形。那么我们需要计算出**被覆盖了两次**的面积，扫描线即可做到 O(矩形个数 log 矩形个数)。由于总共的矩形数量是 $O(n)$，这样也是 $O(n\log n)$。

练习 6.4. 统计有多少个区间满足 $\max-\min\leq \mathrm{len}+k$

例 6.5. P6109. 先进行若干矩形加，然后查询若干次矩形 max，点集为全平面。

例 6.6. P3350. 多询问网格图最短路

6.2　非标准序列分治

如果能保证分治的复杂度为 $T(n)=T(n_1)+T(n_2)+O(\min(n_1,n_2))$，那么复杂度也是 $T(n)=O(n\log n)$。这个结论可以通过将分治过程视为启发式合并的逆过程来证明。

例 6.7. 最值分治/笛卡尔树分治. P4755

解答. 先预处理一个 RMQ，从而能够 $O(1)$ 回答任意区间的最值和最值位置。`solve(l,r)` 计算 $[l,r]$ 内的答案。按照 $[l,r]$ 之内的最值所在位置 m 来分治，递归 `solve(l,m-1)` 和

`solve(m+1,r)` 来计算两边的答案。接下来需要处理跨过中点的区间 $[i,j]$。由于 a_m 是 $[l,r]$ 内的最值，$\max[i,j]=a_m$。枚举 $[l,m]$ 和 $[m,r]$ 中较短的那一侧的每个 i/j，不妨设为 $[l,m]$，则需要询问 $[m,r]$ 中有多少个 j 满足 $a_j \le a_m/a_i$。

可以使用可持久化线段树来在线地回答这些询问，但是更好的方法是**二次离线**。首先差分询问变成 $[1,m-1]$ 内的和 $[1,r]$ 内的数点，我们用一个 (l,m,t,c) 来表示：枚举所有 $i\in[l,m]$，查询 $[1,t]$ 内满足 $a_j \le a_m/a_i$ 的 j 的数量，将结果乘 c 后贡献到最终答案。这样的元组一共有 $O(n)$ 个，一共对应 $O(n\log n)$ 个询问。按照 t 排序后就可以扫描线树状数组做到 $O(n\log^2 n)$ 甚至 $O(n\log^2 n/\log\log n)$，而空间是线性的。

第一次离线是什么？

练习 6.8. 用前面标准序列分治的方法完成，需要做到同样的时空复杂度。

评注. 实际上，这类问题能用最值分治做的似乎总能用标准序列分治做，而反过来则未必。不过最值分治可能好想好写一点。

练习 6.9. P9607, CF1156E

例 6.10. CF475F. 平面上给定 n 个点，找到所有的极大点集满足：这些点集导出的矩形中，每一行和每一列都至少有一个点。

还有的时候，问题只会递归到一侧，或者递归到的子问题的总长度严格小于一个常数比例。此时复杂度通常是线性的。

例 6.11. 二分查找. 二分查找可以视为是 $T(V)=T(V/2)+O(\text{判定复杂度})$ 的分治。

例 6.12. 快速选择算法. 给 n 个数，找出它们的第 k 大。设计期望复杂度线性的算法。

练习 6.13. medians of medians. 把序列每五个分成一组，取出每组的中位数，递归求出这些中位数的中位数作为划分点。证明这样做的复杂度是 $O(n)$。

6.3　其他例子：多项式乘法和矩阵乘法

我们再给出两个有趣的例子。

例 6.14. Karatsuba 大整数乘法/多项式乘法. 需要计算 $A(x)=\sum_{i=0}^{n-1}a_ix^i$ 和 $B(x)=\sum_{i=0}^{n-1}b_ix^i$ 的乘积。令 $X=x^{n/2}$，则问题变成计算 $(A_1(x)X+A_0)(B_1(x)X+B_0)$。直接递归需要计算四个乘积 $A_{0/1}B_{0/1}$，复杂度仍然是 $T(n)=4T(n/2)+O(n)=\Theta(n^2)$。但是注意到我们只需要计算出 A_0B_0, A_1B_1 和 $(A_0+A_1)(B_0+B_1)$ 就可以表达出所需的全部四个乘积了。这样复杂度降低为 $T(n)=3T(n/2)+O(n)=\Theta(n^{\log_2 3})=O(n^{1.585})$。

评注. 这个神奇的构造可以认为是在**插值**：计算出 $(A_1X+A_0)(B_1X+B_0)$ 这个关于 X 的二次多项式的任意三个点值就可以解出所有系数了。据此可以使用更多次的多项式来把复杂度降低至 $n2^{O(\sqrt{\log n})}$，这就是 *Toom-Cook* 算法。更多细节参见*EI* 博客。

例 6.15. Strassen 矩阵乘法. 需要计算 n 阶矩阵 A, B 的乘积 C。将 A, B 划分成四个 $n/2$ 阶的子矩阵，那么直接递归需要计算八个乘积 $A_{0/1}B_{0/1}$，复杂度仍然是 $T(n) = 8T(n/2) + O(n^2) = \Theta(n^3)$。但是 Strassen 找到了构造：

$$
\begin{aligned}
M_1 &= (A_{0,0} + A_{1,1}) \times (B_{0,0} + B_{1,1}) \\
M_2 &= (A_{1,0} + A_{1,1}) \times B_{0,0} \\
M_3 &= A_{0,0} \times (B_{0,1} - B_{1,1}) \\
M_4 &= A_{1,1} \times (B_{1,0} - B_{0,0}) \\
M_5 &= (A_{0,0} + A_{0,1}) \times B_{1,1} \\
M_6 &= (A_{1,0} - A_{0,0}) \times (B_{0,0} + B_{0,1}) \\
M_7 &= (A_{0,1} - A_{1,1}) \times (B_{1,0} + B_{1,1})
\end{aligned}
$$

使得 C 可以被这些矩阵表示：

$$
\begin{aligned}
C_{0,0} &= M_1 + M_4 - M_5 + M_7 \\
C_{0,1} &= M_3 + M_5 \\
C_{1,0} &= M_2 + M_4 \\
C_{1,1} &= M_1 - M_2 + M_3 + M_6
\end{aligned}
$$

这样复杂度降低为 $T(n) = 7T(n/2) + O(n^2) = \Theta(n^{\log_2 7}) = O(n^{2.81})$。

评注. 可以证明矩阵乘法的最优复杂度为 $n^{\omega+\epsilon}$，其中 $\omega = \inf \frac{\log r(\langle n,n,n\rangle)}{\log n}$ 是一个**常量**。目前，人们知道 $\omega < 2.371339$。矩阵乘法的复杂度是一个深刻的问题，参见 *wiki*。

我猜 $\omega = 2$。

6.4　偏序数点/CDQ 分治

在前一章中我们提到过，可以用分治来查询偏序范围最值乃至任意分治信息。我们在此给出做法。

不妨假设范围是 d 个正交半平面围出来的，也就是每个点是一个 d 维带权点 $\mathbf{x}_i$，每次查询 $\mathbf{x}_i \le \mathbf{y}$ 的所有点的分治信息和。

由于这里没有顺序，需要分治信息可交换

我们对第一维进行分治：`solve(l,r)` 处理所有 $x_1, y_1 \in [l, r]$ 的那些点与查询之间的贡献。那么 `solve(l,r)` 可以拆成三部分：`solve(l,m)`，`solve(m+1,r)`，以及处理所有 $x_1 \in [l, m]$，$y_1 \in [m+1, r]$ 的那些点与查询之间的贡献。注意到第三部分变成了一个相同规模但是维数减一的问题，可以递归到维数减一的问题。来算一下复杂度：设 $T(d, n, V)$ 是维数为 d，规模为 n，值域大小为 V 时 `solve` 的复杂度，那么 $T(d, n, V) \le T(d, n_l, V/2) + T(d, n_r, V/2) + T(d-1, n, V_0) + O(n)$。

这里的初值是关键的。考虑 $d = 1$ 的情况，如果直接按照分治的方法，那么我们需要 $O(n \log V)$ 的时间；而如果事先排好序了，那么就只需要 $O(n)$ 的时间了。所以，取决于点是否已经排好序了，$T(1, n, V)$ 为 $O(n)$ 或者 $O(n \log n)$。$d = 2$ 的时候则是标准的二维数点，可以通过上一章中的扫描线方法做到 $O(n \log n)$。不过，我们在这里也介绍一种纯分治的方法。由于当点排好序时，$T(1, n, V) = O(n \log n)$，我们可以让 `solve(l,r)` 顺便支持将其内

的所有点和询问按照第二维排序。这样，在递归处理完两个子问题之后，就可以 $O(n)$ 计算跨过中点的贡献，并且可以通过归并来 $O(n)$ 排序整个区间。这样我们也得到 $T(2,n,V) = T(2,n_l,V/2)+T(2,n_r,V/2)+O(n) = O(n\log V)$。对于更大的 d，使用前面的递归式即可算出 $T(d,n,V) = O(n\log^{d-1} V)$。另外，由于 V 总是可以通过离散化变成 $O(n)$，我们可以在 $O(n\log^{d-1} n)$ 的时间内解决这个问题。

评注. 当分治结构与询问无关时，将分治过程记录下来即可得到支持在线询问的数据结构，即树套树。所以，这个问题也存在 $O(\log^{d-1} n)$ 的在线做法，只不过这超出了大纲的范围。

在实践中，d 几乎总是不超过 3，更大的情况通常需要寻找其他方法。通常来说，分治方法在 $d = 2$ 时的子问题的处理是最关键的。

比如数点的分块 bitset $O(n^2/w)$，一般情况的 KD tree $O(n^{2-1/d})$

练习 6.16. 二维偏序. 计算一个序列的逆序对数量。

练习 6.17. 三维偏序. 给定序列 a,b,c，查询满足 $a_i \le a_j, b_i \le b_j, c_i \le c_j$ 的 (i,j) 的数量。

评注. 如果只是求总对数的话，有一个容斥的 $O(n\log n)$ 做法。

练习 6.18. 带修二维数点. 二维平面，支持插入或删除一个带权点 $\mathbf{x}$，查询所有 $\mathbf{x} \le \mathbf{y}$ 的点的分治信息和。

练习 6.19. 全点集矩形加矩形求和. 维护一个二维数组，支持矩形加，查询矩形和。

评注. 如果不是全点集，那么这个问题是做不了的。实际上，行加列和就已经有矩阵乘法归约了。

练习 6.20. P3157. 每次删除一个数，查询剩下的序列的逆序对数

练习 6.21. P4169. 二维平面，支持插入一个点，查询离一个点最近的点到它的距离，距离为曼哈顿距离 $|\Delta x| + |\Delta y|$。

6.4.1　半在线

我们的分治过程事实上可以支持一种比较弱的在线操作：假设点和询问初始全部已知，且在第一维上有序，但点权不是在最开始即给定的，而是在回答询问后才给出。比如说，给定 $(1,\mathbf{x}_1)$ 的点权，在回答询问 $(2,\mathbf{y}_2)$ 后再给定 $(3,\mathbf{x}_3)$ 的点权，然后再在回答询问 $(4,\mathbf{y}_4)$ 后给定 $(5,\mathbf{x}_5)$ 的点权，以此类推。由于计算跨过中点的贡献时只需要左边的点权全部已知，我们可以先递归左边，然后处理跨过中点的贡献，再递归右边。这在 OI 里通常被称为 **cdq 分治**。

OI 中最早是陈丹琦推广的。但是没有人正式定义过 cdq 分治到底指的是什么。

练习 6.22. P2487. 二维偏序意义下的 LDS，还需要计数并计数每个点在多少种方案中出现。

练习 6.23. P4093.

练习 6.24. 在线卷积/分治 FFT. 假设有一个支持 $O(n\log n)$ 计算多项式乘法的黑盒子，设计一个算法来在线地计算多项式乘法。也就是说，给定两个多项式 A, B，初始仅有 A_0 和 B_0 是已知的，计算出 $C_i = \sum_{j=0}^{i} A_j B_{i-j}$ 后才能获取 A_{i+1} 和 B_{i+1}。

评注. 实际上，由于我们可以 $O(n\log n)$ 计算 DFT，这个问题可以做得更好，参见*EI 博客*。

6.5　整体二分

通过批量地二分来复用预处理的结果，以降低复杂度。

例 6.25. 区间 kth. 查询区间内第 k 小的值。

解答. `solve(l,r,Q)` 实现：对于 Q 中的每个询问 $[L, R, k, k_0]$，已知其结果在答案 $[l, r]$ 之间，且 $[L, R]$ 内小于 l 的数有 k_0 个，需要回答所有询问。

我们对每个询问判断其答案是否 $\le m = (l + r)/2$，并求出新的 k_0，从而递归到子问题。那么只需要计算 $[L, R]$ 内有多少个数在值域 $[l, m]$ 中。将每个值在 $[l, m]$ 中的数视为 1，其他数视为 0，则问题变为一维数点。

由于我们希望在 $\tilde{O}((r - l) + |Q|)$ 的复杂度内完成上面的过程，这个一维数点需要离散化或者使用数据结构。直接做带 log，但是我们可以在递归的过程中顺便离散化，即同时传入当前值域对应的那些位置 A，并在递归过程中将询问的 L, R 的规模降至 $|A|$。这样，复杂度就是 $T(V, |Q|, |A|) = T(V/2, |Q_l|, |A_l|) + T(V/2, |Q_r|, |A_r|) + O(|A| + |Q| + V) = O((|Q| + |A|)\log V)$ 了。

评注. 如果不做离散化，那么分治结构也是与询问无关的。此时，这种做法对应的在线数据结构是权值线段树套前缀和，想要做到 $O(\log n)$ 查询则需要分散层叠来避免多次二分定位 L, R。

常见的在线做法则是前缀和套权值线段树，即在可持久化线段树的两个版本的差分上二分。

练习 6.26. 查询区间内值域在 $[x, y]$ 中的第 k 小值。

练习 6.27. 带修区间 k 小。

例 6.28. P4602. 支持：插入删除一条带权树链，查询不包含某个点的树链的最大权值

做法一. 维护每个点的答案，需要支持链补范围的插入删除，查询单点 max。

做法二. 二分答案，需要判断是否存在一条链不包含该点。可以通过链加减、查单点来实现。

做法三. 注意到做法二中，只需要查询链交是否包含某个点。可以之间在线段树上二分。

例 6.29. P3527. 环形序列 a，序列上每个位置有其所属的国家。支持 a 区间加，对每个 i 计算第 i 个国家拥有的所有位置上的 a 的和最早在何时达到 p_i。

例 6.30. P5163. 支持删除一条边，维护所有强连通分量。

6.6　等价类分治

处理一切范围修改查询问题。大体来说，如果这个问题放在序列区间上可以用标准的线段树完成，那么就可以扩大到任意**范围**上。问题可以被形式化表述如下：

传说中的 TB5。

定义 6.31. 范围修改查询. 有 n 个带权点 a 和一个点的子集族 $\mathcal{S}$，支持：查询某个 $S \in \mathcal{S}$ 内的分治信息和 f，对某个 $S \in \mathcal{S}$ 内的点进行批量修改 Φ，且能够进行快速的

信息合并　可以根据 $f(a;S_1)$ 和 $f(a;S_2)$ 快速计算 $f(a;S_1 \uplus S_2)$

修改合并　可以快速计算 $\Phi_1 \circ \Phi_2$

修改分裂　$S = S_1 \uplus S_2$ 上的修改 $\Phi(a;S)$ 可以快速分解为子集 S_1 和 S_2 上的修改 $\Phi(a;S_1)$ 和 $\Phi(a;S_2)$

应用修改　可以根据 $f(a;S)$ 快速计算 $f(\Phi(a;S);S)$

我们首先对于给定的修改与查询序列定义点的等价关系与等价类。

定义 6.32. 等价关系与等价类. 如果两个点 u,v 满足：涉及到 u 的修改/查询与涉及到 v 的修改/查询完全相同，那么我们称 u,v 是等价的。

显然，同一个等价类内的点可以缩到一起。这样，一次修改与查询的复杂度就只与等价类的数量有关。而修改与查询的数量越少，等价类的数量也就越少。这样，我们可以设计一个关于时间的分治过程。记 $\mathcal{C}[l,r]$ 为 $[l,r]$ 时间内的修改与查询导出的所有等价类。如果我们只需要做 $[l,r]$ 时间内的修改与查询，那么我们就只需要关注 $\mathcal{C}[l,r]$ 中的每个等价类的信息和修改情况。

对于每个等价类 $C_i \in \mathcal{C}[l,r]$，我们希望在 $\texttt{solve}(l,r)$ 开始前就知道其总信息 $f(a;C_i)$，并且在函数结束知道 $[l,r]$ 这段时间内的修改在其上的总修改 $\Phi(a;C_i)$。自然，我们需要利用 $\texttt{solve}(l,m)$ 和 $\texttt{solve}(m+1,r)$ 来完成。由于子区间的等价类集合相当于 $\mathcal{C}[l,r]$ 中的等价类进行一些合并得到的，当我们想要递归进某一边的之前，就可以对 $\mathcal{C}[l,r]$ 内的等价类做一些合并来得到 $\mathcal{C}[l,m]$ 或者 $\mathcal{C}[m+1,r]$，并且做相应的信息合并。当递归完 $[l,m]$ 或者 $[m+1,r]$ 回到 $[l,r]$ 时，我们需要重新分裂先前合并的等价类，并将 $\mathcal{C}[l,m]$ 或者 $\mathcal{C}[m+1,r]$ 中每个等价类上的总修改标记分裂到 $\mathcal{C}[l,r]$ 中的对应等价类上。

算法 6.33. 等价类分治. $\texttt{solve}(l,r,\mathcal{C}[l,r])$ 实现以下功能：在函数开始前，需要保证 $\mathcal{C}[l,r]$ 即为 $[l,r]$ 中的修改查询导出的所有等价类，每个等价类 C_i 由其中的任一代表元 α_i 表示，且其总信息 $f(a;C_i)$ 已经计算完毕。在函数结束后，计算出 $[l,r]$ 内的所有修改在每个等价类 C_i 上的总修改量 $\Phi(a;C_i)$，以及修改后得到的新的总信息 $f(\Phi(a;C_i);C_i)$。

$\texttt{solve}(l,r,\mathcal{C}[l,r])$ 的具体实现如下。令 $m=(l+r)/2$。首先，通过**某种方法**计算出 $\mathcal{C}[l,m]$，以及每个 $C_i \in \mathcal{C}[l,r]$ 在 $\mathcal{C}[l,m]$ 中位于哪个等价类。据此可以利用每个 $C_i \in \mathcal{C}[l,r]$ 的总信息 $f(a;C_i)$ 来合并计算出每个 $C_j \in \mathcal{C}[l,m]$ 的总信息 $f(a;C_j)$。接下来，递归

`solve`$(l,m,\mathcal{C}[l,m])$ 计算出 $[l,m]$ 内的所有修改在每个 $C_j\in\mathcal{C}[l,m]$ 上的总修改量 $\Phi(a;C_j)$。然后，将每个 $\Phi(a;C_j)$ 分解到 $\mathcal{C}[l,r]$ 中对应的等价类上，从而得到每个 $C_i\in\mathcal{C}[l,r]$ 上的总修改 $\Phi(a;C_i)$。将每个 $C_i\in\mathcal{C}[l,r]$ 上的总修改 $\Phi(a;C_i)$ 应用到对应的信息 $f(a;C_i)$ 上即可得到 $[l,m]$ 内的所有修改执行完毕后，每个等价类 C_i 中新的总信息 $f(\Phi(a;C_i);C_i)$。然后同样地对 $[m+1,r]$ 执行以上操作。

假设 i 个范围至多导出 $P(i)$ 个等价类，算法中用于计算等价类划分的**某种方法**面对 n 个点和 m 个范围时具有 $D(m,n)$ 的复杂度，则分治的复杂度为

$$T(m)=2T(m/2)+O(P(m))+D(m/2,P(m)).$$

而在 `solve`$(1,m,\mathcal{C}[1,m])$ 开始前，我们还需要花费 $D(m,n)$ 的时间来计算 $\mathcal{C}[1,m]$，因此总时间为 $T(m)+D(m,n)$。

注意我们还可以对操作序列按 B 大小分块，这样总共就是 $\frac{m}{B}(T(B)+D(B,n))$ 的复杂度。

评注. 实践上，使用等价类分治时最困难的地方基本在于**某种方法**。如果忽略这个部分的话，可以证明这个算法的复杂度是理论最优的 $\Theta(m\cdot\max_{1\le i\le m}P(i)/i)$。

例 6.34. 常规线段树问题. m 个区间至多划分出 $\min(n,2m+1)$ 个等价类，且按端点排序后计算这些等价类只需要 $O(n+m)$ 的时间。所以在最外部排一次序后就有 $T(m)=O(m\log\min(n,m))$，$D(m,n)=O(n+m)$，总复杂度 $O(m\log\min(n,m)+n)$（而不是 $n+m\log n!$）

例 6.35. 矩形加矩形和. m 个矩形至多划分出 $O(\min(m^2,n))$ 个等价类。在外层将询问按照 $\sqrt{n}$ 分块，则可以假设 $m\le\sqrt{n}$。此时，在最外部将两维分别排序后，可以在线性时间内计算所有等价类（行等价类与列等价类的笛卡尔积）。所以 $D(m,n)=O(m^2+n)$，$T(m)=O(m^2)$，总共 $O(n+q\sqrt{n})$。

评注. 半平面加半平面求和、圆加圆求和也是相同的复杂度（忽略 log 因子）。

例 6.36. 树链加树链和. m 个树链可以按照虚树的方式划分出 $O(\min(n,m))$ 个等价类，所以是 $O(m\log\min(n,m)+n)$ 的。

在有位移操作（如区间翻转和平移）时，等价类分治仍然可以应用。只不过此时运行完一侧的修改后，除了每个等价类内部的总修改以外，还可能会发生等价类之间的重新排列。此时，可以先按照函数复合的方式自下而上地计算出等价类的划分，再运行标准的等价类分治。

例 6.37. 文艺平衡树.

例 6.38. *P7723. TB5x

7 分块与莫队

7.1 序列分块基础

序列分块的基本想法是把序列按照某个大小 B 分段（块）。任何一个区间 $[l,r]$ 在这个结构上要么是某个块的一个子段，要么是一个块的后缀加 $O(n/B)$ 个完整块加一个块的前缀。这种结构的好处在于，我们通过**重构**而不是合并来维护信息：在线段树中，当某个结点的子树内部发生了某些修改时，必须要能够通过某种方式正确地计算出此结点上新的信息；而在分块结构中，我们只要根据这个块对应的 $O(B)$ 长度的序列重新计算这个结点的信息就可以了，也就是**重构**。当然，对于整块的修改，还是需要能够快速地把修改标记应用到信息上。

根据是否存在修改可以将问题分为动态问题与静态问题。对于动态问题，我们通常要按顺序思考以下几个事情：

- 把块长设为**待定值** B。
- 如何查询一个完整块的答案？通常来说，这通过对每个完整块维护一个数据结构来实现。
- 如何查询一个零散块的答案？通常使用暴力，或者聪明的暴力。
- 如何在完整块修改的时候快速地更新完整块的数据结构？通常通过打某种标记实现。此外，也许需要改变第一条中的使用的数据结构，以保证在修改时可以快速维护。
- 如何在零散块修改的时候更新对应完整块的数据结构？通常通过**暴力重构**实现。
- 最后，计算你的复杂度，并选择最优的 B。

在实践中，块长 B 也可以通过实验的方式得出，并且这样得到的实际效率也许更高。相比分治数据结构，分块更加灵活，需要思考的事情也更多。我们先前和以后学到的各种思想和技巧也经常在分块中使用。

例 7.1. 区间加，查询区间内小于 x 的数的个数。

评注. 思考一下为什么线段树做不了这个。事实上，即使是区间 ±1，查询全局 0 的个数也是无法 $\mathrm{polylog}n$ 的。

解答.

- 查询完整块的答案：维护排序后的数组，查询时二分。$O(\log B)$ 查询一个块，总共有 qn/B 次查询。
- 查询零散块的答案：遍历即可。$O(B)$ 查询一个块，总共有 q 次查询。
- 完整块的修改：打一个加法标记，$O(1)$。容易发现这与第一条中的数据结构是兼容的。总共有 qn/B 次修改。
- 零散块的修改：重构。直接排序是 $O(B\log B)$ 的，但我们可以注意到可以复用先前排序的数组。把修改过和没修改过的数分开，归并这两类数即可做到 $O(B)$。总共有 q 次修改。
- 计算复杂度：$(qn/B)\log B+qB$。可以这样计算复杂度：首先，可以猜想 $B=n^{\Theta(1)}$，所以 $\log B=\Theta(\log n)$，这样复杂度就变成 $(qn/B)\log n+qB$。接下来，由于两个组成部分的单调性不同，其最优值在渐近复杂度的意义下必然在两部分相等时取到，从而只需

要解方程 $(qn/B)\log n = qB$，即 $B = \sqrt{n\log n}$。这样得到复杂度 $q\sqrt{n\log n}$.

评注. 有比较复杂的严格根号做法，但是不太实用。

例 7.2. P5356. 区间加，查询区间第 k 小。

解答. 首先可以在上一题的基础上套一层二分得到 $O(\sqrt{n\log n}\log V)$ 的做法.

注意到二分的时候并不需要每次都花费 B 的代价查询散块，而是可以先将散块归并成一个临时的整块，查询时在上面二分。这样散块的代价就从 $B\log V$ 降到 $B+\log B\log V$，总复杂度 $O(\sqrt{n\log n\log V})$。

练习 7.3. P3863. 区间加，查询某一个位置在过去的多少个时间内 $\le y$（y 每次不同）

例 7.4. 行加列和. 给定 n 个点，支持把 $x_i \in [X_l, X_r]$ 的点的权值加上一个数，查询 $y_i \in [Y_l, Y_r]$ 的点权和。

解答. 不失一般性，可以假设所有的点的 x, y 都不同，并离散化询问与修改区间。按 x 分块。

- 查询：枚举每个块，查询此块内 y 在 $[Y_l, Y_r]$ 的点的权值和。这可以差分后用前缀和 $s_{i,j}$ 完成：$s_{i,j}$ 表示第 i 个块内 y 排名前 j 的点的权值和。为了 $O(1)$ 得到 j，还需要预处理一个数组 $\mathrm{rank}_{i,y}$ 表示第 i 个块内 $y_j \le y$ 的点的个数。
- 完整块修改：打一个加法标记。在询问时，还需要额外加上标记值乘 $\mathrm{rank}_{i,y}$。
- 散块修改：重算 s_i 即可。

这个做法需要 $O(n\sqrt{n})$ 的空间。

压空间. 由于不同块的修改和查询独立，可以离线后逐块处理，这样只需要存储一个块的 rank_y，空间降至线性。

另一种方式是对 y 也分块，查询时单独查询散块的答案。

另解. 是上一个做法的对偶。按行列分别分块。差分掉修改。

- 查询完整 y 块的答案：维护 y 在此块内的点的权值和
- 查询零散 y 块的答案：暴力。
- 前缀的若干个完整 x 块的修改：首先对每个 x 块打一个标记表示此块上有一个修改（用于零散 y 块的查询），然后考虑如何更新 y 块维护的答案。对每个 y 块，预处理一个系数 $C_{y,x}$，表示当前 x 个 x 块发生了加一的时候，此 y 块的答案会变化多少。据此更新所有 y 块的答案即可。
- 零散 x 块的修改：暴力修改点权。

练习 7.5. CodeChef Chef and Churu. 维护一个序列 a 和一个函数列 f，每个函数都是 a 的一个区间和。支持：序列的单点修改；查询函数列的区间和。

例 7.6. 区间众数. 查询区间内出现次数最多的数的出现次数。

解答. 首先用 $O(n^2/B)$ 的时间计算出数组 $M_{x,y}$ 表示第 x 到第 y 个块中出现次数最多的数的

出现次数。查询时，还需要枚举散块中的数判断能否更新答案。具体来说，每次需要做 $O(B)$ 次判断：值 w 在 $[l,r]$ 中的出现次数是否至少是 t。聪明的办法是：对每个值维护一个 `vector` 存储其所有出现，则只需要找到目前枚举的 w 在这个 `vector` 中出现的位置 p，然后判断 `vector` 的 $p+t$ 位置是否 $\le r$ 即可。

例 7.7. 区间逆序对. 查询区间内逆序对的数量。

解答. 首先用 $O(n^2/B)$ 的时间计算出数组 $M_{x,y}$ 表示第 x 到第 y 个块中的逆序对数量。查询时，还需要加上散块的贡献。聪明的办法是：首先处理一个散块内部的贡献，这可以预处理；然后处理两个散块之间的贡献，这可以通过归并实现；最后处理散块与完整块之间的贡献，这相当于 B 次查询：第 x 到第 y 个块中有多少个数 $<w$，可以差分后预处理。

但是这还遗漏了一种情况：询问区间完全被包含在某个块中。这个情况可以通过差分变成两个散块前缀之间的询问。

评注. 未知是否存在 $\tilde{O}(n)$ 空间的做法，但是已知使用 $O(1)$ 二维数点来回答散块询问的话一定做不到 $\tilde{O}(n)$ 空间。

评注. 以上两个问题可以视为在线莫队。

7.2　复杂度平衡

线段树可以实现 $O(\log n)$ 的单点修改和区间查询。然而，有时修改和查询的数量是不同的。此时，可以使用多叉树来降低某一侧的复杂度而提高另一侧的复杂度，从而平衡整体的复杂度。

例 7.8. 使用 B 叉树分别实现：

- $O(B\log_B n)$ 单点加，$O(\log_B n)$ 查询区间和。
- $O(\log_B n)$ 单点加，$O(B\log_B n)$ 查询区间和。

练习 7.9. 对偶. 实现

- $O(B\log_B n)$ 区间加，$O(\log_B n)$ 查单点。
- $O(\log_B n)$ 区间加，$O(B\log_B n)$ 查单点。

练习 7.10. 差分. 实现

- $O(B\log_B n)$ 区间加，$O(\log_B n)$ 查询区间和。
- $O(\log_B n)$ 区间加，$O(B\log_B n)$ 查询区间和。

练习 7.11. 分别代入 $B=\log n$，$B=\sqrt{n}$ 和 $B=n^\epsilon$，观察以上几个问题的复杂度。

练习 7.12. 分别实现数据结构，支持

- $O(V^\epsilon/\epsilon)$ 插入删除，$O(1/\epsilon)$ 查询 $<x$ 的数的数量（即 x 的排名）。
- $O(1/\epsilon)$ 插入删除，$O(V^\epsilon/\epsilon)$ 查询 $<x$ 的数的数量。

例 7.13. 使用 B 叉树实现：$O(B^2 \log_B n)$ 单点修改，$O(\log_B n)$ 查询区间分治信息和（假设分治信息可以 $O(1)$ 合并）。提示

评注. 在修改的时候，还需要能够重新计算每个结点的 B 个孩子的总信息，这往往使得修改难以优于 $B \log_B n$。这篇文章证明了对于一般的半群信息，在常规的模型下，如果修改复杂度是 $O(f(n))$，那么无法做到 $\frac{n}{e^{O(f)}}$ 的全局查询。

不过，在信息比较特殊的情况下，还是有可能利用位运算之类的 *word RAM* 特性加速的，例如 $O(\log n / \log w)$ 的单点加一查询区间和，以及先前见到的线性 *RMQ*。此外，通过非常复杂的方法还可以做到 $O(\log n / \log \log n)$ 的单点修改 *RMQ*。

例 7.14. 维护一个多重集，分别支持

- $O(\log_B V)$ 插入删除，$O(B \log_B V)$ 查询最大值。
- $O(B \log_B V)$ 插入删除，$O(\log_B V)$ 查询最大值。

例 7.15. 维护一个**多重集**，分别支持

- $O(\log_B V)$ 插入删除，$O(B \log_B V)$ 查询第 k 小。
- $O(B \log_B V)$ 插入删除，$O(\log_B V)$ 查询第 k 小。

解答. 对于前者，在 B 叉树上类似二分地做即可。

对于后者，在 B 叉树的每个结点上维护所有第 k 小所在的孩子。修改时只需要更改 B 个位置。

例 7.16. 维护一个序列，分别支持

- $O(B \log_B n)$ 单点修改，$O(\log_B n)$ 查询区间最值。
- $O(\log_{B_1} n \log_{B_2} V)$ 单点修改，$O((B_1 \log_{B_1} n)(B_2 \log_{B_2} V))$ 查询区间最值。（这没有那么简单）

解答. 值域树套区间树。

7.3　莫队

莫队算法可以视为一般的扫描线：我们用数据结构维护当前的某个集合 S，通过加删元素来在不同的 S 之间转移。形式化的描述如下：

定义 7.17. 莫队. 给定一个集合族 $S_1, \cdots, S_m$。你有一个初始为空的集合 S，每次可以进行以下操作之一：

- 向 S 中添加一个元素
- 删除 S 中的某个元素
- 报告 i 表示当前的 S 是 S_i。
- 结束操作

需要保证在结束操作前，每个 S_i 都被正确报告过。莫队算法的目标是构造一个尽量短的操

作序列。

莫队问题有一些常见的变体：

定义 7.18. 不删除莫队/回滚莫队. 在莫队问题中，不允许删除元素，而是改为允许撤销上一次添加的元素。

定义 7.19. 在线莫队. S_i 变成在线给定的，而操作额外允许记录某一个版本以及转移到某个记录的版本上。

例 7.20. 前缀莫队/扫描线. 每个 S_i 是一个前缀 $[1,i]$。

评注. 这自然是不删除莫队。

接下来，我们给出常见的几种莫队构造。

例 7.21. 区间莫队. 每个 S_i 是一个区间 $[l_i,r_i]$。

解答. 将序列按照 B 分块，按照 $(l_i$ 所在块$,r_i)$ 排序。对于 l_i 在同一个块内的询问，转移代价不超过 B；l_i 所在块至多变化 n/B 次，每次变化的代价至多为 n，所以总代价不超过 $qB+n^2/B$。取 $B=n/\sqrt{q}$ 时得到最优复杂度 $O(n\sqrt{q})$。

评注. 注意这里的块长并不是常规的 $\sqrt{n}$。事实上，如果取 $B=\sqrt{n}$ 的话可以得到 $(n+q)\sqrt{n}$ 的复杂度，这比 $n\sqrt{q}$ 严格更差。

练习 7.22. 给出在线不删除莫队的构造。

例 7.23. 单点补莫队. 每个 S_i 是元素 i 的补集。

练习 7.24. 说明区间补莫队与区间莫队等难。

例 7.25. 单点补不删除莫队/线段树分治. 同上，但是需要不删除莫队。

解答. DFS 线段树，进入一个结点的时候将其兄弟结点对应的区间加入，退出结点的时候撤销。

练习 7.26. 假设 $K=O(1)$，构造所有长度不超过 K 的区间的补集的不删除莫队。

例 7.27. 对偶区间莫队/线段树分治. 每个元素是一个区间，而 S_i 是所有包含 i 的区间的集合。

解答. DFS 线段树，进入一个结点后保证所有覆盖此结点对应区间的区间都在集合中。则把所有区间在线段树上分解，DFS 到一个结点时加入此结点上的区间即可。

可以一边 DFS 一边分解区间来将空间降到线性。

练习 7.28. 对于允许离线的问题，可以通过这种方式将删除变为撤销。一个经典的例子是**动态图连通性**，即加删边，查询两点连通性。

练习 7.29. 证明不删除莫队的代价至多比普通莫队多一个 log。

例 7.30. 子树莫队/dsu on tree. 给定一棵树，每个 S_i 是一个子树。

练习 7.31. 函数 `solve(u,0/1)` 表示构造出 u 子树内所有子树对应的操作序列，且函数结束后是否保留此子树内点，0/1 分别对应不保留/保留。首先树链剖分，然后调用 `solve(root,1)`。$\texttt{solve}(\texttt{u},0/1)$ 具体实现如下：首先递归所有轻儿子 $\texttt{solve}(\texttt{v}_1,0)$，然后递归重儿子 $\texttt{solve}(\texttt{v}_\texttt{h},1)$，最后加入 u 及其所有轻子树。如果不保留 u 子树内的点，则撤销 $\texttt{size}_\texttt{u}$ 次即可。总复杂度为所有点的轻子树大小之和，这是 $O(n\log n)$ 的。

定理 7.32. 莫队之理. 设 $P'(i)$ 为 i 个点能够划分出的最多询问等价类数量，则莫队的最优复杂度是 $\tilde{\Theta}(n\max_i P'(i)/i)$。这个最优复杂度通过可以如下实现近似：找到 x 使得 $P'(x)\approx m$，随机选取 x 个点，对每个集合 S_i 构造 01 向量 B_i，其中 $B_i(j)=[j\in S_i]$。按照这些向量的字典序排序即可得到一个合适的顺序，此顺序可以保证总代价高概率不超过最优代价乘 $\log n\log m$。

证明. 道理比较复杂，在此省略。可以参考 critnos 的博客。

□

定理 7.33. 对于在线莫队，只需在每个等价类中任取一个集合加入需要预处理的集合即可。在同一个等价类内，无论如何转移，复杂度都是正确的。

在完成了莫队操作序列的构造之后，我们就可以使用数据结构来维护这些操作了。对于静态问题，只要不强制在线，莫队就总是好于分块。

例 7.34. 小 Z 的袜子. 查询区间内 $a_i=a_j$ 的点对数量。另一种表述是每种数出现次数的平方和。

评注. *存在基于矩阵乘法的 $O(n^{\frac{2\omega}{\omega+1}})$ 做法*

例 7.35. 区间众数. 查询区间内出现次数最多的数的出现次数。

评注. *存在 $O(n^{1.49})$ 做法。*

例 7.36. T142605. 查询区间内出现次数第 k_1 小的数里面的第 k_2 小。

例 7.37. 区间逆序对. 查询区间内逆序对的数量。

解答. 使用区间莫队，转移时需要查询当前集合内有多少个 $a_i<x$。直接用树状数组做是 $O(n\sqrt{m}\log n)$ 的。

注意贡献可以差分，即每次都询问 $[1,r]$ 内有多少个 $a_i<x$。这样问题变成在线的 $O(n\sqrt{m})$ 次二维数点，可以通过可持久化的 B 叉树做到 $O(1)$ 询问。

注意到并不需要真的在线。每一批贡献都形如

$$\sum_{i=r+1}^{r'}([l,i-1]\text{ 内满足}a_j<a_i\text{ 的数的数量}).$$

差分后，就变成

$$\sum_{i=r+1}^{r'}([1,l-1]\text{ 内满足}a_j<a_i\text{ 的数的数量})$$

和

$$\sum_{i=r+1}^{r'}([1,i-1]\text{ 内满足}a_j<a_i\text{ 的数的数量}).$$

对于前者，可以用一个 $(r,r',l-1)$ 的元组来表示一批贡献；对于后者，可以用一个 (r,r') 的元组来表示一批贡献。对于这 $O(m)$ 个元组重新离线即可去掉可持久化。

评注. 存在在线的基于矩阵乘法的 $O(n^{1.41})$ 做法。

例 7.38. P4869. 给一棵树，记 S_u 为 u 的子树，查询 $\sum_{i\in S_u}\sum_{j\in S_v}[a_i=a_j]$。

评注. 这是伪双子树莫队。真双子树莫队是 *P9988*。

例 7.39. 查询区间最小差. 查询区间内最小的 $|a_i-a_j|$（$i\neq j$）

评注. 这个其实可以 *polylog*，参见 *CF765F*。

例 7.40. k 区间颜色数. 给一个序列，每次给出 k 个区间，求 k 个区间的并内有多少个种出现过的值。$O(nm/w)$

7.4　常用技巧

这里是一些常用的技巧，但是暂时没有合适的例题来展示。

7.4.1　逐块处理

如果可以离线，并且块之间的贡献相对独立，那么将所有修改和查询离线后**逐块处理**总是更好的选择。

7.4.2　操作分块/定期重构

如果操作变多时处理的代价变大（如 B^2），但是重构的代价不随操作数量增加，那么可以考虑每处理一批操作就进行一次重构。

7.4.3　分块后分治

与上一条类似，如果一个长为 n 的序列上直接做会带来较高的代价（如 n^2），但是可以把序列分成对询问相对独立的块，那么可以考虑先对序列做一次分块。

例 7.41. P5611.

8 常见特殊性质

8.1 均摊

有时候，虽然某些操作的单次复杂度可能很高，但这种操作做完之后会使得数据结构的状态变得更好，从而所有操作总共的复杂度并不会很高。我们首先回顾单调栈的例子：

例 8.1. 单调栈. 虽然一次插入可能需要删除多个元素，但这也会导致栈中的元素减少。而栈中总共只插入了 n 次，所以删除的总次数不会超过 n，也就是说每次操作的平均复杂度是 $O(1)$。

这种想法可以发展为**势能分析**。我们寻找一个关于当前**数据结构状态**的势能函数 $\Phi(S)$，表示当前状态的“坏的程度”。假设执行完前 i 次操作后，数据结构的状态为 S_i。当我们执行第 i 次操作时，会产生一个实际代价 T_i；同时，这个操作会导致数据结构的势能从 $\Phi(S_{i-1})$ 变成 $\Phi(S_i)$。我们定义 $\hat{T}_i = T_i + \Phi(S_i) - \Phi(S_{i-1})$ 为第 i 次操作的**均摊代价**，则

$$\begin{aligned}\sum_{i=1}^{n} T_i &= \sum_{i=1}^{n} \hat{T}_i - \sum_{i=1}^{n} \Phi(S_i) - \Phi(S_{i-1}) \\ &= \Phi(S_0) - \Phi(S_n) + \sum_{i=1}^{n} \hat{T}_i \qquad \leq \Phi(S_0) + \sum_{i=1}^{n} \hat{T}_i.\end{aligned}$$

也就是说，只要我们能够说明数据结构的均摊代价之和不高，且数据结构的初始势能不高，就能说明实际代价之和也不高，即总复杂度不高。

练习 8.2. 单调栈. 定义 $\Phi(S)$ 为当前栈中元素的个数。

练习 8.3. 双端队列. 定义 $\Phi(S)$ 为当前双栈两侧元素个数的差的绝对值。

例 8.4. 开方、除法. 支持：区间加、区间开方（下取整）、区间除法（下取整），查询区间和与区间 max。

解答. 对于在线段树分解出的一个终止区间，如果极差足够小，那么区间除法和开方可以变成区间减法；否则暴力递归。

除法/开方至少会使得极差减半/开方。定义每个结点的势能为 log 极差 即可。

例 8.5. 取模. 支持：单点修改、区间对某个数取模，查询区间和与区间 max。

解答. 如果 $\max \geq x$ 那么递归，否则取模后无变化。

注意如果取模后发生变化，那么这个数至少会减半。因此每个数在两次单点修改之间只会被有效修改 $O(\log V)$ 次。

评注. 这个问题加上区间加的话就不能 *polylog* 了，可以用区间加区间 $< x$ 的数的个数归约：根据 a 是否 $< x$，$(a + M) \bmod (M + x)$ 的取值分别为 $a + M$ 和 $a - x$。

练习 8.6. P3747.

例 8.7. Segtree Beats. 单点修改，区间和 x 取 min，查询区间和。

解答. 维护每个区间的最大值和严格次大值。对于线段树上分解出的一个终止区间，如果修改不会影响严格次大值，那么只需要将所有最大值都减去一个数；否则，递归处理。

定义一个结点的势能为区间内不同的数的个数。

练习 8.8. 额外支持区间和 x 取 min。

例 8.9. Segtree Beats 2. 用相同的方法额外支持区间加。

解答. 定义一个结点的势能为 $[\max \text{ 与父亲不同}] \cdot \text{depth}$。

例 8.10. 连续段均摊/颜色段均摊/珂朵莉树/ODT. 如果序列可以被分成若干连续段，满足：

- 每次操作只会导致连续段数量增加 $O(1)$
- 一个需要访问多个连续段的操作会保证在此次操作结束后把这些连续段合并成一个

那么整个过程中访问的连续段总数是 $O(n+m)$。

这里放不下这么多算法。

评注. 最常见的情况是区间赋值，此时可以把每个极长等值段视为一个连续段。

例 8.11. CF444C. 区间染色，每次染色后会给对应位置的权值加上新旧颜色差的绝对值；查询区间权值和。

解答. 用 `set` 之类的东西维护极长同色段，另一个数据结构维护每个位置的权值。区间染色时，先在 `set` 上提取出 $[l,r]$ 对应的那些段（可能需要分裂 l,r 所在的段），每个段在维护权值的数据结构上修改，然后把这些段合并成一个。

例 8.12. P8512. 区间赋值，查询只考虑一段时间内的修改时某个区间的和。

解答. 使用连续段均摊把区间赋值变成区间加。

8.2　特殊函数

像区间内所有数的按位或、按位与、gcd 这样的函数具有特殊的性质：当左/右端点固定时，$f(l,r)$ 至多变化 $\log V$ 次。因此，可以像单调栈一样在扫描 r 的过程中维护所有的 $f(l,r)$。

```
// O(n log V)
for(int i=1;i<=n;i++)
{
    f[i]=a[i];
    for(int j=i-1;j>=1;j--)
```

```
    {
        if((f[j]&a[i])!=f[j])
            f[j]=f[j]&a[i];
        else break;
    }
}
```

例 8.13. 查询最短的按位或和 $\geq x$ 的区间长度，x 每次不同。

解答. 对每个 l，$f(l,r)$ 关于右端点 r 形成至多 $\log V$ 个等值连续段，每个连续段只有第一个位置可能是最短的。所以只需要计算出所有这样的 $O(n\log V)$ 个区间，然后简单查询即可。

不难发现这样的区间就是上面的暴力中 if 条件成立的那些 $[j,i]$。

练习 8.14. P5065. 额外支持单点修改。

例 8.15. P9335. 查询区间内所有子区间的按位或、按位与、gcd 的乘积的和。

解答. 类似扫描线维护历史和，对每个 l 维护 f_l 表示子区间左端点固定为 l 时乘积的历史和，扫到 r 时查询 f 的后缀和即可。由于大部分时候乘积不发生变化，可以将 f_l 设为一次函数 k_lr+b_l，当乘积变化时更新 k_l 和 b_l 即可。由于每次只会修改 f 的一个后缀，可以顺便维护 f 的前缀和来回答询问。

8.3　按阈值分类讨论/阈值分治

在很多问题中，会有这样的情况出现：代价大的东西个数很少，而大部分东西代价很小。这时候可以将这两部分分开处理。

8.3.1　约数、倍数、取模

注意 x 的倍数不超过 V/x 个，x 的约数不超过 $d(n)=o(\sqrt{x})$ 个。所以，在处理约数、倍数和取模相关的问题时，经常把 $\leq\sqrt{V}$ 和 $>\sqrt{V}$ 的除数分开处理。

实际上有 $d(n)=n^{O(1/\log\log n)}$。

例 8.16. 等差数列位置加，查询单点。

解答. 如果 $d\geq\sqrt{n}$，那么暴力加就是 $O(\sqrt{n})$。否则，按照 d 分类。对于一个 d，按照 $i \bmod d$ 分类可以形成 d 个长为 n/d 的数列。一次修改相当于在某个 d 的某个数列上区间加，查询相当于询问每个 d 上的某个序列的单点。这样是 $O(q)$ 次修改和 $O(q\sqrt{n})$ 次询问，平衡一下即可。

例 8.17. P5309 强化版. 等差数列位置加，区间和

解答. 对于 $d>\sqrt{n}$ 的修改，相当于 $O(q\sqrt{n})$ 个单点加和 $O(q)$ 个区间和。

考虑 $d \le \sqrt{n}$ 的修改。对于每个 d，把序列按照 d 分块，那么询问在一个 d 上体现为若干整块询问和最多两个散块询问。一个修改在整块询问上的影响相当于区间加，在散块上相当于在 (块间坐标，块内坐标) 这些点上取出 $x \le r, y = b$ 的部分做加法。一个整块询问查询区间和，散块询问则查询 $x = i, y \le b$ 的点的权值和。所以整块的部分就是 $O(q)$ 个区间加和 $O(q\sqrt{n})$ 个区间和，散块的部分可以变成 $O(q)$ 个点和 $O(q\sqrt{n})$ 次二维数点。

评注. 等差数列位置加、等差数列位置求和暂时不知道是否有 $\tilde{O}(\sqrt{n})$ 的做法，但是 $\tilde{O}(n^{2/3})$ 是比较容易的。此外，如果是没有左右端点限制的等差数列位置修改查询，那么存在使用多项式乘法的 $\tilde{O}(\sqrt{n})$ 做法。

例 8.18. P3591. 树链上等差数列位置求和

例 8.19. P9809. 插入一个数，查询 $\bmod x$ 的最小值

例 8.20. P5071. 查询区间乘积的约数个数

8.3.2　度数

在一张图中，由于所有的点的总度数为 $2m$，所以度数 $> \sqrt{2m}$ 的点的个数不超过 $\sqrt{2m}$，而其他的点的度数都不超过 $\sqrt{2m}$。

例 8.21. 单点加一圈和. 给一张图，支持：把一个点的点权加一个数，查询一个点的邻居的点权之和。

解答. 查询点 u 时，如果 $\deg_u \le \sqrt{2m}$，那么暴力就是 $O(\sqrt{m})$ 的。对于 $\deg > \sqrt{2m}$ 的 $O(\sqrt{m})$ 个点，可以单独维护。那么在修改 u 时，需要更新 u 的那些 $\deg > \sqrt{2m}$ 的邻居的答案，这可以通过在建图时就计算出这些邻居的列表来完成。

例 8.22. 三元环计数. 给一张无向图，计算三元环的数量。

解答. 首先我们有一个 $\sum_u \sum_{v \in N(u)} \deg_v = \sum_v \deg_v^2$ 的做法：枚举一个点 u，标记其所有邻居；然后枚举 u 的邻居 v，再枚举 v 的邻居 w，如果 w 被标记了，那么 u, v, w 就构成一个三元环。此外，每个三元环在这个算法中都会被枚举到三次。

不过我们可以做得更聪明一点:如果可以把枚举 v 的邻居的代价从 $\deg_v$ 降低至 $\min(\deg_v, \sqrt{2m})$，那么总复杂度就可以降到 $O(m\sqrt{m})$。这可以通过强制 $\deg_u \le \deg_v \le \deg_w$ 完成。或者说，我们把所有点按照度数排序，然后把每条无向边定向，使得每条边都是靠前的点连向靠后的点。这样，当 $\deg_v > \sqrt{2m}$ 时，对应的 w 必须满足 $\deg_w > \sqrt{2m}$，而这样的点数不超过 $\sqrt{2m}$。在这个算法中，每个三元环恰好被枚举到一次。

评注. 可以用矩阵乘法做到 $O(n^\omega)$。

练习 8.23. 计算有向图三元环数量。

8.3.3　出现次数

在一个序列中，出现次数 $> \sqrt{n}$ 的数的个数不超过 $\sqrt{n}$，而其他数的出现次数都不超过 $\sqrt{n}$。这也可以说明：不同的出现次数只有 $\leq 2\sqrt{n}$ 种。

例 8.24. 多次询问值为 x 的位置集合和值为 y 的位置集合之间的最小距离。

解答. 按照每个值的出现次数是否 $> \sqrt{n}$ 分成两类。如果 x, y 出现次数都不超过 $\sqrt{n}$，那么可以归并计算。否则，通过预处理完成。

对每个出现次数 $> \sqrt{n}$ 的值 x，对所有 y 计算 $ans_{x,y}$。这可以对每个 x 正反扫两遍序列得到。

可以离线逐 x 计算来做到线性空间。

评注. 另一个做法是只预处理出现次数 $> \sqrt{n}$ 的 y。当 x, y 有一个出现次数不超过 $\sqrt{n}$ 时，暴力枚举那个值的所有出现，在另一个值的出现数组中二分即可。这样空间是线性的，但是时间带 log。可以使用 B 叉树将二分变成 $O(1)$，但是空间会增大。不知道在线能否做到 $n\mathrm{polylog}(n)$ 的空间。

此外，这个问题似乎被称为 *color-distance oracle*。

练习 8.25. 额外把位置集合限制到区间 $[l, r]$ 内。

8.3.4　字符串

有的问题给很多字符串，而它们的总长不超过 S。那么长度 $> \sqrt{S}$ 的字符串的个数不超过 $\sqrt{S}$，而其他字符串的长度都不超过 $\sqrt{S}$。

8.4　位运算

可以使用 word RAM 的特性——$O(1)$ 操作 $w = 64$ 个 bit 来做到很多奇妙的事情。

例 8.26. 常见操作.

- clz：查询前导零数量
- ctz：查询末尾有几个零
- popcount：查询 1 的数量
- lg：查询 $\lfloor \log_2 x \rfloor$

通常来说我们可以在 $V^{O(1)}$ 预处理一些东西之后 $O(1)$ 回答询问。

评注. 这里有一些更好的复杂度和其他讨论。不过，虽然常见的函数都是能快速计算的，但是绝大多数函数都需要至少 $\Omega(2^w/w)$ 大小的电路才能计算，这是**电路复杂性**的一个经典结论。

例 8.27. bitset. 在 $O(n/w)$ 时间内支持

- 计算两个集合的交/差/并
- 查询某个数的前驱后继

- 计算上面的几种常见操作

例 8.28. 维护 01 序列. 维护一个 01 序列，支持单点修改，查询区间与、或、异或和。做到 $O(\log n/\log\log n)$。

解答. w 叉树。

例 8.29. 维护数列，支持单点修改，查询区间和 $\bmod 2^k$。需要做到 $O(\frac{\log n}{\log(w/k)})$。此外，这个界是紧的。

解答. w/k 叉树。

例 8.30. $O(\log n/\log\log n)$ 支持：单点加一，查询区间和。

解答. 高低位分开，低位变成上一个题，高位变成修改次数只有 $m/2^b$ 的子问题。

例 8.31. 加法电路. 只使用位运算实现加法。

解答. 模拟二进制竖式加法。从低位到高位依次计算每一位的结果和进位。每一位的结果可以通过异或计算得到，进位可以通过与和或计算得到。

例 8.32. 并行加法器. 有 n 个数，需要统计每一位上总共有多少个 1。

解答. 分治计算每一位的答案。那么合并的时候需要做 w 个 $\log n$ bit 的加法，可以将加法电路并行来做到 $O(\log n)$ 而不是 $O(w)$。具体来说，常规的每位加法维护了 $w\times\log n$ 的 01 数组，这里则是维护转置后 $\log n\times w$ 的 01 数组。

练习 8.33. 并行乘法器. 在 $O(b^2)$ 的时间内计算 w 个 b 位数乘法

例 8.34. 第七学区. 求所有子区间按位或的和。

解答. 首先考虑一个拆位的 $O(n\log V)$ 做法。对于每一位，相当于有多少个区间包含 1，反过来就是求全 0 区间的个数。可以通过合并分治信息的方式计算。

合并分治信息可以通过并行加法/乘法器来实现。

9 树上问题初步

树是最简单的图。序列可以视为一种特殊的树——链。相比序列，树失去了全序性质，分治结构也变得更加复杂。因此，树上问题往往要比序列问题复杂得多。

9.1 树的基本性质

无根树是一个无向图，具有以下几个基本性质：

- 无环，连通
- 点数减边数等于 1
- 任意两点之间有且仅有一条路径

如果去除连通性要求，那么就得到森林。森林具有以下几个基本性质：

- 无向无环图
- 连通块数等于点数减边数
- 任意两点之间至多有一条路径

第二条性质常常用来计算森林的连通块数，或者判断某个子图是否是树。

例 9.1. ABC390F. 给一个序列 a，定义区间 $[l,r]$ 的权值为 $a[l,r]$ 这些值形成的值域连续段个数。求所有区间的权值和。

练习 9.2. 给一棵树和一个序列 a，定义区间 $[l,r]$ 的权值为 $a[l,r]$ 在树上形成的导出子图连通块个数，求所有区间的权值和。提示

练习 9.3. 给一棵树和一个序列 a，定义区间 $[l,r]$ 的权值为 $a[l,r]$ 在树上形成的导出子图连通块个数。多次询问某个区间的权值和。

练习 9.4. [SDCPC 2019. Connected Intervals] 给一棵树和一个序列 a，定义区间 $[l,r]$ 的权值为 $a[l,r]$ 在树上形成的导出子图连通块个数。求有多少个区间的权值为 1。

如果指定某一个点作为根，则得到有根树。有根树导出了一个偏序关系：父节点大于子节点。在这个偏序关系中，两个点可比（即 $x<y$ 或者 $x>y$）当且仅当它们之间有祖先后代关系。任意点集的公共祖先集合都是一条从根出发的链，其中深度最大的点称为它们的最近公共祖先（Lowest Common Ancestor，LCA）。有根树的深度定义为从根到该点的路径长度（有时也定义为路径上的点权和）。

9.1.1 树的子结构

树的常见子结构有以下几种：子树、路径/链、邻域。它们都是特殊的连通子图。

定义 9.5. 在有根树中，**子树**是某个结点及其所有后代导出的子图。无根树的子树则是一个有歧义的概念，既可以指一个连通块，也可以指不包含某个点的极大连通块。因此，我们不对无根树使用这个术语。
路径/链指的是某两个点之间的唯一路径。在有根树中，链可以分为直上直下的链和非直上直下的链两类。前者的性质通常与序列相似。任何一条链都可以由不超过两条直上直下的链拼起来得到。
点 u 的 k-**邻域**定义为距离 u 不超过 k 的点集。

给定点集 S，在树上包含它们的最小连通块是唯一的，这个连通块通常被称为**虚树**。

9.2　树的序列化

可以通过某种方式遍历树来得到一个序列。根据遍历方式的不同，得到的序列会有不同的性质。

9.2.1　DFS 序

DFS 序是最常用的遍历顺序，它的性质是子树内的点在 DFS 序中是一段连续的区间。因此，可以通过这种方式将子树问题转为区间问题。不过，子树问题通常要比区间问题更容易。结点在 DFS 序中的位置通常用时间戳 `dfn[u]` 表示。

9.2.2　BFS 序

树的 BFS 序是用得比较少的遍历顺序。它的性质是一个点的所有孩子在 BFS 序中连续。对 BFS 序的一种更灵活的理解方式是把树按深度分层。

例 9.6. 邻域修改查询. 给一棵树，支持把一个点的 k-邻域内的点权加，查询一个点的 k-邻域内的点权和。$k = O(1)$。

9.2.3　欧拉序

欧拉序是树按照一个“欧拉回路”访问点得到的顺序。在有根树中，可以这样得到一个欧拉序：

```
1 void dfs(int u) {
2     euler.push_back(u);
3     for (int v : son[u]) {
4         dfs(v);
5         euler.push_back(u);
6     }
7 }
```

性质 9.7. 在欧拉序中，取 u 和 v 的任意一对出现位置，则这两个位置之间的区间会包含 u, v 链上的所有点的至少一次出现，且不包含 $\mathrm{lca}(u, v)$ 子树之外的点。因此，这段区间中深度最小的点全都是 $\mathrm{lca}(u, v)$。

9.2.4　括号序

括号序是在 DFS 进入和离开结点 u 时分别写下 u^+ 和 u^- 得到的序列。设 $l = \mathrm{lca}(u, v)$。括号序的性质是：假设 u 在 v 前被访问到，则括号序列在 u^+（不含）和 v^+（含）之间的部分抵消掉成对的 w^+ 和 w^- 后，剩下的是 $[u, l)$ 这条链上的 w^- 和 $(l, v]$ 这条链上的 w^+。通过这种方式可以把一些链上问题变成序列上的问题。

例 9.8. 括号序树链莫队. 给一棵带颜色的树，多次询问一条链上有多少种出现次数为奇数的颜色。

评注. 完全的树链莫队需要树分块。

9.3　LCA 的计算

有根树上的各种计算往往都需要 LCA。这里给出 LCA 的若干计算方式。

9.3.1　向上跳

首先考虑一个暴力：将深度更大的那个点向上跳到与另一个点深度相同，然后在两个点相遇之前不断地一起往上跳。我们可以通过一些手段优化这个暴力：

- 倍增：预处理 $f_{i,j}$ 表示点 i 向上跳 2^j 步跳到的点即可。预处理 $O(n \log n)$，单次询问 $O(\log n)$，空间 $O(n \log n)$。
- 链剖分：在两个点跳到同一条链之前，每次让链顶深度更大的点向上跳到链顶的父亲；跳到同一条链之后返回深度小的点。通常使用重链剖分，此时预处理 $O(n)$，单次询问 $O(\log n)$，且常数较小。

评注. 这两种方法较难做到 $O(1)$ 询问。倍增法的空间可以通过斜二倍增优化到 $O(n)$。

9.3.2　RMQ

利用树的序列化可以将 LCA 问题变成 RMQ 问题。

- 欧拉序：直接使用欧拉序找到两点在欧拉序中第一次出现的区间，然后在这个区间内查询深度最小的点。
- DFS 序：设 $\mathrm{dfn}[u] < \mathrm{dfn}[v]$，则 dfn 在 $(\mathrm{dfn}[u], \mathrm{dfn}[v]]$ 中的点都在 $\mathrm{lca}(u, v)$ 的子树（不含本身）内，且其中必然包含 $\mathrm{lca}(u, v)$ 的一个儿子。因此，取 $\mathrm{dfn}[fa_x]$ 最小的 $\mathrm{fa_x}$ 即可。使用这种方法容易做到 $O(1)$ 询问。

9.4　树的子结构的性质

9.4.1　子树

在已知 S 的导出子图连通的情况下，可以进一步检查

$$\sum_{u\in S}\deg(u)=2|S|-1.$$

是否满足来判断 S 是否为一个非整棵树的子树。

删去一个点集 S 后，剩余图仍然连通，当且仅当 S 的每个连通分量都至多通过一条边与 $V\setminus S$ 相连。等价地，若保留下来的点集 $C=V\setminus S$ 非空，则 C 有唯一一个深度最小的点 r_C，且 C 中任意点到 r_C 的路径都完全包含在 C 中；这时 S 可以看成 $V\setminus\mathrm{subtree}(r_C)$ 加上若干个从 C 上剪掉的完整子树。

练习 9.9. P8990.

子树之间要么包含，要么不交。反过来，具有这种性质的集合族也可以构建为一棵树。

9.4.2　路径/链

练习 9.10. 判断点是否在链上. 给定点 u,v,x，判断 x 是否在 u 到 v 的链上。

解答. 方法一：检查 $d(u,x)+d(x,v)=d(u,v)$。

方法二：将链分解为两条直上直下的链上分别判断。直上直下的链上的判定可以转为子树判定，这可以通过 DFS 序实现。

性质 9.11. 两条链的交要么是空集，要么仍然是一条链。具体来说，可以这样计算 (u_1,v_1) 和 (u_2,v_2) 的交：考虑 $\mathrm{lca}(u_1,u_2),\mathrm{lca}(u_1,v_2),\mathrm{lca}(v_1,u_2),\mathrm{lca}(v_1,v_2)$ 这四个点中按重数取深度最大的两个点 u,v（可能相同），那么链交要么是 (u,v)，要么是空集，可以通过判断 u 是否同时在 (u_1,v_1) 和 (u_2,v_2) 上来区分。

9.4.3　虚树

点集 S 的虚树有一个紧凑的刻画方式。

性质 9.12. 固定点集 S。树上的一个点 u 为关键点当且仅当 $u\in S$，或者 u 有至少两个儿子的子树与 S 有交。一个关键点的父关键点定义为其祖先中深度最大的关键点。则 S 的虚树是所有关键点和其父关键点之间的链的边不交并。把每条这样的链压成一条边，就得到通常意义下的虚树。

练习 9.13. 令 $\bar{S}=\{\mathrm{lca}(u,v):u,v\in S\}$，说明关键点集合即为 $\bar{S}$。

练习 9.14. 证明关键点集合的大小不超过 $2|S|-1$。

例 9.15. 树上 TSP. 给一棵树和一个点集 S，选择 S 的一个排列 $u_1, \ldots, u_m$，分别使得

- $d(u_1, u_m) + \sum_{i=2}^{m} d(u_{i-1}, u_i)$ 最小
- $\sum_{i=2}^{m} d(u_{i-1}, u_i)$ 最小

例 9.16. 链并. 给一棵树和树上的 m 条链，将这些链的并集表示为 $O(m)$ 条不交的直上直下的链的并。

9.5　树上差分与前缀和

对于可差分的问题，可以将链用两类性质较好的集合表示：从根出发的链，以及子树。这两者互为对偶。

一条链可以表示为两条直上直下的链的不交并（且上端点大致相同），而直上直下的链又可以表示成两条从根出发的链的差。通过这种方式，我们可以把任意一条链表示为至多四条从根出发的链的加减。

记 l_v 为从根到 v 的链，s_u 为 u 的子树，则 $u \in l_v$ 当且仅当 $v \in s_u$。通过这种方式，可以在这两种范围之间做对偶。例如，链加、查询单点可以对偶到单点加、查询子树和。

在 $\{l_u\}$ 结构上存在自然的前缀和与差分算子：$Sf(u) = Sf(\mathrm{fa}_u) + f(u)$，$Df(u) = f(u) - f(\mathrm{fa}(u))$，容易验证它们互逆。在 $\{s_u\}$ 结构上的前缀和与差分算子则更加复杂一些：$Sf(u) = f(u) + \sum_{v \in \mathrm{son}_u} Sf(v)$，$Df(u) = f(u) - \sum_{v \in \mathrm{son}_u} f(v)$。我们在组合数学课程中学习的前缀和与差分技巧也可以在这里使用。

练习 9.17. 应用一. 给定一个 LCA 黑盒。在 $O(1)$ 时间内查询一条链的点权和以及边权和，或者某个子树的点权和以及边权和。

练习 9.18. 据此得到一个 $O(1)$ 查询两点间距离的算法。

例 9.19. 应用二. 给定一个 LCA 黑盒。在 $O(n+q)$ 时间内支持：一条链上的点权/边权加，某个子树内的点权/边权加，在所有修改结束后输出所有点权和边权。

例 9.20. P4211. 给一棵树，多次询问

$$\sum_{i=l}^{r} f(\mathrm{lca}(i, x)).$$

（假设有一个支持链修改查询的黑盒）

评注. 此题还有线段树套虚树、点分树等做法。

9.6　直径

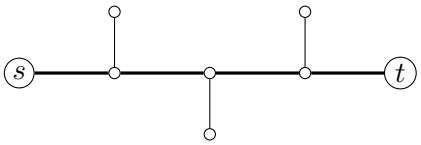
树的直径定义为距离最远的两点之间的距离。这里距离定义为两个点之间唯一路径上的边权和/点权和/边权和与点权和的和。我们也把这样的两个点之间的路径称为直径。直径可

能存在多条。

评注. 在有负权时，这样定义的距离的性质通常比较差。如无特殊说明，我们默认树无负权。

评注. 在树退化成一条链的情况下，求直径相当于求最大子段和。这可以说明一些树上问题的困难性，如链加、查询直径。

在确定直径后，可以将树画成以下的形式来帮助直观思考：



有以下性质成立：

性质 9.21. 假设树无负权。任取一条直径 (s,t)，则对于任何一个点 u，s,t 中至少有一个是距离 u 最远的点之一。换句话说，距离一个点最远的点一定可以是直径的一个端点。

练习 9.22.

- 说明在有负权时不成立。
- 在边权和点权均非负，且距离 $d(u,v)$ 定义为路径上边权和加上 u,v 两点的点权和时，证明该性质仍然成立。

练习 9.23. 据此得到无负权树的直径的算法。

练习 9.24. 在有负权时，使用 DP 来计算直径。

下面的性质给出了连接两棵树后直径的变化。

性质 9.25. 假设树 T_1 的一条直径是 (s_1,t_1)，树 T_2 的一条直径是 (s_2,t_2)，并且边无负权。如果在两棵树中间加一条边得到一棵新树，那么新树中存在一条两个端点都在集合 $\{s_1,t_1,s_2,t_2\}$ 内的直径。

练习 9.26. 说明如何加边才能使得新树的直径最大/最小。

练习 9.27. 支持加边（保证任意时刻是森林）、查询某个连通块的直径。

练习 9.28. P3596. 给一棵树，对每条边，计算删除这条边然后再加一条新边后形成的树的直径可能的最大值和最小值。

对于边权为正的树，还有以下结论：

性质 9.29. 所有直径的中点（允许在边上）都相同。这个点称为树的中心（center），它最小化到树上所有点的最大距离。

评注. 对于有零边的情况，可以把零边缩起来。

练习 9.30. P1099/P2491. 给定一棵非负权的树 T 和一个常数 s，求一对点 (u,v) 使得 $d(u,v)\le s$ 且 $\max_x \min_{y\in P_{uv}} d(x,y)$ 最小。也就是找到一条长度不超过 s 的路径，使得到这条路径的最远距离最小。需要 $O(n)$。

练习 9.31. P3304. 询问有多少条边满足所有的直径都经过它。

我们也可以定义一个点集内的直径。对于点集 S，定义 S 内的直径为 S 中距离最远的两点之间的距离。此时，有以下性质成立。

性质 9.32. 假设树无负权。任取点集 S 内的一条直径 (s,t)，则对于树上的任何一个点 u，s,t 中至少有一个是 S 内距离 u 最远的点之一。

练习 9.33. 证明以下结论：任取点集 S_1 内的一条直径 (s_1,t_1) 和点集 S_2 内的一条直径 (s_2,t_2)，则 $S_1\cup S_2$ 内存在一条两个端点都在 $\{s_1,t_1,s_2,t_2\}$ 内的直径。因此，点集内的直径是一种可以快速合并的分治信息。

练习 9.34. 多次询问 $S=[l,r]$ 内的直径。

练习 9.35. 给一棵树，维护一个点集 S，支持添加/删除点到 S 中，查询 S 内的直径。

评注. *有负权时可以通过动态 DP 之类的方法做到 $O(\log n)$，参见 P4115。*

例 9.36. CF1192B. 修改边权，查询直径。保证边权非负。

评注. *如果没有边权非负的保证，那么可以用静态 Top Tree 等方法做到 $O(\log n)$。*

9.7　重心

假设树 T 带有非负的点权，且总点权大于零。定义一个点集 S 的重量 $w(S)$ 为该点集内的点权和。树的重心（median/centroid）有以下两个等价定义：

- 以重心 u 为根时，$\max_{v\in \mathrm{son}_u} w(\mathrm{subtree}(v))$（最大的子树重量）最小。
- 以重心 u 为根时，任何 $v\neq u$ 都满足 $w(\mathrm{subtree}(v))\le w(T)/2$（每个子树的重量都不超过总重量的一半）。

评注. *注意重心未必在直径上。*

直观上，重心可以这样取得：假设当前在点 u，考虑删掉 u 形成的所有连通块，如果存在重量 $>w(T)/2$ 的那么就往这个连通块移动，否则 u 本身就是重心。利用这个直观，我们可以得到以下性质：

性质 9.37.

- 对一棵以 r 为根的树，若 r 不是重心，则存在唯一一个重量 $>w(T)/2$ 的儿子子树，并且所有重心都在这棵子树中。
- 假设 u 是树的重心，并把树以 u 为根。那么树除了 u 之外的所有重心，恰好是满足

$w(\text{subtree}(v)) = w(T)/2$ 的点 v。在点权均为正的情况下，这样的点 v 至多只有一个，故此时重心至多有两个且二者相邻。

对于一棵以 r 为根的有根树，在切换到以 u 为根时，除了原本的子树外，还会产生一个“向上”的子树，这个子树的重量为 $w(T) - w(\text{subtree}(u))$。可以据此判定一个点是否为树的重心。

练习 9.38. P5666. 给一棵树，计算删除每条边后，形成的两个连通块中的所有重心的编号之和。

下面的性质将重心与距离联系在一起。

性质 9.39. 假设树的边权非负，距离定义为两点之间的路径的边权和。树的重心可以最小化所有点到它的距离之和，即 $\sum_v w_v \cdot d(v,u)$。

也可以据此定义点集内的重心：

定义 9.40. 对于点集 S，定义 S 内的重心为距离 S 内所有点的距离之和最小的 S 内的点，即 $\arg\min_{u\in S} \sum_{v\in S} w_v \cdot d(u,v)$。这里距离定义为两点之间路径上的边数（事实上任何非负边权导出的距离都可行）。

下面的性质说明了连接两棵树后重心的变化情况。

性质 9.41. 假设树 T_1, T_2 的点权均为正，且 u_1, u_2 分别是它们的一个重心。如果在两棵树中间加一条边得到一棵新树，那么新树的所有重心都在 u_1 和 u_2 之间的链上。

例 9.42. P4299. 加边（保证任意时刻是森林），查询连通块重心。

下面的性质给出了重心与 DFS 序的联系。

性质 9.43. DFS 序的带权中点定义为满足 $\sum_{i<k} w_{p_i} \le \frac{W}{2}$ 且 $\sum_{i>k}^{n} w_{p_i} \le \frac{W}{2}$ 的 k。这样的 p_k 在深度最小的那个重心的子树内。

练习 9.44. 子树点权加/赋值，查询重心。

评注. 后面我们学习链剖分之后还能支持链修改。

练习 9.45. 查询 $S = [l, r]$ 内的重心。

解答. 通过可持久化线段树/整体二分之类的方法求出 S 在 DFS 序中的带权中点，然后从那个点向上跳到最后一个子树内有至多 $w(S)/2$ 重量的点即可。

9.8　链剖分

链剖分把树的**点集**划分为若干条直上直下的链，两个端点在同一条链中的边称为链内边/重边/实边，不在同一条链中的边称为链间边/轻边/虚边。也可以用另一种方式理解这个链划分：每个结点选择至多一个儿子作为特殊儿子，对应的边为链内边，其它边为链间边。根据选择儿子的方式不同，可以得到以下几种链剖分：

注意这些边并没有一个公认的命名。

- 重链剖分：每个结点选择子树大小最大的儿子作为特殊儿子，称为重儿子。此时，链内边称为重边，其它边称为轻边，划分出的链称为重链。
- 长链剖分：每个结点选择子树深度最大（指从根到最深叶子的长度）的儿子作为特殊儿子，称为长儿子/重儿子。此时，链内边称为长边/重边，其它边称为短边/轻边，划分出的链称为长链。
- LCT 中使用的动态链剖分：每个结点选择任意一个儿子作为特殊儿子，称为实儿子。此时，链内边称为实边，其它边称为虚边，划分出的链称为实链。

由于绝大多数情况下使用到的链剖分都是重链剖分，人们常常也会把重链剖分的术语用到其他链剖分上。

在下文中，我们称链剖分中的一条链为**实链**，而链则用于指代普通的树上路径。son_u 表示结点 u 的特殊儿子，child_u 表示结点 u 的所有儿子组成的集合。一条实链中最浅的结点称为这条实链的链顶，u 所在实链的链顶记作 $\mathrm{top}[u]$。

链剖分的基本性质是每条链间边的较深结点都是一条实链的链顶；反过来，一条实链的链顶要么是根，要么是某条链间边的较深结点。不过，需要注意链间边的较浅结点未必是一条实链的链底。根据这个性质，有以下结果成立：

$$\sum_u \sum_{v\in \mathrm{child}_u\setminus\{\mathrm{son}_u\}} v \text{ 到所在实链底的距离} \le \sum_{\text{链}C} |C| = n.$$

这是分析复杂度时常用的一个结论。

练习 9.46. 令 l_u 为 u 到其子树内最深叶子的长度，即长链剖分的划分依据。在长链剖分中，证明

$$\sum_u \sum_{v\in \mathrm{child}_u\setminus\{\mathrm{son}_u\}} l_v \le n.$$

这是长链剖分优化 DP 的一个依据。

链剖分的一个简单应用是寻找同一条实链上某个深度的点。

例 9.47. 给定一个链剖分，$O(1)$ 回答从 u 出发沿着其所在链向上/下走 k 步到达的点，或者报告离开了所在链。

练习 9.48. k 级祖先. 给一棵有根树，多次询问一个点的 k 级祖先。离线 $O(n)-O(1)$，在线 $O(n\log n)-O(1)$。

练习 9.49. 给一棵有根树，儿子有序排列。多次询问从某个点出发，每次都走最小的儿子，走 k 步后到达的点。保证询问点 u 子树内任何叶子到 u 的距离都至少为 k

任何一条直上直下的链都可以自下而上地分解为如下结构：（实链前缀、链间边）重复若干次，再加上一个实链的一段。如果是到根的链，则最后一段也是一个实链前缀。我们可以

对每条实链维护区间数据结构，从而支持一些链信息查询。我们也可以利用实链快速上跳来计算 LCA。

算法 9.50. 链剖分计算 LCA. 在 u,v 处于同一条实链前，反复执行以下过程：每次从 u,v 中选择链顶深度较大的一个，不妨设为 u，将 u 跳到 $fa[\mathrm{top}[u]]$。对于同一条实链上的 u,v，LCA 即为深度较小的那个。复杂度为 u,v 路径上链间边的个数。

练习 9.51. 对于线段树可以直接 $O(\log n)$ 维护的静态区间信息查询，设计一个 $O(u,v$ 路径上的链间边个数 $+\log n)$ 的算法来支持树路径上的相同信息查询。

练习 9.52. 对于线段树可以直接 $O(\log n)$ 维护的带修区间信息查询，设计一个 $O(u,v$ 路径上的链间边个数 $\log n)$ 的算法来支持树路径上的相同修改查询。

评注. 链剖分 + 线段树并不是一个完全平衡的结构，这导致复杂度有一个额外的 log 因子。可以使用全局平衡二叉树来做到 $O(\log n)$ 修改查询。

显然，链剖分的复杂度分析依赖于链间边的个数。不同的链剖分方式会导致不同的复杂度。

引理 9.53. 在重链剖分中，任意根到点的路径上经过的轻边数不超过 $\log_2 n$。

证明. 从上往下每经过一条轻边，子树大小至少翻倍。 □

练习 9.54. 证明长链剖分中，任意根到点的路径上经过的短边数为 $O(\sqrt{n})$。

可以配合 DFS 序来同时支持维护子树信息。在 DFS 时，我们优先递归 son_u，这样就能保证一条重链在 DFS 序中是连续的。按照这种 DFS 序构建的线段树能够同时支持重链和子树的修改查询。我们也可以设计其他的结点排序方式来支持更复杂的询问。

例 9.55. 毛毛虫剖分. 对于路径 P_{uv}，$\{x : d(x,P_{uv}) \le 1\}$ 称为 u,v 的毛毛虫。构造一个结点排序方式，使得子树可以划分为 $O(1)$ 个连续区间，且一个毛毛虫可以划分为 $O(\log n)$ 个连续区间。

解答. 重链剖分，然后按照如下顺序构造顺序：写下所有儿子，递归重儿子，然后递归所有轻儿子。这样，对于每条重链 C，设其自上而下为 $u_1,\cdots,u_k$，则递归到 u_1 时会连续写下 $\mathrm{child}_{u_1},\mathrm{child}_{u_2},\cdots$。也就是说一条重链对应的毛毛虫中，除了链顶之外都是连续的。同理，对于子树，除了子树根以外都是连续的。

练习 9.56. k-毛毛虫剖分. 对于路径 P_{uv}，$\{x : d(x,P_{uv}) \le k\}$ 称为 u,v 的 k-毛毛虫。构造一个结点排序方式，使得子树可以划分为 $O(k)$ 个连续区间，且一个 k-毛毛虫可以划分为 $O(k\log n)$ 个连续区间。